

INSTITUT D'ENSEIGNEMENT À DISTANCE — UNIVERSITÉ PARIS 8

LICENCE 3 INFORMATIQUE

Interprétation & Compilation — Chapitre 10 : Projet Final

Conception et implémentation de l'interpréteur Ilang

*Un mini-langage impératif réalisé avec Flex et Bison,
de l'analyse lexicale à la génération de code*

AIT BENAISSA Ilias

N° étudiant : 82508880

Enseignant : Philippe Kislin-Duval

29 mai 2026

Résumé

Ce rapport présente la conception et l'implémentation complète d'un interpréteur pour ILANG, un mini-langage de programmation impératif conçu dans le cadre du projet final du cours d'Interprétation & Compilation. Le langage supporte les variables, les expressions arithmétiques avec priorité des opérateurs, les structures de contrôle (conditions, boucles), les fonctions utilisateur récursives et les entrées-sorties. Sa réalisation réinvestit l'ensemble de la chaîne de compilation étudiée en cours : analyse lexicale par automate fini (Flex), analyse syntaxique par grammaire LALR(1) (Bison), construction d'un arbre syntaxique abstrait (AST), optimisations sur cet arbre (pliage de constantes, élimination du code mort), puis évaluation récursive.

Au-delà du cahier des charges, le projet a été prolongé par trois extensions (nombres flottants, instructions **arrêter/continuer**, localisation des erreurs à la ligne) et par une exploration hors-périmètre particulièrement formatrice : un *second back-end* qui, à partir du même AST, génère du code assembleur i386 32 bits compilable avec `gcc -m32`. Cette dualité interpréteur/compilateur illustre concrètement le principe d'un front-end commun suivi de back-ends interchangeable.

L'implémentation, écrite en C, compile sans aucun avertissement ni conflit grammatical et ne présente aucune fuite mémoire (vérifié avec Valgrind). Une campagne de tests systématique, couvrant les programmes de démonstration, la gestion des erreurs, les cas limites et les mesures de performance, valide le bon fonctionnement et met en évidence les forces et les limites de l'approche, notamment la profondeur de récursion bornée par la pile de l'hôte et l'écart de performance considérable entre l'interprétation et le code natif généré.

Mots-clés : interpréteur, compilation, Flex, Bison, LALR(1), arbre syntaxique abstrait, table des symboles, pliage de constantes, génération de code, assembleur i386.

Table des matières

Résumé	1
1 Introduction et contexte	9
1.1 Contexte et problématique	9
1.1.1 Contexte pédagogique et scientifique	9
1.1.2 Identification du problème	10
1.1.3 Questions directrices	10
1.2 Objectifs et contributions	10
1.2.1 Objectifs	10
1.2.2 Contributions	11
1.2.3 Cadrage du projet par les cinq questions	11
1.2.4 Budget, temps et coûts	11
1.3 Réinvestissement des acquis du cours	12
1.4 Structure du document	12
2 État de l’art	13
2.1 Revue de littérature	13
2.1.1 Travaux fondateurs	13
2.1.2 Des outils générateurs plutôt que du code écrit à la main	14
2.1.3 Les stratégies d’exécution	14
2.1.4 Analyse comparative	15
2.1.5 Les familles d’outils d’analyse syntaxique	15
2.2 Positionnement	16
2.2.1 Lacunes de l’approche antérieure	16
2.2.2 Opportunités et justification de mon approche	16
3 Analyse et conception	18
3.1 Analyse des besoins	18
3.1.1 Besoins fonctionnels	18
3.1.2 Besoins non-fonctionnels	18
3.1.3 Cas d’utilisation	19
3.2 Conception architecturale	20
3.2.1 La chaîne de traitement	20

3.2.2	Patrons de conception (<i>patterns</i>)	20
3.2.3	Choix du langage d'implémentation	20
3.2.4	Justification des principales décisions de conception	21
3.3	Méthodologie et planification	22
3.3.1	Méthodologie retenue	22
3.3.2	Identification des tâches et planning	22
3.4	Conception détaillée	22
3.4.1	La grammaire (BNF)	22
3.4.2	Structure de l'arbre syntaxique abstrait	23
3.4.3	La table des symboles	23
3.4.4	Algorithmes clés	24
4	Implémentation	27
4.1	Environnement technique	27
4.2	Architecture logicielle	27
4.2.1	Organisation du code	27
4.2.2	Interfaces entre modules	27
4.3	Le cœur : du texte à l'exécution	28
4.3.1	Analyse lexicale (Flex)	28
4.3.2	Analyse syntaxique et construction de l'AST (Bison)	29
4.3.3	L'évaluateur	31
4.4	Aspects techniques notables	32
4.4.1	Le type <code>Value</code> : faire cohabiter entiers et flottants	32
4.4.2	La portée lexicale	32
4.4.3	Localisation des erreurs à la ligne	33
4.4.4	Les optimisations et le garde-fou de la division par zéro	33
4.4.5	Gestion de la mémoire	34
4.4.6	Sûreté et robustesse	34
4.4.7	Le back-end assembleur (extension hors-périmètre)	34
4.4.8	Compilation automatisée	36
5	Validation et évaluation	37
5.1	Stratégie de test	37
5.2	Protocole expérimental	37
5.2.1	Conditions et métriques	37
5.2.2	Jeux d'essai	37
5.2.3	Reproductibilité	38
5.3	Résultats fonctionnels	38
5.3.1	Programmes de démonstration	38
5.3.2	Gestion des erreurs	39
5.4	Résultats de performance	40
5.4.1	Interprétation contre code natif	40
5.4.2	Effet des optimisations	40

5.4.3	Limite de profondeur de récursion	40
5.4.4	Vérification mémoire	41
5.4.5	Équivalence interpréteur / code généré	41
5.5	Étude de cas de bout en bout	41
6	Discussion	43
6.1	Analyse critique	43
6.1.1	Forces	43
6.1.2	Faiblesses et limitations	43
6.1.3	Comparaison avec l'existant	44
6.2	Difficultés rencontrées et solutions	44
6.2.1	Le « dangling else » qui n'existait pas	44
6.2.2	L'ordre d'évaluation et la portée	45
6.2.3	Le piège du pliage de la division par zéro	46
6.2.4	Détails d'environnement et de robustesse	46
6.3	L'extension : du parcours d'arbre au code natif	47
6.3.1	Modèle et choix	47
6.3.2	Obstacles spécifiques	47
6.3.3	Enseignement	47
6.4	Compromis effectués	47
7	Conclusion et perspectives	49
7.1	Synthèse des contributions	49
7.2	Prolongements possibles	49
7.3	Applications potentielles	50
7.4	Recommandations	51
7.5	Apports personnels et conclusion	51
	Bibliographie et webographie	52
A	Code source complet	54
A.1	ast.h	54
A.2	ast.c	57
A.3	syntab.h	62
A.4	syntab.c	63
A.5	ilang.l : analyseur lexical (Flex)	67
A.6	ilang.y : analyseur syntaxique (Bison)	69
A.7	eval.c : évaluateur	73
A.8	optim.c : optimisations	80
A.9	codegen.c : back-end assembleur (extension)	84
A.10	main.c : point d'entrée	92
A.11	Makefile	94
A.12	Programmes de démonstration	96
A.12.1	fibonacci.ilang	96

A.12.2	pgcd.ilang	96
A.12.3	interactif.ilang	96
A.12.4	flottants.ilang (extension)	97
A.12.5	boucles.ilang (extension)	97
B	Manuel d’installation	99
B.1	Prérequis	99
B.2	Compilation	99
C	Guide utilisateur	100
C.1	Invocation	100
C.2	Aide-mémoire du langage	100
D	Données expérimentales	101
E	Assembleur i386 généré (exemples)	102
E.1	PGCD (boucle <code>tant que</code>)	102
E.2	Factorielle (fonction récursive)	103
F	Contenu de l’archive	106
G	Glossaire	108

Table des figures

3.1	Diagramme des cas d'utilisation.	19
3.2	Chaîne de traitement d'ILANG. Le front-end (Flex, Bison) est commun ; seul le back-end diffère entre l'interprétation et la génération de code.	20
3.3	Diagramme de Gantt du déroulement du projet.	25
3.4	Structure des nœuds de l'AST (vue UML simplifiée).	26
3.5	AST de l'expression $2 * 3 + 4$. La priorité de $*$ sur $+$ se traduit par la position des nœuds dans l'arbre.	26
3.6	Pile d'environnements pendant l'évaluation de <code>fib(3)</code> . Le lien <i>parent</i> sert à restaurer le frame courant en sortie de fonction ; la résolution des variables, elle, est lexicale (frame courant puis global).	26
4.1	Automate fini (simplifié) reconnaissant les entiers (N) et les identifiants (I). Les états doublés sont acceptants.	28
4.2	AST de l'instruction <code>x = 2 * 3 + 4</code>	30
4.3	Pliage de constantes : l'arbre de $2 + 3 * 4$ est réduit à la feuille 14.	34
5.1	Croissance du temps d'exécution de Fibonacci. La courbe de l'interpréteur révèle bien la complexité exponentielle de l'algorithme ; le code natif, sur la même échelle, reste collé à l'axe.	40
5.2	Temps de la boucle de 30 M d'itérations, avec et sans pliage de constantes. Le gain ($\approx 33\%$) provient de la suppression du calcul arithmétique répété à chaque tour.	41

Listings

3.1	Exemple : factorielle récursive et boucle d’affichage.	19
3.2	Grammaire BNF d’ILANG.	22
3.3	Principe de l’évaluateur (pseudo-code).	24
4.1	Extrait de <code>ilang.l</code> : littéraux numériques.	28
4.2	Gestion des commentaires de bloc par état exclusif (extrait de <code>ilang.l</code>).	29
4.3	Action sémantique d’une règle (extrait de <code>ilang.y</code>).	29
4.4	Union sémantique et déclaration des tokens (extrait de <code>ilang.y</code>).	29
4.5	Déclarations de priorité (extrait de <code>ilang.y</code>).	30
4.6	Appel de fonction : ordre d’évaluation et pile (extrait de <code>eval.c</code>).	31
4.7	Cohabitation entiers/flottants et garde-fou (extrait de <code>eval.c</code>).	31
4.8	Le type <code>Value</code> (extrait de <code>ast.h</code>).	32
4.9	Fonction de hachage <code>djb2</code> (extrait de <code>symtab.c</code>).	32
4.10	Garde-fou du pliage (extrait de <code>optim.c</code>).	33
4.11	Génération d’un binop en machine à pile (extrait de <code>codegen.c</code>).	35
4.12	Assembleur généré pour <code>carre</code> (annoté).	35
4.13	Schéma de traduction d’une boucle <code>tant que</code>	36
4.14	Séparation des jeux d’options (extrait du <code>Makefile</code>).	36
5.1	Cœur du script de test des erreurs.	38
5.2	Sortie de <code>fibonacci.ilang</code> (15 premiers termes).	38
5.3	Sortie de <code>pgcd.ilang</code> : <code>pgcd(48, 18)</code>	38
5.4	Sortie de <code>interactif.ilang</code> pour les entrées 7 puis <code>-5</code>	38
5.5	Sortie de <code>flottants.ilang</code>	38
5.6	Sortie de <code>boucles.ilang</code> (arrêter puis continuer puis arrêter dans un <code>tant que</code>).	39
5.7	Sorties réelles sur des programmes fautifs.	39
5.8	Verdict Valgrind (programme <code>pgcd.ilang</code>).	41
6.1	Les deux règles du <code>si</code> , sans aucune directive de priorité.	44
6.2	L’ordre correct : évaluer les arguments avant d’empiler le cadre.	45
6.3	Cas qui révèle la portée dynamique (corrigé depuis).	45
6.4	Résolution lexicale corrigée (extrait de <code>symtab.c</code>).	45
6.5	Le pliage n’est tenté que si l’opération est sûre (extrait de <code>optim.c</code>).	46
6.6	Avant et après, dans <code>ilang.l</code>	47
7.1	Esquisse d’extension de <code>Value</code> aux chaînes.	49
7.2	Esquisse d’un jeu d’instructions de bytecode.	50

../ast.h	54
../ast.c	57
../syntab.h	62
../syntab.c	63
../ilang.l	67
../ilang.y	69
../eval.c	73
../optim.c	80
../codegen.c	84
../main.c	92
../Makefile	94
../programmes/fibonacci.ilang	96
../programmes/pgcd.ilang	96
../programmes/interactif.ilang	97
../programmes/flottants.ilang	97
../programmes/boucles.ilang	97
generated/pgcd.s	102
generated/factorielle.s	103

Chapitre 1

Introduction et contexte

1.1 Contexte et problématique

1.1.1 Contexte pédagogique et scientifique

Ce projet constitue l'évaluation finale du cours d'Interprétation et Compilation de la Licence 3 Informatique de l'IED, à l'Université Paris 8. Il occupe une place particulière dans la progression du semestre, car il ne porte pas sur une notion isolée mais demande au contraire de rassembler en un seul logiciel fonctionnel l'ensemble des concepts étudiés tout au long de l'année. Les chapitres précédents avaient abordé séparément l'assembleur et la pile d'appels, la chaîne de compilation, les automates finis, les expressions régulières, les grammaires, puis les optimisations. Le projet final consiste à montrer qu'on est capable de faire tenir tous ces éléments ensemble, de les articuler les uns aux autres, et d'obtenir au bout un programme qui exécute réellement du code écrit dans un langage que l'on a soi-même défini.

Il faut rappeler à quel point la traduction des langages occupe une position centrale en informatique. Aucun programme écrit dans un langage de haut niveau n'est directement compréhensible par le processeur, qui ne sait exécuter que des instructions machine élémentaires. Il existe donc toujours, quelque part, un logiciel chargé de faire le pont entre ce que le programmeur écrit et ce que la machine exécute. Deux grandes familles de stratégies remplissent ce rôle. La compilation traduit le programme une fois pour toutes en code machine, qui sera ensuite exécuté de façon autonome. L'interprétation, à l'inverse, exécute le programme directement, en parcourant une représentation interne, sans produire de binaire séparé. Ces deux familles, malgré leur différence de finalité, partagent exactement la même première moitié de traitement : analyser le texte source, le décomposer en unités lexicales, vérifier sa conformité à une grammaire, et le représenter sous une forme structurée. Elles ne divergent qu'ensuite, sur la manière d'exploiter cette représentation. C'est précisément cette architecture en deux temps, un tronc commun suivi de traitements spécialisés, que mon projet met en évidence, d'autant plus nettement qu'il en propose deux exploitations différentes du même tronc.

1.1.2 Identification du problème

Le problème posé peut s'énoncer simplement : concevoir un langage de programmation complet, si modeste soit-il, et écrire le logiciel capable de l'exécuter. Le mot « complet » mérite une précision. Il ne signifie pas riche en fonctionnalités, mais suffisant pour exprimer n'importe quel calcul, c'est-à-dire Turing-complet. En pratique, cela impose au minimum de disposer de variables pour mémoriser des valeurs, de conditionnelles pour choisir entre plusieurs comportements, de boucles pour répéter des traitements, et d'un mécanisme de composition, ici les fonctions, pour structurer et réutiliser le code.

Concevoir un tel langage ne se réduit pas à écrire du code C. C'est un travail préalable de définition qui porte sur trois aspects indissociables, et c'est seulement une fois ces trois aspects fixés que l'écriture de l'interpréteur prend son sens. Le premier aspect est la syntaxe concrète, c'est-à-dire l'apparence qu'aura un programme valide : quels mots-clés, quels opérateurs, quelle ponctuation. Le deuxième aspect est la grammaire, c'est-à-dire la définition précise des suites de symboles acceptées et de la manière dont elles s'organisent en structures imbriquées. Le troisième aspect, enfin, est la sémantique, c'est-à-dire la signification de chaque construction, ce que fait réellement le programme lorsqu'on l'exécute. La syntaxe relève de Flex, la grammaire de Bison, et la sémantique de l'évaluateur que j'ai écrit.

1.1.3 Questions directrices

Plutôt que d'aborder le projet comme une simple commande à satisfaire, j'ai préféré l'organiser autour de quelques questions concrètes d'ingénierie, qui ont guidé mes choix et qui structurent ce rapport. La première question concerne les outils : qu'apporte réellement le recours à Flex et Bison par rapport à une analyse écrite à la main, et ce gain justifie-t-il la courbe d'apprentissage de ces outils ? La deuxième question concerne la représentation interne : comment représenter un programme en mémoire de manière à pouvoir aussi bien l'évaluer, l'optimiser, voire le compiler, sans avoir à tout réécrire pour chaque usage ? La troisième question touche à la sémantique des fonctions : comment gérer correctement la portée des variables et la récursion, qui sont les points les plus délicats d'un langage à fonctions ? La dernière question, plus ouverte, est celle des limites : jusqu'où peut-on pousser un projet de ce type avant de buter sur des obstacles de fond, qu'il s'agisse de performance, de profondeur de récursion ou de typage ?

1.2 Objectifs et contributions

1.2.1 Objectifs

L'objectif principal est de livrer un interpréteur ILANG qui soit à la fois fonctionnel, robuste et fidèle au cahier des charges. Cet objectif se décompose en plusieurs sous-objectifs concrets. Il faut d'abord écrire un analyseur lexical qui reconnaisse toutes les unités lexicales du langage, des nombres aux mots-clés en passant par les commentaires. Il faut ensuite écrire une grammaire LALR(1) qui soit non ambiguë et qui construise au fil de l'analyse l'arbre syntaxique. Il faut implémenter un évaluateur capable de traiter les variables, les expressions, les structures de contrôle, les fonctions récursives et les entrées-sorties. Il faut mettre en place une table des

symboles correcte du point de vue de la portée. Il faut réaliser les deux passes d’optimisation demandées. Il faut traiter proprement les erreurs courantes en les localisant à la ligne. Enfin, et ce dernier point a compté autant que les autres à mes yeux, il faut atteindre un certain niveau de qualité d’ingénierie : une compilation sans le moindre avertissement, une grammaire sans conflit, et une exécution sans fuite mémoire.

1.2.2 Contributions

Au-delà de ce que demandait strictement la commande, mon travail apporte trois contributions que je revendique. La première est un ensemble de trois extensions du langage. J’ai ajouté un type flottant, conçu de façon à cohabiter avec les entiers sans compromettre la sémantique entière d’origine, ce qui était la vraie difficulté. J’ai ajouté les instructions de rupture de boucle **arrêter** et **continuer**. J’ai enfin systématisé la localisation des erreurs à la ligne. La deuxième contribution est un second back-end, qui génère du code assembleur i386 à partir du même arbre syntaxique. Je l’ai développé par curiosité, en sachant qu’il dépassait le périmètre du projet, mais il démontre concrètement que le front-end que j’avais écrit était réutilisable tel quel pour produire du code natif, ce qui valide a posteriori toute l’architecture. La troisième contribution est une démarche de validation que j’ai voulue sérieuse, comprenant des tests fonctionnels, des tests d’erreurs, une campagne de cas limites, une analyse de la mémoire et des mesures de performance comparant l’interprétation au code natif.

1.2.3 Cadrage du projet par les cinq questions

Pour cadrer clairement le travail, il est utile de répondre explicitement aux cinq questions classiques de la conduite de projet, souvent appelées les cinq W. La question du *quoi* a déjà été posée : il s’agit d’un interpréteur pour le mini-langage impératif ILANG, accompagné de ses programmes de démonstration et de sa documentation. La question du *pourquoi* renvoie à la finalité pédagogique : il s’agit de démontrer la maîtrise de la chaîne de compilation complète, ce qui est l’objectif assigné au projet final du chapitre 10. La question du *qui* est vite réglée, puisqu’il s’agit d’un travail individuel évalué, réalisé par un seul étudiant. La question du *quand* renvoie à la période allouée en fin de semestre, dont la répartition est présentée sous forme de diagramme de Gantt au chapitre 3. La question du *où*, enfin, désigne l’environnement de travail, à savoir un système GNU/Linux exécuté sous WSL, avec la chaîne d’outils GCC, Flex et Bison.

1.2.4 Budget, temps et coûts

S’agissant d’un projet académique individuel, la question financière se règle en une phrase : le coût direct est nul. Tous les outils que j’ai employés, du compilateur GCC aux générateurs Flex et Bison, en passant par Make, Valgrind et la distribution T_EX Live qui a servi à produire ce rapport, sont des logiciels libres et gratuits. La seule ressource réellement consommée est le temps. J’ai réparti cet effort de manière progressive, en commençant par la conception de la grammaire, puis l’écriture du cœur du langage, ensuite la table des symboles et les optimisations, puis une phase substantielle de débogage et de tests, et enfin les extensions avant la rédaction. Cette répartition est détaillée et justifiée au chapitre 3, où elle est présentée sous forme de planning.

1.3 Réinvestissement des acquis du cours

Le projet est explicitement conçu comme une synthèse, et il me semble important de rendre cette intention visible dès l'introduction. Le tableau 1.1 met en regard chaque chapitre du cours et la manière précise dont il se retrouve mobilisé dans ILANG. Cette mise en correspondance n'est pas un simple exercice de style. Elle constitue un fil conducteur que je m'efforce de suivre tout au long du rapport, en signalant, à chaque fois que c'est pertinent, à quelle notion du cours se rattache tel choix d'implémentation.

TABLE 1.1 – Réinvestissement des chapitres du cours dans le projet.

Chapitre du cours	Concept réinvesti dans Ilang
2, Assembleur	Pile d'appels et cadre <code>%ebp</code> , reproduits pour les fonctions, et générés tels quels par le back-end assembleur.
3, Programme C compilé	Niveaux d'optimisation et pliage de constantes.
4, Analyse lexicale et syntaxique	Analyseur lexical écrit avec Flex et parseur écrit avec Bison.
5, Yacc/Bison	Grammaire LALR(1) et résolution, ou plutôt constat d'absence, des conflits shift/reduce.
7, Génération de code	Arbre syntaxique abstrait, table des symboles, évaluation récursive.
8, Optimisations	Pliage de constantes et élimination du code mort sur l'arbre.
9, Automates et regex	Tokens reconnus par l'automate fini engendré par Flex.

1.4 Structure du document

Le rapport suit une progression que je veux la plus naturelle possible, allant du général au particulier puis à la prise de recul. Le chapitre 2 dresse un état de l'art des techniques d'interprétation et de compilation et explique comment mon travail se positionne par rapport à elles. Le chapitre 3 présente l'analyse des besoins, l'architecture que j'ai retenue et la conception détaillée, en particulier la grammaire, l'arbre syntaxique et les algorithmes principaux. Le chapitre 4 décrit l'implémentation proprement dite, module par module, avec de nombreux extraits de code commentés et des exemples concrets. Le chapitre 5 expose ma stratégie de test et les résultats obtenus, y compris les mesures de performance et une étude de cas de bout en bout. Le chapitre 6 prend du recul, analyse de façon critique les forces et les limites du projet, et revient en détail sur les difficultés que j'ai rencontrées. Le chapitre 7 conclut et ouvre des perspectives. Les annexes regroupent enfin l'intégralité du code source, les manuels d'installation et d'utilisation, l'assembleur réellement généré et les données expérimentales.

Chapitre 2

État de l’art

Concevoir un langage et l’outil capable de l’exécuter n’est pas une démarche isolée. C’est au contraire s’inscrire dans une tradition technique et théorique qui s’est constituée sur plus d’un demi-siècle, et qui a fini par dégager des méthodes et des outils devenus standards. Avant de présenter mes propres choix, il me paraît donc indispensable de situer le projet dans ce paysage : de rappeler d’où viennent les concepts que j’ai utilisés, quelles solutions existent pour chaque problème que j’ai eu à résoudre, et pourquoi j’ai retenu telle approche plutôt que telle autre. C’est l’objet de ce chapitre, organisé en une revue des travaux et des outils existants, suivie du positionnement de mon travail.

2.1 Revue de littérature

2.1.1 Travaux fondateurs

Tout ce projet repose, souvent sans qu’on y pense, sur des résultats théoriques établis entre les années 1950 et 1970. Le premier de ces apports est la classification des grammaires formelles proposée par Noam Chomsky en 1956 [6]. Chomsky a montré qu’on peut hiérarchiser les langages selon la puissance de la machine nécessaire pour les reconnaître. Deux niveaux de cette hiérarchie nous intéressent directement. Les langages dits réguliers, les plus simples, sont reconnus par des automates finis : ils suffisent à décrire la forme des mots du langage, c’est-à-dire les nombres, les identifiants ou les opérateurs. Les langages dits hors-contexte, plus expressifs, sont reconnus par des automates à pile : ils sont nécessaires pour décrire l’imbrication des parenthèses, des blocs et des expressions. Cette distinction n’est pas une subtilité académique. Elle se traduit très concrètement dans l’architecture de n’importe quel compilateur, et dans la mienne en particulier, par la séparation en deux étages bien distincts. L’analyse lexicale, confiée à Flex, traite le niveau régulier. L’analyse syntaxique, confiée à Bison, traite le niveau hors-contexte. J’ai trouvé éclairant de constater que cette frontière théorique correspond exactement à la frontière entre mes deux premiers fichiers source.

Le deuxième apport fondateur concerne l’analyse syntaxique elle-même. En 1965, Donald Knuth a décrit l’analyse LR [5], une méthode ascendante, déterministe et efficace, capable de reconnaître

une très large classe de grammaires hors-contexte. Le mot « ascendante » signifie que l’analyseur part des symboles élémentaires et reconstruit progressivement les structures englobantes, par opposition aux méthodes descendantes qui partent du but et le décomposent. Les variantes plus économes en mémoire de cette méthode, en particulier l’analyse LALR(1) qui n’utilise qu’un seul symbole de prévision, sont précisément celles que met en œuvre l’outil Bison. Autrement dit, lorsque j’écris une grammaire pour Bison, je m’appuie sans le réécrire sur un algorithme qui a près de soixante ans et qui fait toujours référence.

L’ensemble de cette chaîne, de l’analyse lexicale jusqu’à la génération de code en passant par les arbres syntaxiques et les optimisations, a été synthétisé et codifié dans un ouvrage devenu la référence absolue du domaine, communément appelé le « Dragon Book », d’Aho, Lam, Sethi et Ullman [1]. La structure même de mon projet, et donc de ce rapport, suit fidèlement le découpage proposé par cet ouvrage. Je m’y suis appuyé en particulier pour comprendre le rôle pivot de l’arbre syntaxique abstrait et la notion de table des symboles.

2.1.2 Des outils générateurs plutôt que du code écrit à la main

Sur le plan des outils, l’apport décisif date de 1975, aux laboratoires Bell, avec deux programmes qui ont changé la manière d’écrire les compilateurs. Le premier, `lex`, dû à Mike Lesk et Eric Schmidt [7], engendre un analyseur lexical à partir d’une simple liste d’expressions régulières. Le second, `yacc` pour *Yet Another Compiler-Compiler*, dû à Stephen Johnson [8], engendre un analyseur syntaxique à partir d’une grammaire. L’idée commune à ces deux outils est qu’on ne devrait pas écrire un analyseur à la main, instruction par instruction, mais le *décrire* de façon déclarative et laisser une machine produire le code correspondant. Cette idée s’est imposée durablement, et les réimplémentations libres de ces outils, Flex [3] pour l’analyse lexicale et Bison [4] pour l’analyse syntaxique, sont aujourd’hui omniprésentes. Ce sont elles que j’ai employées. L’ouvrage de John Levine [2] m’a servi de guide pratique pour leur utilisation conjointe, notamment pour comprendre comment le lexer et le parseur communiquent à travers la variable `yylval` et la table des tokens.

2.1.3 Les stratégies d’exécution

Une fois le programme analysé et représenté en mémoire, encore faut-il l’exécuter. Plusieurs stratégies existent, et le choix de l’une d’elles est probablement la décision d’architecture la plus structurante d’un tel projet. Je les présente ici de la plus simple à la plus performante, car cette progression correspond aussi à une montée en complexité de réalisation.

La première stratégie est l’interprétation par parcours d’arbre, souvent désignée par son nom anglais de *tree-walking*. Le principe consiste à parcourir récursivement l’arbre syntaxique et à évaluer chaque nœud au moment où on le rencontre. Un nœud d’addition évalue ses deux fils puis additionne les résultats, un nœud de condition évalue son test puis descend dans la bonne branche, et ainsi de suite. C’est l’approche la plus directe et, surtout, la plus lisible : la sémantique du langage se lit presque mot pour mot dans le code de l’évaluateur. Son inconvénient est la lenteur, car chaque exécution d’une instruction suppose de re-parcourir une portion de l’arbre et de déterminer à chaque nœud de quel type il s’agit.

La deuxième stratégie repose sur une machine virtuelle à bytecode. Le programme est d’abord traduit dans un jeu d’instructions compact et de bas niveau, le bytecode, qui est ensuite exécuté par une boucle d’interprétation dédiée. C’est l’approche retenue par des langages très répandus comme Python, Java ou Lua. Elle est sensiblement plus rapide que le parcours d’arbre, car le travail d’analyse des structures est fait une fois pour toutes au moment de la traduction, tout en restant portable d’une machine à l’autre.

La troisième stratégie, la compilation à la volée ou *JIT*, va plus loin en traduisant en code machine, pendant l’exécution, les portions de programme les plus sollicitées. C’est l’approche des moteurs JavaScript modernes ou de la machine virtuelle Java dans sa version optimisée. Elle offre d’excellentes performances au prix d’une complexité d’implémentation considérable. La quatrième et dernière stratégie est la compilation anticipée vers du code machine, telle que la pratiquent GCC ou Clang pour le langage C. Elle produit les binaires les plus rapides, mais elle suppose une étape de compilation séparée et un résultat lié à une architecture précise.

2.1.4 Analyse comparative

Le tableau 2.1 résume les compromis que présentent ces quatre stratégies. Il ne faut pas le lire comme un classement, car aucune approche n’est supérieure dans l’absolu : tout dépend de l’objectif poursuivi. Pour un produit industriel soucieux de performance, le bytecode ou la compilation s’imposent. Pour un projet dont la finalité est de *comprendre* et de *montrer* comment un langage fonctionne, c’est au contraire la lisibilité qui prime, et le parcours d’arbre devient le meilleur choix.

TABLE 2.1 – Comparaison des stratégies d’exécution.

Approche	Performance	Complexité	Lisibilité
Parcours d’arbre	faible	faible	excellente
Machine à bytecode	moyenne	moyenne	moyenne
Compilation JIT	élevée	élevée	faible
Compilation anticipée	très élevée	élevée	faible

Un point mérite d’être souligné, car il a orienté toute la suite du projet. ILANG illustre en réalité deux points opposés de cet axe. La version principale, l’interpréteur, relève du parcours d’arbre. L’extension que j’ai développée par curiosité, le générateur de code assembleur, relève de la compilation anticipée. Or les deux partent du *même* arbre syntaxique. Cette coïncidence n’en est pas une : elle matérialise le fait, central dans toute la théorie de la compilation, que l’analyse d’un programme est indépendante de ce qu’on décide d’en faire ensuite. C’est cette idée que j’ai trouvée la plus instructive de tout le projet, et j’y reviendrai longuement.

2.1.5 Les familles d’outils d’analyse syntaxique

Puisque le cœur de mon travail repose sur un générateur d’analyseur syntaxique, il est utile de rappeler les grandes familles d’outils disponibles, afin de mieux justifier ensuite le choix de Bison. La première famille regroupe les générateurs ascendants de type LR ou LALR, dont Yacc et Bison sont les représentants les plus connus. Ils construisent automatiquement une table

d'analyse à partir de la grammaire, reconnaissent une large classe de langages, et possèdent une vertu précieuse pour qui apprend : ils signalent les ambiguïtés de la grammaire sous forme de conflits. La deuxième famille regroupe les générateurs descendants de type LL, comme ANTLR ou JavaCC. Ils sont souvent jugés plus intuitifs à déboguer, car le déroulement de l'analyse suit visiblement la structure de la grammaire, mais ils sont restreints à une classe de grammaires plus étroite et ont longtemps été gênés par la récursion à gauche, qu'il faut éliminer à la main.

La troisième famille n'est pas un outil mais une technique : la descente récursive écrite directement à la main, sans générateur. Elle est souple, lisible pour de petits langages, et ne dépend d'aucun outil externe. Son défaut majeur, du point de vue de ce projet, est que la grammaire y reste implicite, dispersée dans le code, et que rien ne signale ses éventuelles ambiguïtés. La quatrième famille, plus récente, regroupe les grammaires d'expression d'analyse, dites PEG, et les combineurs d'analyseurs. Elles fusionnent souvent l'analyse lexicale et syntaxique et reposent sur un choix ordonné des règles, ce qui présente l'avantage de ne jamais produire d'ambiguïté, mais au prix de la masquer plutôt que de la révéler. Cette dernière propriété est exactement à l'opposé de ce que je recherchais : je voulais un outil qui me *dise* si ma grammaire était ambiguë, pas un outil qui décide silencieusement à ma place.

2.2 Positionnement

2.2.1 Lacunes de l'approche antérieure

Le cahier des charges fait explicitement référence à un exercice antérieur du cours, le mini-langage PARLOT, qui réalisait son analyse lexicale au moyen d'expressions régulières écrites en Python et son analyse syntaxique par une descente récursive codée à la main. Cette approche fonctionne, et elle a le mérite de la simplicité initiale, mais elle souffre de deux faiblesses que j'ai cherché à corriger. La première est que la grammaire n'y est formalisée nulle part. Elle est diluée dans le code impératif de la descente récursive, si bien qu'on ne peut ni la lire d'un seul tenant, ni raisonner sur ses propriétés, ni détecter ses ambiguïtés. La seconde faiblesse découle de la première : comme aucune description formelle n'existe, rien ne vérifie que tous les cas sont traités. C'est au programmeur seul d'y veiller, et la moindre ambiguïté passe inaperçue jusqu'à ce qu'elle provoque un comportement inattendu.

2.2.2 Opportunités et justification de mon approche

Mon projet corrige ces deux lacunes en adoptant les outils standards du cours. En décrivant la grammaire de façon déclarative dans le fichier Bison, j'obtiens deux bénéfices que je considère comme décisifs. Le premier est que la grammaire devient un objet explicite, écrit en notation BNF, que l'on peut lire, discuter, vérifier et faire évoluer indépendamment du reste du code. Le second, plus important encore, est que Bison détecte automatiquement les ambiguïtés sous forme de conflits. Cette détection n'est pas restée théorique dans mon cas : elle m'a permis de vérifier formellement que ma grammaire était non ambiguë, et même de faire une découverte qui m'a marqué, l'absence du fameux conflit du « dangling else », que je détaille au chapitre 6.

Reste à justifier le choix de l'interprétation par parcours d'arbre plutôt qu'une machine à bytecode.

Ce choix obéit au même principe directeur que le précédent, à savoir privilégier la clarté. Une machine à bytecode aurait été plus rapide, mais elle aurait introduit un niveau d'indirection supplémentaire, le jeu d'instructions intermédiaire, qu'il aurait fallu concevoir, documenter et déboguer pour un gain de performance sans intérêt dans un cadre pédagogique. Le parcours d'arbre, au contraire, maintient une correspondance immédiate et visible entre la structure du programme, représentée par l'arbre, et sa signification, codée dans l'évaluateur. Ce choix s'est révélé pleinement justifié a posteriori : sa régularité m'a permis d'ajouter trois extensions au langage, puis un générateur de code complet, sans jamais avoir à remettre en cause l'architecture de départ.

Chapitre 3

Analyse et conception

3.1 Analyse des besoins

3.1.1 Besoins fonctionnels

Le langage ILANG et son interpréteur doivent offrir les fonctionnalités suivantes, directement issues du cahier des charges.

Types de données.

Entiers (seul type initialement requis) ; les booléens sont encodés en entiers (0 = faux, tout le reste = vrai), à la manière du C.

Variables.

Déclaration implicite à la première affectation, sans mot-clé. Portée globale pour le programme principal, locale pour les fonctions.

Expressions arithmétiques.

Opérateurs + - * / %, moins unaire, parenthèses, avec priorité et associativité correctes.

Comparaisons.

== != < > <= >=, renvoyant 0 ou 1.

Structures de contrôle.

Conditionnelle `si/sinon`, boucles `tant que` et `pour`.

Fonctions.

Déclaration avec `fonction`, paramètres, valeur de retour via `retourner`, appels récursifs.

Entrées-sorties.

`afficher(expr)` et `lire()`.

Commentaires.

De ligne (`//`) et de bloc (`/* */`).

3.1.2 Besoins non-fonctionnels

Correction.

L'interpréteur doit produire les résultats attendus sur les programmes de démonstration et

gérer proprement les erreurs.

Robustesse.

Aucune entrée, même fautive, ne doit provoquer de comportement indéfini silencieux ; les erreurs sont signalées et localisées.

Qualité du code.

Compilation sans avertissement (`-Wall -Wextra`), sans conflit grammatical, sans fuite mémoire.

Portabilité.

Code C standard, compilable sur toute plateforme POSIX disposant de GCC, Flex et Bison.

Lisibilité et maintenabilité.

Découpage en modules à responsabilité unique, code commenté.

3.1.3 Cas d'utilisation

L'acteur unique est l'utilisateur (ici, le développeur de programmes ILANG). Les cas d'utilisation principaux sont : (1) exécuter un fichier source (`./ilang prog.ilang`) ; (2) saisir un programme au clavier en mode interactif (`./ilang`) ; (3) en extension, produire l'assembleur correspondant (`./ilang -s prog.ilang`). Dans tous les cas, le programme lit le code source, le traduit, l'exécute (ou le compile), et restitue les résultats ou les messages d'erreur. La figure 3.1 en donne la vue « cas d'utilisation ».

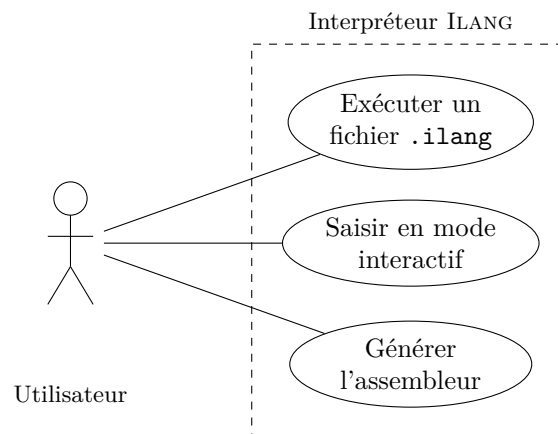


FIGURE 3.1 – Diagramme des cas d'utilisation.

Un programme Ilang complet. Pour fixer les idées, voici un programme exerçant la plupart des constructions du langage, à savoir une fonction récursive, une conditionnelle, une boucle et un affichage :

```

1 fonction factorielle(n) {
2     si (n <= 1) {
3         retourner(1)
4     }
5     retourner(n * factorielle(n - 1)) // appel recursif
6 }
7
8 pour (i = 1; i < 6; i = i + 1) {
9     afficher(factorielle(i)) // affiche 1, 2, 6, 24, 120
  
```

10 }

Listing 3.1 – Exemple : factorielle récursive et boucle d’affichage.

Ce programme combine déclaration de fonction, récursion, structure conditionnelle, boucle `pour` et entrée-sortie : il servira de fil conducteur dans la suite du rapport.

3.2 Conception architecturale

3.2.1 La chaîne de traitement

L’architecture suit le modèle classique d’un *pipeline* de compilation, où la sortie de chaque étage est l’entrée du suivant. La figure 3.2 en donne la vue d’ensemble.

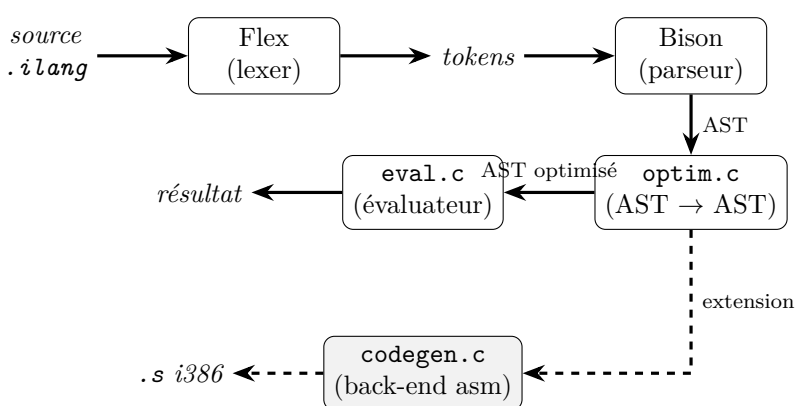


FIGURE 3.2 – Chaîne de traitement d’ILANG. Le front-end (Flex, Bison) est commun ; seul le back-end diffère entre l’interprétation et la génération de code.

3.2.2 Patrons de conception (*patterns*)

Deux patrons structurent le logiciel.

- **Pipeline** (chaîne de responsabilité) : chaque module transforme une représentation en une autre (texte $\rightarrow$ tokens $\rightarrow$ AST $\rightarrow$ AST optimisé $\rightarrow$ valeur). Cette séparation rend chaque étage testable et remplaçable indépendamment.
- **Parcours d’arbre type visiteur** : l’évaluateur, l’optimiseur et le générateur de code sont trois « visiteurs » distincts du même AST. En C, sans classes, le patron se concrétise par une fonction récursive qui distribue (`switch`) sur le type du nœud. C’est cette uniformité qui a rendu l’ajout du back-end assembleur peu coûteux : un troisième visiteur, sans toucher aux deux autres.

3.2.3 Choix du langage d’implémentation

L’interpréteur est écrit en C. Ce choix n’est pas neutre et se justifie :

1. Flex et Bison génèrent du code C ; écrire le reste en C évite toute couche d’interfaçage entre langages.

2. Le C impose de gérer explicitement la mémoire et les structures de données, ce qui est exactement l’objet du cours (pile d’appels, nœuds alloués dynamiquement).
3. Le back-end assembleur produit du code 32 bits lié à la *libc* C ; rester en C garantit une intégration naturelle.

3.2.4 Justification des principales décisions de conception

Plusieurs décisions de conception méritent d’être explicitées, car à chaque fois une autre solution était envisageable et le choix retenu résulte d’un arbitrage que je veux assumer.

La première décision est de construire un arbre syntaxique abstrait plutôt que d’évaluer le programme directement pendant l’analyse syntaxique. Cette seconde option, appelée évaluation dirigée par la syntaxe, est parfaitement possible pour une calculatrice : chaque règle de la grammaire calcule immédiatement sa valeur. Je l’ai écartée pour une raison de fond. Évaluer pendant l’analyse couplerait définitivement la syntaxe et la sémantique, interdirait toute passe d’optimisation, qui suppose de disposer du programme entier avant de l’exécuter, et rendrait impossible la réutilisation du même travail d’analyse pour produire du code. En matérialisant le programme sous forme d’arbre, je me donne au contraire la liberté de le parcourir autant de fois que nécessaire et de plusieurs manières. C’est exactement cette liberté qui m’a permis, plus tard, d’ajouter un optimiseur puis un générateur de code sans rien changer à l’analyse.

La deuxième décision concerne la table des symboles. J’aurais pu me contenter d’une unique table globale, ce qui aurait été plus simple, mais cela aurait rendu impossible la gestion correcte de la portée locale et de la récursion : deux appels simultanés de la même fonction auraient partagé les mêmes variables. À l’inverse, une structure trop élaborée aurait été disproportionnée. La pile d’environnements, où chaque appel dispose de son propre cadre, est la structure qui correspond exactement au besoin, et elle a en outre le mérite de reproduire fidèlement la pile d’appels matérielle étudiée au chapitre 2. Le choix d’une table de hachage à l’intérieur de chaque cadre, plutôt qu’une simple liste chaînée, vise la rapidité de recherche : l’accès à une variable est l’opération la plus fréquente de tout l’évaluateur, et il doit rester proche du temps constant.

La troisième décision, introduite avec l’extension des flottants, est de représenter une valeur par une union étiquetée distinguant entier et flottant, plutôt que de tout convertir en flottant. Cette dernière solution aurait été plus courte à écrire, mais elle aurait silencieusement détruit la sémantique entière d’origine, en particulier la division entière et le modulo dont dépend le calcul du PGCD. L’union étiquetée coûte quelques lignes de plus à chaque opération, mais elle préserve exactement le comportement attendu, ce qui était une contrainte non négociable de non-régression.

La quatrième décision concerne la gestion des ruptures de flot, c’est-à-dire le retour de fonction et les instructions `arrêter` et `continuer`. J’aurais pu employer les sauts non locaux du C, à savoir `setjmp` et `longjmp`, ou faire remonter un code de statut par chaque appel de la fonction d’évaluation. J’ai préféré des indicateurs globaux que les blocs et les boucles consultent. Ce choix privilégie la lisibilité : le mécanisme tient en quelques lignes et reste facile à suivre, ce qui compte davantage qu’une élégance théorique dans un interpréteur mono-thread. J’assume que

cette solution introduit un état global, et j'en discute les limites au chapitre 6.

3.3 Méthodologie et planification

3.3.1 Méthodologie retenue

Le développement a suivi une démarche **incrémentale et pilotée par les tests**. Plutôt que de tout écrire avant d'exécuter quoi que ce soit, le cœur (entiers seulement) a d'abord été rendu fonctionnel de bout en bout, validé sur les programmes de démonstration, puis enrichi par étapes (optimisations, puis extensions, puis back-end). Chaque ajout était suivi d'une vérification de non-régression. Cette stratégie « marcher avant de courir » s'est révélée payante : le passage délicat des entiers aux flottants, par exemple, aurait été bien plus risqué sur une base non encore éprouvée.

3.3.2 Identification des tâches et planning

Le projet se décompose en huit grandes tâches, présentées sous forme de diagramme de Gantt (figure 3.3). Chacune est développée dans la suite du rapport.

3.4 Conception détaillée

3.4.1 La grammaire (BNF)

Le cœur de la conception syntaxique est la grammaire, donnée ci-dessous en notation BNF. C'est elle que Bison transforme en analyseur LALR(1).

```

1 programme ::= liste_instructions
2
3 instruction ::= affectation osemi
4               | si_alors | tant_que | pour
5               | declaration_fonction
6               | appel_fonction osemi
7               | AFFICHER '(' expression ')' osemi
8               | RETOURNER '(' expression ')' osemi
9               | ARRETER osemi | CONTINUER osemi
10
11 osemi        ::= /* vide */ | ';'          /* point-virgule optionnel */
12
13 affectation  ::= IDENTIFIANT '=' expression
14
15 expression  ::= expression '+' expression | expression '-' expression
16               | expression '*' expression | expression '/' expression
17               | expression '%' expression
18               | expression '==' expression | expression '!=' expression
19               | expression '<' expression | expression '>' expression
20               | expression '<=' expression | expression '>=' expression
21               | '-' expression %prec UMINUS
22               | '(' expression ')'
23               | NOMBRE | FLOTTANT | IDENTIFIANT
24               | appel_fonction | LIRE '(' ')'

```

```

25
26 si_alors      ::= SI '(' expression ')' bloc
27                | SI '(' expression ')' bloc SINON bloc
28 tant_que     ::= TANTQUE '(' expression ')' bloc
29 pour         ::= POUR '(' affectation ';' expression ';' affectation ')' bloc
30 bloc         ::= '{' liste_instructions '}'
31
32 declaration_fonction ::= FONCTION IDENTIFIANT '(' liste_params ')' bloc
33 appel_fonction      ::= IDENTIFIANT '(' liste_args ')'

```

Listing 3.2 – Grammaire BNF d'ILANG.

La priorité des opérateurs n'est pas exprimée dans la BNF elle-même (qui est volontairement ambiguë sur ce point) mais par des directives de priorité, de la plus faible à la plus forte : comparaisons, puis $+$ $-$, puis $*$ $/$ $\%$, puis le moins unaire. Ce mécanisme, propre à Bison, évite de devoir réécrire la grammaire en cascade de non-terminaux (*terme*, *facteur*, etc.).

Un point mérite une explication, car il révèle une tension interne au cahier des charges. La grammaire BNF de référence fait figurer un point-virgule à la fin de l'affectation, de l'appel, de l'afficher et du retourner. Or aucun des programmes de démonstration fournis n'utilise de point-virgule, sauf à l'intérieur de l'en-tête de la boucle **pour**. Plutôt que de trancher arbitrairement en faveur de l'une des deux sources, j'ai choisi de rendre le point-virgule *optionnel* en fin d'instruction, au moyen du non-terminal **osemi** qui se réduit soit sur un point-virgule, soit sur le vide. Ainsi les deux styles d'écriture sont acceptés : les programmes de démonstration sans point-virgule comme les programmes rédigés à la lettre de la BNF. Cette souplesse n'introduit aucun conflit dans la grammaire, car le point-virgule n'apparaît comme séparateur obligatoire que dans le **pour**, un contexte distinct.

3.4.2 Structure de l'arbre syntaxique abstrait

Chaque nœud de l'AST est une structure C uniforme : un champ **type** indiquant la nature du nœud, et un ensemble de champs dont seuls certains sont significatifs selon le type. La figure 3.4 en donne une vue de type UML, et la figure 3.5 montre l'arbre obtenu pour une expression.

Le langage compte quatorze types de nœuds, recensés dans le tableau 3.1 avec les champs significatifs de chacun. Cette uniformité (une seule structure **Node**, un champ **type**) est ce qui rend les trois visiteurs, à savoir l'évaluateur, l'optimiseur et le générateur de code, aussi réguliers à écrire.

3.4.3 La table des symboles

La gestion de la portée repose sur une **pile d'environnements** (*frames*), chaque environnement étant lui-même une table de hachage. Un environnement est empilé à l'entrée d'une fonction et dépilé à sa sortie, à l'image de la pile d'appels matérialisée par le registre **%ebp** en assembleur i386 (chapitre 2 du cours). La figure 3.6 illustre l'état de la pile pendant un appel récursif.

TABLE 3.1 – Types de nœuds de l’AST et champs significatifs.

Type	Rôle	Champs utilisés
NODE_NUM	littéral numérique	num
NODE_VAR	référence de variable	name
NODE_BINOP	opération binaire	op, left, right
NODE_UNOP	moins unaire	op, left
NODE_ASSIGN	affectation	name, left
NODE_IF	conditionnelle	cond, body, elsebody
NODE_WHILE	boucle tant que	cond, body
NODE_FOR	boucle pour	left, cond, right, body
NODE_FUNC_DECL	déclaration de fonction	name, args, nargs, body
NODE_FUNC_CALL	appel de fonction	name, args, nargs
NODE_PRINT	afficher	left
NODE_READ	lire	aucun
NODE_RETURN	retourner	left
NODE_BREAK / NODE_CONTINUE	ruptures de boucle	aucun
NODE_BLOCK	suite d’instructions	args, nargs

3.4.4 Algorithmes clés

Évaluation. L’évaluateur est une fonction récursive renvoyant une *Value*. Son principe, pour les nœuds principaux, est résumé en pseudo-code dans le listing 3.3.

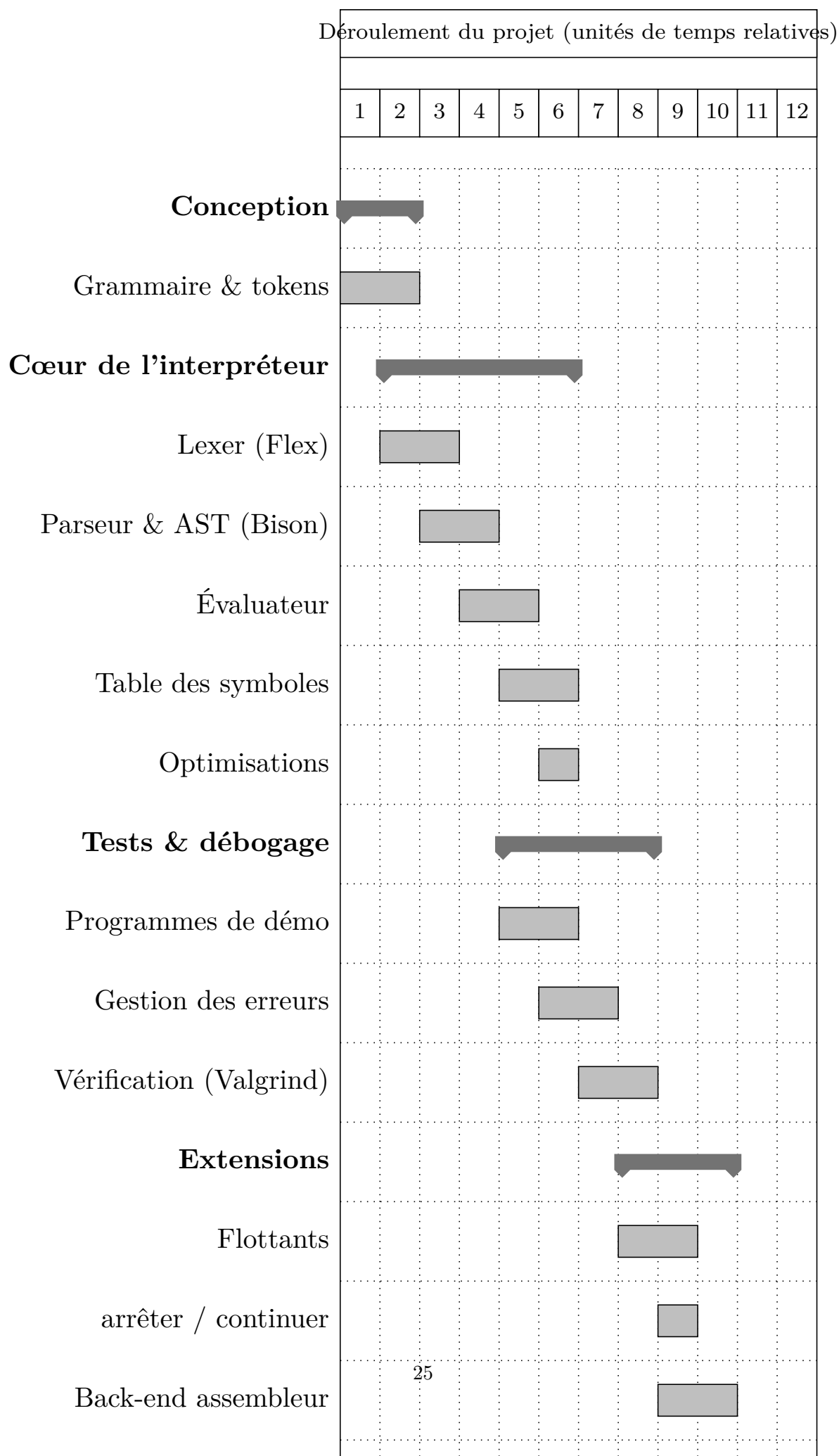
```

1 Value eval(Node n):
2   selon n.type:
3     NUM      -> renvoyer n.num
4     VAR      -> chercher n.name dans l'environnement (erreur si absent)
5     BINOP    -> a=eval(gauche); b=eval(droite); appliquer l'opérateur
6     ASSIGN   -> v=eval(expr); ranger v sous n.name; renvoyer v
7     IF       -> si eval(cond) vrai: eval(alors) sinon: eval(sinon)
8     WHILE/FOR -> boucler tant que eval(cond), en surveillant
9                 les signaux retourner / arreter / continuer
10    FUNC_CALL -> evaluer les args (frame appelant), empiler un frame,
11                 lier les parametres, evaluer le corps, depiler
12    RETURN    -> return_value=eval(expr); lever le drapeau de retour
13    BLOCK     -> evaluer chaque instruction, en s'arretant des qu'un
14                 signal de rupture de flot est leve

```

Listing 3.3 – Principe de l’évaluateur (pseudo-code).

Optimisation. L’optimiseur est un second parcours qui *réécrit* l’arbre. Deux transformations sont appliquées : le *pliage de constantes* (un nœud binaire dont les deux fils sont des littéraux est remplacé par le littéral résultat) et l’*élimination du code mort* (une conditionnelle dont la condition est un littéral est remplacée par la seule branche atteignable). Le détail, y compris le garde-fou contre la division par zéro, est exposé au chapitre 4.



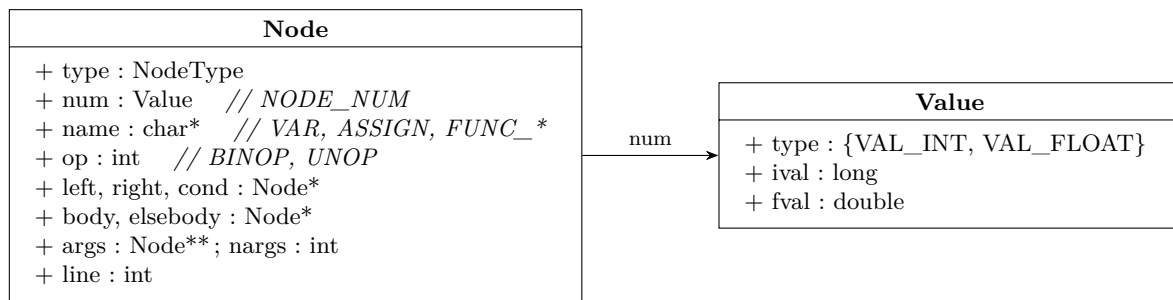
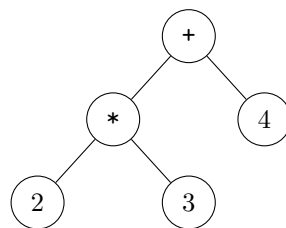
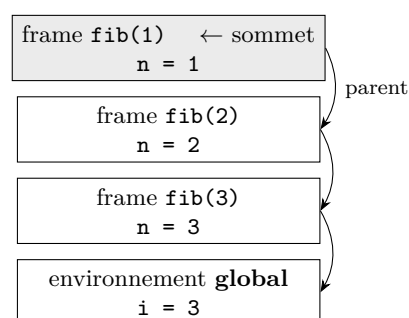


FIGURE 3.4 – Structure des nœuds de l’AST (vue UML simplifiée).

FIGURE 3.5 – AST de l’expression $2 * 3 + 4$. La priorité de $*$ sur $+$ se traduit par la position des nœuds dans l’arbre.FIGURE 3.6 – Pile d’environnements pendant l’évaluation de `fib(3)`. Le lien *parent* sert à restaurer le frame courant en sortie de fonction ; la résolution des variables, elle, est lexicale (frame courant puis global).

Chapitre 4

Implémentation

4.1 Environnement technique

L'ensemble a été développé sous GNU/Linux (Ubuntu, via WSL sur Windows). La chaîne d'outils est entièrement libre :

TABLE 4.1 – Outils et versions employés.

Outil	Rôle
GCC	compilateur C (et assembleur <code>-m32</code> pour l'extension)
GNU Bison 3.8	générateur d'analyseur syntaxique LALR(1)
Flex 2.6	générateur d'analyseur lexical
GNU Make	automatisation de la compilation
Valgrind	détection des fuites et erreurs mémoire
T _E X Live	rédaction du présent rapport

4.2 Architecture logicielle

4.2.1 Organisation du code

Le code est découpé en modules à responsabilité unique, listés dans le tableau 4.2. Cette séparation reflète directement le pipeline de la figure 3.2.

4.2.2 Interfaces entre modules

Les interfaces sont minimales et explicites. Flex et Bison communiquent par le type `YYSTYPE` (une union) et la table des tokens engendrée (`ilang.tab.h`). Bison renseigne une variable globale `program_root` pointant la racine de l'AST. `main.c` appelle successivement `yyparse()`, `optimize()` puis `eval_program()` (ou `generate_asm()`). L'évaluateur et le générateur de code dépendent de `ast.h`; l'évaluateur dépend en plus de `syntab.h`.

TABLE 4.2 – Modules du projet et leur rôle.

Fichier	Responsabilité
ilang.l	analyse lexicale (Flex)
ilang.y	grammaire et construction de l'AST (Bison)
ast.h / ast.c	définition et manipulation de l'AST, type Value
syntab.h / syntab.c	table des symboles (pile de frames + hachage)
eval.c	évaluateur récursif
optim.c	optimisations sur l'AST
codegen.c	back-end assembleur i386 (extension)
main.c	point d'entrée
Makefile	compilation automatisée

4.3 Le cœur : du texte à l'exécution

4.3.1 Analyse lexicale (Flex)

Le lexer reconnaît les nombres, identifiants, mots-clés, opérateurs et délimiteurs, et ignore espaces et commentaires. Sous le capot, Flex compile les expressions régulières en un *automate fini déterministe* (chapitre 9 du cours). La figure 4.1 en donne une vue schématique pour la reconnaissance des entiers et des identifiants : depuis l'état initial, le premier caractère lu oriente vers l'une ou l'autre des branches, et l'automate reste dans l'état d'acceptation tant que des caractères valides se présentent (principe du *plus long préfixe*).

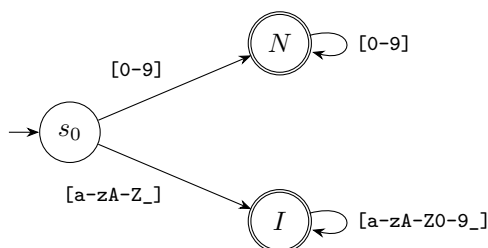


FIGURE 4.1 – Automate fini (simplifié) reconnaissant les entiers (N) et les identifiants (I). Les états doublés sont acceptants.

Au-delà de ce schéma général, deux points méritent l'attention.

D'abord, le mot-clé « tant que » comporte un espace : il est capturé par un motif dédié placé *avant* la règle des identifiants. Ensuite, les flottants sont testés avant les entiers, et acceptent les formes 3.14, 3. et .5 :

```

1 ({CHIFFRE}+"."{CHIFFRE}*)|("."{CHIFFRE}+) { yylval.fnum = strtod(yytext, NULL);
   return FLOTTANT; }
2 {CHIFFRE}+ { yylval.num = strtol(yytext, NULL, 10); return NOMBRE; }
3 {ID} { yylval.str = strdup(yytext); return IDENTIFIANT; }

```

Listing 4.1 – Extrait de ilang.l : littéraux numériques.

Les commentaires de bloc sont gérés par un *état exclusif* `%x COMMENT`. Le mécanisme mérite d'être détaillé, car il illustre la notion d'*automate à états* vue au chapitre sur les automates finis :

```

1  "/*"          { BEGIN(COMMENT); }    /* on entre dans l'état COMMENT */
2  <COMMENT>"*/"  { BEGIN(INITIAL); }    /* on en sort */
3  <COMMENT>\n    { /* contenu ignore, mais yylineno est incremente */ }
4  <COMMENT>.     { /* tout autre caractere du commentaire est ignore */ }
5  <COMMENT><<EOF>> { fprintf(stderr, "Erreur lexicale ligne %d : commentaire "
6                  "de bloc non termine\n", yylineno); exit(1); }

```

Listing 4.2 – Gestion des commentaires de bloc par état exclusif (extrait de `ilang.1`).

Une fois dans l'état `COMMENT`, seules les règles préfixées par `<COMMENT>` sont actives : le contenu est consommé caractère par caractère sans produire de token, jusqu'à la séquence fermante `*/`. La règle `<COMMENT><EOF>` attrape le cas d'un commentaire jamais refermé.

Exemple de flux de tokens. Pour bien voir ce que produit le lexer, considérons la ligne source `resultat = (a + b) * 2`. L'analyseur lexical la découpe en la suite d'unités lexicales suivante (les commentaires et espaces étant ignorés) :

TABLE 4.3 – Flux de tokens produit pour `resultat = (a + b) * 2`.

Texte	Token	Valeur sémantique	Motif Flex
resultat	IDENTIFIANT	"resultat" (strdup)	[a-zA-Z_]...
=	'='	aucune	"="
(	'('	aucune	"("
a	IDENTIFIANT	"a"	[a-zA-Z_]...
+	'+'	aucune	"+"
b	IDENTIFIANT	"b"	[a-zA-Z_]...
)	')'	aucune	")"
*	'*'	aucune	"*"
2	NOMBRE	2 (strtol)	[0-9]+

C'est cette suite de tokens, et non le texte brut, que reçoit l'analyseur syntaxique.

4.3.2 Analyse syntaxique et construction de l'AST (Bison)

La grammaire (listing 3.2) est traduite presque mot pour mot en règles Bison, chaque règle portant une *action sémantique* qui construit le nœud d'AST correspondant. Par exemple, la règle d'addition s'écrit :

```

1  expression : expression '+' expression { $$ = new_binop(OP_ADD, $1, $3); }

```

Listing 4.3 – Action sémantique d'une règle (extrait de `ilang.y`).

Le type des valeurs sémantiques portées par les symboles est une union, déclarée ainsi :

```

1  %union {
2      long    num;      /* valeur d'un NOMBRE (entier 64 bits) */
3      double fnum;     /* valeur d'un FLOTTANT */
4      char    *str;     /* nom d'un IDENTIFIANT */
5      Node    *node;    /* sous-arbre construit */
6      NodeList *list;   /* liste de parametres/arguments en construction */

```

```

7 }
8 %token <num>   NOMBRE
9 %token <fnum>  FLOTTANT
10 %token <str>   IDENTIFIANT

```

Listing 4.4 – Union sémantique et déclaration des tokens (extrait de `ilang.y`).

La priorité et l’associativité des opérateurs sont déclarées en tête de fichier, de la plus faible à la plus forte. C’est ce bloc, et non la forme de la grammaire, qui lève l’ambiguïté de la règle « plate » des expressions :

```

1 %left EQ NE LT GT LE GE      /* comparaisons : priorite la plus faible */
2 %left '+' '-'                /* addition / soustraction */
3 %left '*' '/' '%'            /* multiplication / division / modulo */
4 %right UMINUS                /* moins unaire : priorite la plus forte */

```

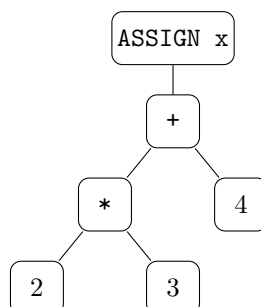
Listing 4.5 – Déclarations de priorité (extrait de `ilang.y`).

Concrètement, `%left '*'` placé *après* `%left '+'` signifie que `*` lie plus fort que `+` : l’expression `2 + 3 * 4` est donc analysée comme `2 + (3 * 4)`. L’associativité à gauche (`%left`) impose que `8 - 3 - 2` se lise `(8 - 3) - 2`.

Exemple de construction d’AST. Suivons la construction de l’arbre pour l’affectation `x = 2 * 3 + 4`. À mesure que Bison réduit les règles, les actions sémantiques appellent les constructeurs et font « remonter » des sous-arbres via la pile sémantique :

1. `NOMBRE(2)` se réduit en `expression` $\rightarrow$ `new_num(2)` ; de même pour 3 et 4.
2. La sous-expression `2 * 3` est réduite la première (priorité de `*`) $\rightarrow$ `new_binop(OP_MUL, num(2), num(3))`.
3. Puis l’addition $\rightarrow$ `new_binop(OP_ADD, mul, num(4))`.
4. Enfin l’affectation $\rightarrow$ `new_assign("x", add)`.

L’arbre obtenu est celui de la figure 4.2. On remarque que la priorité, déclarée en quelques lignes, se matérialise dans la *forme* de l’arbre : la multiplication est plus « profonde » que l’addition, donc évaluée avant.

FIGURE 4.2 – AST de l’instruction `x = 2 * 3 + 4`.

Les listes de paramètres et d’arguments sont accumulées dans une petite structure jetable `NodeList`, dont le tableau interne est ensuite transféré dans le nœud final (`NODE_FUNC_DECL`

ou `NODE_FUNC_CALL`), puis seule la coquille est libérée, une subtilité mémoire que Valgrind a confirmée sans fuite.

4.3.3 L'évaluateur

L'évaluateur réalise le parcours décrit au listing 3.3. Les ruptures de flot non locales (retour de fonction, `arrêter`, `continuer`) sont gérées par des drapeaux globaux que les blocs et les boucles surveillent. Le traitement d'un appel de fonction illustre la gestion de la pile d'environnements :

```

1  /* 1) Evaluer les arguments dans l'environnement de l'APPELANT. */
2  for (i = 0; i < node->nargs; i++) argvals[i] = eval(node->args[i]);
3  /* 2) Empiler un nouveau frame et y lier les parametres. */
4  env_push();
5  for (i = 0; i < decl->nargs; i++) env_set(decl->args[i]->name, argvals[i]);
6  /* 3) Evaluer le corps, recuperer la valeur de retour. */
7  return_flag = 0; eval(decl->body);
8  result = return_flag ? return_value : make_int(0);
9  /* 4) Depiler le frame. */
10 env_pop();

```

Listing 4.6 – Appel de fonction : ordre d'évaluation et pile (extrait de `eval.c`).

L'ordre est crucial : les arguments sont évalués *avant* l'empilement du nouveau frame, faute de quoi une expression comme `fib(n-1)` chercherait `n` dans le frame vide de la fonction appelée.

Le cœur du calcul arithmétique se trouve dans `eval_binop`, qui applique la règle de cohabitation entiers/flottants. En voici un extrait commenté :

```

1  Value a = eval(node->left);
2  Value b = eval(node->right);
3  int both_int = (a.type == VAL_INT && b.type == VAL_INT);
4  switch (node->op) {
5  case OP_ADD:                                /* si deux entiers -> entier */
6      return both_int ? make_int(a.ival + b.ival)
7                      : make_float(as_double(a) + as_double(b)); /* sinon flottant */
8  case OP_DIV:
9      if (both_int) {
10         if (b.ival == 0) {                    /* division entiere par zero : */
11             fprintf(stderr, "Erreur ligne %d : division par zero\n", node->line);
12             ;
13             return make_int(0);                /* message emis, le programme continue */
14         }
15         return make_int(a.ival / b.ival);    /* division ENTIERE preservee */
16     }
17     /* ... cas flottant ... */
18 }

```

Listing 4.7 – Cohabitation entiers/flottants et garde-fou (extrait de `eval.c`).

Exemple : trace d'exécution de `fib(3)`. Pour rendre concret le mécanisme d'appel récursif et de pile d'environnements, déroulons l'évaluation de `fib(3)` avec la définition usuelle. Le tableau 4.4 montre l'enchaînement des appels, l'état du sommet de pile et la valeur retournée.

TABLE 4.4 – Trace simplifiée de l'évaluation de `fib(3)`.

Appel	n	Action	Retour
<code>fib(3)</code>	3	$n \geq 2$: calcule <code>fib(2)+fib(1)</code>	2
<code>fib(2)</code>	2	$n \geq 2$: calcule <code>fib(1)+fib(0)</code>	1
<code>fib(1)</code>	1	$n < 2$: retourner(1)	1
<code>fib(0)</code>	0	$n < 2$: retourner(0)	0
<code>fib(1)</code>	1	$n < 2$: retourner(1)	1

Chaque ligne indentée correspond à un `env_push` (nouveau frame avec son propre `n`) suivi, au retour, d'un `env_pop`. Le drapeau de retour (`return_flag`) est levé par `retourner` et « consommé » par la boucle d'appel, qui récupère alors `return_value`. Le résultat final, `fib(3)` = 2, est conforme à l'exécution réelle (cf. chapitre 5).

4.4 Aspects techniques notables

4.4.1 Le type `Value` : faire cohabiter entiers et flottants

L'extension des flottants a transformé le type des valeurs manipulées. Plutôt que de tout passer en `double` (ce qui aurait cassé la division entière et le modulo), une *union étiquetée* distingue les deux natures :

```
1 typedef enum { VAL_INT, VAL_FLOAT } ValType;
2 typedef struct Value { ValType type; long ival; double fval; } Value;
```

Listing 4.8 – Le type `Value` (extrait de `ast.h`).

La règle sémantique est simple : une opération entre deux entiers reste entière ; dès qu'un flottant intervient, le calcul bascule en `double`. Ainsi `10 / 4` vaut 2 (sémantique d'origine préservée) tandis que `10.0 / 4` vaut 2.5.

4.4.2 La portée lexicale

Chaque environnement est une table de hachage de 211 alvéoles (un nombre premier, pour mieux répartir les collisions), indexée par la fonction `djb2` de Dan Bernstein :

```
1 static unsigned int hash(const char *s) {
2     unsigned int h = 5381;
3     int c;
4     while ((c = (unsigned char)*s++) != 0)
5         h = ((h << 5) + h) + c;    /* h * 33 + c */
6     return h % HASH_SIZE;
7 }
```

Listing 4.9 – Fonction de hachage `djb2` (extrait de `syntab.c`).

La résolution d’une variable se fait dans le frame courant puis, en repli, dans l’environnement global, *sans* traverser les frames intermédiaires de la pile d’appel. C’est ce qui garantit une portée **lexicale** : une fonction ne voit jamais les variables locales de la fonction qui l’a appelée. Ce point a fait l’objet d’une correction après une première version (involontairement) à portée dynamique, détaillée au chapitre 6.

Exemple : la table des symboles pendant `pgcd(48, 18)`. Lors de l’appel à la fonction `pgcd`, un frame est empilé contenant les paramètres `a` et `b`, auxquels s’ajoute la variable locale `temp` créée par la première affectation. Le tableau 4.5 montre l’évolution du contenu de ce frame au fil des itérations de la boucle `tant que`.

TABLE 4.5 – Évolution du frame de `pgcd` (algorithme d’Euclide sur 48 et 18).

Itération	a	b	temp
entrée	48	18	(non créé)
1	18	12	18
2	12	6	12
3	6	0	6
$b = 0$: sortie de boucle, <code>retourner(a)</code> ⇒ 6			

4.4.3 Localisation des erreurs à la ligne

Tous les messages d’erreur indiquent la ligne fautive, au format **Erreur ligne N :** Le choix de conception décisif est de *figer le numéro de ligne dans chaque nœud au moment du parsing* (`node->line = yylineno` dans le constructeur), et non à l’évaluation. En effet, au moment où l’évaluation commence, l’analyse lexicale est terminée : `yylineno` vaut alors la *dernière* ligne du fichier. L’utiliser dans l’évaluateur ferait pointer toutes les erreurs sur la fin du fichier. En gravant la ligne dans le nœud à sa création, chaque diagnostic d’exécution désigne le bon emplacement, comme le confirme le test du « mauvais nombre d’arguments », qui signale correctement la ligne 5, celle de l’appel fautif, et non la fin du fichier.

4.4.4 Les optimisations et le garde-fou de la division par zéro

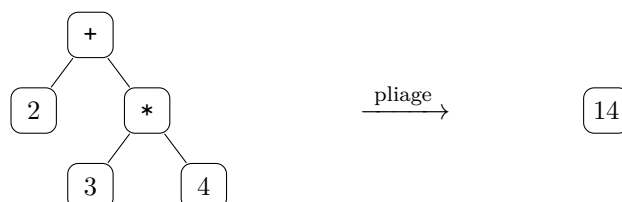
Le pliage de constantes calcule à la compilation les sous-expressions entièrement constantes. Un piège guette : `10 / 0` est, lui aussi, composé de deux littéraux. Le plier provoquerait soit un SIGFPE pendant la compilation, soit un court-circuitage du message d’erreur attendu à l’exécution. Une fonction `can_fold` interdit donc de plier toute division ou modulo qui échouerait ; ces cas restent dans l’arbre et c’est l’évaluateur qui émet le diagnostic.

```

1 static int can_fold(int op, Value a, Value b) {
2     int both_int = (a.type == VAL_INT && b.type == VAL_INT);
3     if (op == OP_DIV) return both_int ? (b.ival != 0) : (as_double(b) != 0.0);
4     if (op == OP_MOD) return both_int && (b.ival != 0); /* modulo : entiers, !=
5         0 */
6     return 1; /* toutes les autres operations sont pliables */
7 }
```

Listing 4.10 – Garde-fou du pliage (extrait de `optim.c`).

Exemple : avant/après pliage. Sur l'expression $2 + 3 * 4$, l'optimiseur reconnaît que $3 * 4$ a deux fils littéraux et le remplace par 12, puis que $2 + 12$ est à son tour pliable, et le remplace par 14. L'arbre de gauche (figure 4.3) est ainsi réduit à une unique feuille. À l'exécution, `afficher(2 + 3 * 4)` n'effectue donc *aucun* calcul : la valeur 14 est déjà dans l'arbre.

FIGURE 4.3 – Pliage de constantes : l'arbre de $2 + 3 * 4$ est réduit à la feuille 14.

De même, l'élimination du code mort transforme `si (0) { A } sinon { B }` en le seul bloc B, et `si (1) { A }` en le seul bloc A, la condition constante étant évaluée une fois pour toutes à la compilation.

4.4.5 Gestion de la mémoire

L'AST est entièrement alloué dynamiquement, puis libéré récursivement par `free_node`. Le transfert du tableau d'arguments depuis la `NodeList` vers le nœud (sans le dupliquer ni le libérer deux fois) a demandé une attention particulière. La vérification Valgrind confirme l'absence de fuite (voir chapitre 5), après ajout d'un appel à `yylex_destroy()` pour libérer le tampon interne de Flex.

4.4.6 Sûreté et robustesse

Bien qu'ILANG ne soit pas exposé à un contexte adverse, plusieurs propriétés de *sûreté* ont été soignées. La mémoire est intégralement maîtrisée : allocations vérifiées (tout `malloc/realloc` dont l'échec est fatal est testé), libération récursive de l'AST, et absence de fuite confirmée par Valgrind. Aucune entrée utilisateur n'est copiée dans un tampon de taille fixe : les identifiants sont dupliqués par `strdup` (allocation dimensionnée), ce qui exclut tout débordement de tampon à l'analyse. Les conditions d'erreur (variable ou fonction indéfinie, mauvais nombre d'arguments, division par zéro, rupture de flot hors boucle, caractère ou syntaxe invalides) sont toutes interceptées et signalées plutôt que de produire un comportement indéfini silencieux. La seule limite de sûreté identifiée est le débordement de la pile de l'hôte en cas de récursion très profonde (cf. chapitre 5), inhérent à l'évaluation récursive.

4.4.7 Le back-end assembleur (extension hors-périmètre)

Le module `codegen.c` parcourt le même AST mais émet du code assembleur i386 (syntaxe AT&T) au lieu d'évaluer. Il applique la convention d'appel `cdecl` : arguments empilés de droite

à gauche, résultat dans `%eax`, cadre de pile géré par `%ebp/%esp`. Les variables du programme principal deviennent des globales en `.bss`, les paramètres et locales des fonctions sont rangés en déplacement par rapport à `%ebp`. Les expressions sont compilées en machine à pile. Ce module, entièrement optionnel, n'altère en rien l'interpréteur ; il est décrit en détail au chapitre 6 et son code figure en annexe A.

La traduction d'une opération binaire illustre la stratégie de machine à pile :

```

1  gen_expr(out, n->left);                /* operande gauche -> %eax */
2  fprintf(out, "\tpushl %%eax\n");        /* sauvegarde sur la pile */
3  gen_expr(out, n->right);                /* operande droit -> %eax */
4  fprintf(out, "\tmovl %%eax, %%ecx\n");  /* droit -> %ecx */
5  fprintf(out, "\tpopl %%eax\n");         /* gauche -> %eax (restaure) */
6  switch (n->op) {
7  case OP_ADD: fprintf(out, "\taddl %%ecx, %%eax\n"); break; /* eax = eax + ecx
   */
8  case OP_MUL: fprintf(out, "\timull %%ecx, %%eax\n"); break;
9  /* division : cltd etend le signe, idivl place le quotient dans %eax */
10 case OP_DIV: fprintf(out, "\tcltd\n\tidivl %%ecx\n"); break;
11 }

```

Listing 4.11 – Génération d'un binop en machine à pile (extrait de `codegen.c`).

Exemple : assembleur produit pour une fonction. Pour la fonction `fonction carre(x)` { retourner(`x * x`) }, le générateur produit le code du listing 4.12, ici annoté. On y reconnaît le prologue `cdecl`, l'accès au paramètre par déplacement positif depuis `%ebp`, la machine à pile pour le produit, et l'épilogue.

```

1  il_carre:
2      pushl %ebp                # prologue : sauvegarde l'ancien base pointer
3      movl %esp, %ebp           # etablit le nouveau cadre de pile
4      movl 8(%ebp), %eax        # charge le parametre x (1er arg en +8)
5      pushl %eax                # empile x (operande gauche)
6      movl 8(%ebp), %eax        # recharge x (operande droit)
7      movl %eax, %ecx           # x -> %ecx
8      popl %eax                 # x -> %eax
9      imull %ecx, %eax          # %eax = x * x
10     leave                     # epilogue : restaure %esp et %ebp (retourner)
11     ret                       # retour, resultat dans %eax

```

Listing 4.12 – Assembleur généré pour `carre` (annoté).

Côté appelant, `afficher(carre(7))` empile la constante 7, appelle `il_carre`, nettoie la pile (`addl $4, %esp`), puis transmet le résultat (`%eax`) à `printf` via le format `"%d\n"`. Cette correspondance directe entre les concepts de l'évaluateur (pile d'appels, liaison des paramètres) et leur réalisation matérielle (cadre `%ebp`, déplacements) est précisément ce que le chapitre 2 du cours cherche à faire comprendre.

Traduction des boucles. Les structures de contrôle deviennent des *labels* et des *sauts*. Une boucle `tant que (c) { corps }` est encadrée par deux labels : un label de tête (avant le

test) et un label de fin. La condition échouée saute vers la fin ; le corps se termine par un saut inconditionnel vers la tête. Les instructions **arrêter** et **continuer** se traduisent alors trivialement par un **jmp** vers, respectivement, le label de fin et le label de tête (ou d'incrément pour une boucle **pour**). Le générateur maintient à cet effet une pile de labels de boucles, exactement comme l'évaluateur maintient un signal de rupture. Schématiquement :

```

1  .Ltete:                                # etiquette de tete
2      < code de la condition c, resultat dans %eax >
3      cmpl $0, %eax
4      jz  .Lfin                          # si c est faux -> sortir
5      < code du corps >                  # 'arreter' = jmp .Lfin ; 'continuer' = jmp .Ltete
6      jmp .Ltete                        # boucler
7  .Lfin:                                # etiquette de fin

```

Listing 4.13 – Schéma de traduction d'une boucle **tant que**.

4.4.8 Compilation automatisée

Le Makefile sépare deux jeux d'options : le code du projet est compilé avec **-Wall -Wextra** (zéro avertissement exigé), tandis que le code engendré par Flex et Bison est compilé sans **-Wextra**, ces fichiers machine-générés pouvant contenir des constructions qui déclencheraient des avertissements non imputables au projet. Les dépendances entre fichiers générés (**lex.yy.c** dépend de **ilang.tab.h**) sont explicitées pour que **make** ordonne correctement la compilation.

```

1  CFLAGS    = -Wall -Wextra -std=gnu11 -g    # notre code : tous les avertissements
2  GENFLAGS  = -std=gnu11 -g                  # code genere : sans -Wextra
3
4  # Bison : grammaire -> parseur + en-tete des tokens
5  ilang.tab.c ilang.tab.h: ilang.y
6      $(YACC) -d -o ilang.tab.c ilang.y
7  # Flex : depend de l'en-tete des tokens engendre par Bison
8  lex.yy.c: ilang.l ilang.tab.h
9      $(LEX) -o lex.yy.c ilang.l

```

Listing 4.14 – Séparation des jeux d'options (extrait du Makefile).

Le point d'entrée **main.c** se charge enfin d'analyser les arguments (chemin du fichier, options **-s** et **-no-opt**), d'enchaîner **yyparse()** → **optimize()** → **eval_program()** (ou **generate_asm()**), puis de libérer l'AST et la table des symboles avant de rendre la main.

Chapitre 5

Validation et évaluation

5.1 Stratégie de test

La validation combine quatre niveaux complémentaires :

Tests fonctionnels.

Exécution des programmes de démonstration et comparaison des sorties avec les résultats attendus.

Tests de gestion d’erreurs.

Soumission de programmes fautifs pour vérifier que chaque catégorie d’erreur est détectée, localisée et signalée.

Tests de non-régression.

Après chaque évolution (optimisations, extensions, corrections), re-vérification que les sorties de référence restent identiques.

Tests de robustesse mémoire et cas limites.

Analyse Valgrind et campagne de cas extrêmes (fichier vide, EOF, grands nombres, récursion profonde, ruptures de flot mal placées).

5.2 Protocole expérimental

5.2.1 Conditions et métriques

Les mesures de performance ont été réalisées sur une machine équipée d’un processeur AMD Ryzen 5 5500 (12 fils d’exécution), sous Ubuntu (WSL). Chaque temps rapporté est le *meilleur* de trois exécutions (pour limiter le bruit lié à l’ordonnancement), mesuré par `/usr/bin/time`. Les métriques retenues sont : le temps d’exécution (secondes), le résultat calculé (pour la correction), et le verdict Valgrind (octets perdus, erreurs).

5.2.2 Jeux d’essai

Les programmes de démonstration servent de jeu d’essai fonctionnel : Fibonacci récursif, PGCD d’Euclide, programme interactif, plus les deux programmes illustrant les extensions (flottants,

boucles avec `arrêter/continuer`). Les benchmarks utilisent des programmes paramétrés (Fibonacci à différentes tailles, boucle de 30 millions d'itérations, récursion profonde).

5.2.3 Reproductibilité

Les tests sont automatisés et rejouables. La cible `make test` compile puis exécute l'ensemble des programmes de démonstration ; un script dédié soumet les programmes fautifs et affiche, pour chacun, le message d'erreur et le code de sortie. Ce script tient en quelques lignes et illustre bien la simplicité du dispositif :

```

1 for f in variable divzero fonction arguments lexical syntaxe; do
2     echo "==== $f ====="
3     ./ilang "tests/err_$f.ilang"    # le programme fautif
4     echo "(code de sortie : $?)"    # 1 si erreur fatale, 0 si poursuite
5 done

```

Listing 5.1 – Cœur du script de test des erreurs.

Cette automatisation a servi de filet de sécurité tout au long du développement. Après chaque modification, une simple commande permettait de vérifier qu'aucune sortie de référence n'avait changé. C'est précisément ce dispositif qui m'a permis de garantir la non-régression lors de l'ajout des flottants, puis des deux autres extensions, puis du back-end assembleur, sans craindre de casser silencieusement ce qui fonctionnait déjà.

5.3 Résultats fonctionnels

5.3.1 Programmes de démonstration

Les trois programmes du cahier des charges produisent les sorties attendues.

```

1 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```

Listing 5.2 – Sortie de `fibonacci.ilang` (15 premiers termes).

```

1 6

```

Listing 5.3 – Sortie de `pgcd.ilang` : `pgcd(48, 18)`.

```

1 999      999
2 49       5

```

Listing 5.4 – Sortie de `interactif.ilang` pour les entrées 7 puis -5.

Les programmes d'extension confirment le bon fonctionnement des flottants et des ruptures de boucle :

```

1 6.28      (3.14 * 2)
2 3.5       (2.5 + 1)
3 2         (10 / 4 : division entiere preservee)
4 2.5       (10.0 / 4 : division reelle)
5 19.6349   (aire d'un disque de rayon 2.5)

```

```
6 1      (3.5 > 3)
```

Listing 5.5 – Sortie de `flottants.ilang`.

```
1 0 1 2 3 4      (arret a 5)
2 1 3 5 7 9      (saut des pairs)
3 10 9 8          (arret a 7)
```

Listing 5.6 – Sortie de `boucles.ilang` (arrêter puis continuer puis arrêter dans un tant que).

5.3.2 Gestion des erreurs

Chaque catégorie d’erreur du cahier des charges est traitée et localisée à la ligne (tableau 5.1). Cas notable, la division par zéro émet un message *et laisse le programme continuer*, conformément à la spécification.

TABLE 5.1 – Gestion des erreurs : message émis et code de sortie.

Erreur	Message	Sortie
Variable non définie	Erreur ligne 1 : variable 'z' non definie	1
Division par zéro	Erreur ligne 1 : division par zero	0 (continue)
Fonction non définie	Erreur ligne 1 : fonction 'inconnue' non definie	1
Mauvais nb d’arguments	... attend 2 argument(s), 1 fourni(s)	1
Caractère invalide	Erreur lexicale ligne 1 : caractere invalide '@'	1
Erreur de syntaxe	Erreur de syntaxe ligne 1 : syntax error...	1
Rupture hors boucle	Erreur ligne 2 : 'arreter' utilise hors d’une boucle	1

Pour illustrer concrètement le traitement des erreurs, voici les sorties réelles produites par l’interpréteur sur une série de programmes fautifs. On notera la localisation systématique à la ligne et, pour la division par zéro, la poursuite de l’exécution.

```
1 $ ./ilang tests/err_variable.ilang
2 Erreur ligne 1 : variable 'z' non definie
3
4 $ ./ilang tests/err_divzero.ilang
5 Erreur ligne 1 : division par zero
6 0
7 123          <- le programme continue apres l'erreur
8
9 $ ./ilang tests/err_fonction.ilang
10 Erreur ligne 1 : fonction 'inconnue' non definie
11
12 $ ./ilang tests/err_arguments.ilang
13 Erreur ligne 5 : fonction 'pgcd' attend 2 argument(s), 1 fourni(s)
14
15 $ ./ilang tests/err_lexical.ilang
16 Erreur lexicale ligne 1 : caractere invalide '@'
17
18 $ ./ilang tests/err_syntaxe.ilang
19 Erreur de syntaxe ligne 1 : syntax error, unexpected IDENTIFIANT, expecting '('
```

Listing 5.7 – Sorties réelles sur des programmes fautifs.

5.4 Résultats de performance

5.4.1 Interprétation contre code natif

Le tableau 5.2 et la figure 5.1 comparent le temps de calcul de Fibonacci par l'interpréteur et par le binaire issu du back-end assembleur. L'écart est spectaculaire : sur `fib(32)`, le code natif est de l'ordre de *plusieurs centaines de fois* plus rapide. Cela illustre concrètement le coût du parcours d'arbre (distribution sur le type de nœud, appels de fonction C en cascade) face à du code machine compilé.

TABLE 5.2 – Temps d'exécution de Fibonacci : interpréteur vs assembleur i386.

n	résultat	interpréteur (s)	asm i386 (s)
28	317 811	0,43	$\approx 0,00$
30	832 040	1,13	$\approx 0,00$
32	2 178 309	2,97	0,01

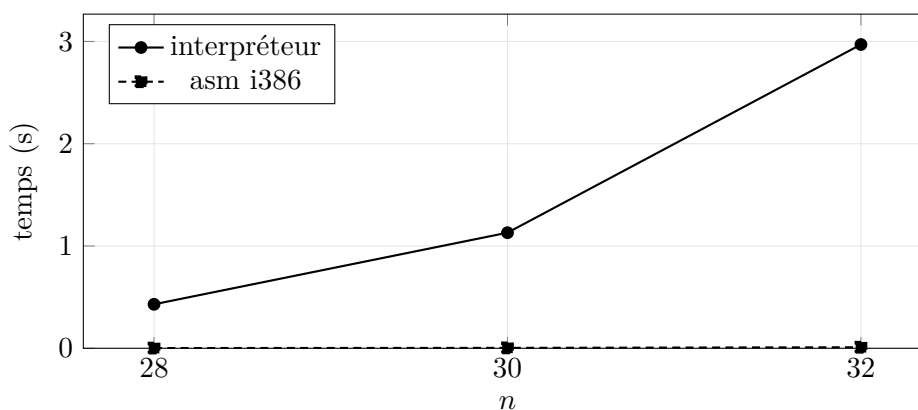


FIGURE 5.1 – Croissance du temps d'exécution de Fibonacci. La courbe de l'interpréteur révèle bien la complexité exponentielle de l'algorithme ; le code natif, sur la même échelle, reste collé à l'axe.

5.4.2 Effet des optimisations

Pour isoler l'effet du pliage de constantes, un programme effectue 30 millions d'itérations d'un corps de boucle contenant une expression constante (`s = s + (2*3 + 4*5 - 6)`, qui se replie en `s + 20`). Le drapeau `-no-opt` permet de désactiver l'optimiseur. Le pliage fait gagner environ un tiers du temps (figure 5.2).

5.4.3 Limite de profondeur de récursion

Comme l'évaluateur récurse en C (un appel ILANG se traduit par une cascade d'appels de `eval`), la profondeur de récursion est bornée par la pile de l'hôte. La sonde du tableau 5.3 montre que `somme(5000)` aboutit, mais que `somme(10000)` provoque un débordement de pile (signal SIGSEGV, code de sortie 139). C'est une limite de fond de l'approche par parcours d'arbre, discutée au chapitre 6.

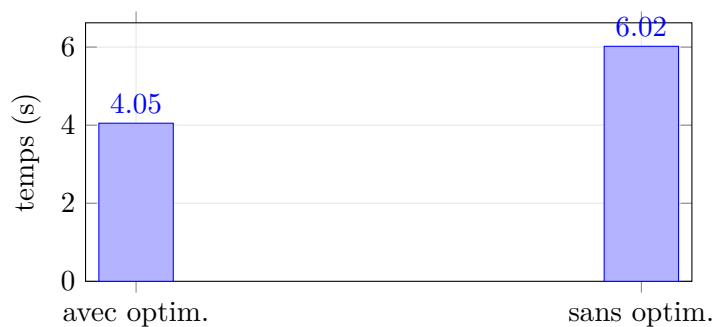


FIGURE 5.2 – Temps de la boucle de 30 M d’itérations, avec et sans pliage de constantes. Le gain ($\approx 33\%$) provient de la suppression du calcul arithmétique répété à chaque tour.

TABLE 5.3 – Profondeur de récursion : `somme(n)` récursive.

n	résultat
1 000	500 500 (ok)
5 000	12 502 500 (ok)
10 000	débordement de pile (SIGSEGV)

5.4.4 Vérification mémoire

L’exécution sous Valgrind, après ajout de `yylex_destroy()`, ne signale **aucune fuite ni erreur** sur l’ensemble des programmes :

```

1 in use at exit: 0 bytes in 0 blocks
2 ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Listing 5.8 – Verdict Valgrind (programme `pgcd.ilang`).

5.4.5 Équivalence interpréteur / code généré

Enfin, pour chaque programme entier, la sortie du binaire assemblé a été comparée à celle de l’interpréteur : elles sont **identiques** (Fibonacci, PGCD, boucles avec `arrêter/continuer`, et même le programme interactif qui passe par `scanf/printf`). Le back-end refuse proprement les programmes utilisant des flottants. Cette équivalence constitue une validation croisée précieuse : deux back-ends indépendants, partant du même AST, calculent les mêmes résultats.

5.5 Étude de cas de bout en bout

Pour conclure ce chapitre, suivons le programme `factorielle` (présenté au chapitre 3) tout au long de la chaîne, afin de relier les résultats aux étapes décrites.

1. **Analyse lexicale.** Le source est découpé en tokens : `FONCTION`, `IDENTIFIANT(factorielle)`, `'(`, `IDENTIFIANT(n)`, `)'`, `'{'`, `SI`, ... Les commentaires et espaces sont éliminés.
2. **Analyse syntaxique.** Bison réduit ces tokens selon la grammaire et bâtit l’AST : une `NODE_FUNC_DECL` (paramètre `n`, corps contenant un `NODE_IF` et un `NODE_RETURN` avec un `NODE_BINOP *` dont le fils droit est un `NODE_FUNC_CALL` récursif).

3. **Optimisation.** Le sous-arbre $n - 1$ n'est pas constant (il dépend de n) : rien n'est plié ici, la passe laisse l'arbre inchangé, ce qui est correct.
4. **Évaluation.** La fonction est d'abord enregistrée (pré-passe), puis la boucle **pour** appelle `factorielle(1)` à `factorielle(5)`. Chaque appel empile un frame, lie n , évalue le corps, et dépile. Les valeurs retournées sont 1, 2, 6, 24, 120.
5. **Sortie.** `afficher` imprime chaque résultat :

```
1 $ ./ilang factorielle.ilang
2 1
3 2
4 6
5 24
6 120
```

Le même programme passé à `./ilang -s` produit l'assembleur de l'annexe E qui, compilé et exécuté, affiche exactement la même chose. La boucle complète, depuis le texte source jusqu'au résultat et par deux back-ends indépendants, est ainsi vérifiée de bout en bout.

Chapitre 6

Discussion

6.1 Analyse critique

6.1.1 Forces

Le projet atteint l'ensemble de ses objectifs et présente plusieurs qualités notables. Sur le plan de l'**ingénierie**, la compilation est propre (aucun avertissement, aucun conflit grammatical) et la gestion mémoire est exempte de fuite. Sur le plan de la **conception**, l'architecture en pipeline avec front-end commun s'est révélée juste : elle a permis d'ajouter un back-end de génération de code sans toucher à l'interpréteur, validant a posteriori le choix d'un AST comme représentation pivot. Sur le plan de la **robustesse**, toutes les erreurs prévues sont détectées et localisées, et la campagne de cas limites n'a révélé aucun plantage non maîtrisé.

6.1.2 Faiblesses et limitations

L'honnêteté impose de recenser les limites, dont certaines sont des choix assumés et d'autres des bornes de fond de l'approche.

Profondeur de récursion bornée.

L'évaluateur récursant en C, la profondeur d'appels ILANG est limitée par la pile de l'hôte (rupture autour de quelques milliers de niveaux, cf. tableau 5.3). Une machine virtuelle à pile explicite lèverait cette limite.

Performance de l'interprétation.

Le parcours d'arbre est intrinsèquement lent (cf. figure 5.1). C'est le prix de la lisibilité ; un bytecode l'améliorerait sensiblement.

Absence de portée de bloc.

Les variables déclarées dans un bloc (**si**, boucle) vivent dans le frame de la fonction, pas dans un frame de bloc. C'est cohérent avec la spécification (« globale au principal, locale aux fonctions ») mais diffère de langages à portée de bloc.

Back-end assembleur restreint aux entiers.

Le générateur de code refuse les flottants et ne reproduit pas la gestion « douce » de la division par zéro (un `idiv` par zéro lève un SIGFPE). Choix assumé, hors périmètre.

Pas de système de types.

Le langage est dynamiquement typé au sens minimal (entier ou flottant), sans vérification statique.

6.1.3 Comparaison avec l'existant

Par rapport à l'approche antérieure (le mini-langage PARLOT et son analyse manuelle en Python), ILANG gagne sur deux tableaux : la grammaire est formalisée et vérifiable, et l'usage d'outils générateurs élimine des classes entières d'erreurs (oubli de cas, ambiguïté non détectée). En contrepartie, la courbe d'apprentissage de Flex et Bison est plus raide qu'une descente récursive écrite à la main. C'est toutefois un investissement qui se rentabilise dès que la grammaire se complexifie, et qui paie immédiatement sur le plan de la fiabilité, puisqu'il supprime des familles entières d'erreurs au lieu de les laisser à la vigilance du programmeur.

6.2 Difficultés rencontrées et solutions

Cette section revient sur les obstacles techniques marquants et la manière dont ils ont été levés.

6.2.1 Le « dangling else » qui n'existait pas

Le conflit syntaxique du *dangling else*, c'est-à-dire la question de savoir à quel **si** rattacher un **sinon** dans une imbrication, est l'exemple canonique de conflit shift/reduce enseigné dans tous les cours de compilation. M'attendant à le rencontrer, j'avais d'abord protégé ma grammaire par une résolution de priorité, en introduisant un pseudo-token de plus faible priorité que **SINON**. Par acquit de conscience, j'ai ensuite retiré cette protection pour vérifier si le conflit existait réellement, et Bison n'en a signalé absolument aucun. Mes deux règles, données ci-dessous, se compilent sans la moindre ambiguïté.

```

1 si_alors
2   : SI '(' expression ')' bloc           { $$ = new_if($3, $5, NULL); }
3   | SI '(' expression ')' bloc SINON bloc { $$ = new_if($3, $5, $7); }
4   ;

```

Listing 6.1 – Les deux règles du **si**, sans aucune directive de priorité.

L'explication tient entièrement à un détail de ma grammaire que je n'avais pas mesuré au départ : les corps de conditionnelle sont *obligatoirement* placés entre accolades, car la règle **bloc** l'impose. Or, dans le cours, le conflit naît de corps réduits à une instruction nue. Ici, comme tout corps se termine par une accolade fermante, un **sinon** est nécessairement précédé du symbole **}**. Au moment précis où l'analyseur pourrait hésiter, après avoir reconnu **SI (expression) bloc**, le symbole de prévision n'est jamais **SINON** mais toujours **}** ou un début d'instruction. Autrement dit, **SINON** n'appartient pas à l'ensemble FOLLOW de la réduction du **si** sans **sinon**, et l'ambiguïté n'a tout simplement pas lieu d'être. J'ai donc supprimé la directive de priorité devenue inutile. Je retiens de cet épisode une leçon de méthode : il vaut mieux *tester* une hypothèse avec l'outil que recopier par réflexe une solution apprise, car le contexte précis de ma grammaire rendait cette solution superflue.

6.2.2 L'ordre d'évaluation et la portée

La gestion des fonctions a révélé deux subtilités que je n'avais pas anticipées et qui, toutes deux, touchent à la portée des variables.

La première est une question d'**ordre**. Lorsqu'on appelle une fonction, il faut impérativement évaluer les arguments dans l'environnement de l'appelant *avant* d'empiler le cadre de la fonction appelée. Dans ma toute première version, j'avais inversé ces deux étapes, et une expression aussi banale que `fibonacci(n - 1)` échouait : une fois le nouveau cadre empilé, la variable `n` était cherchée dans ce cadre encore vide au lieu du cadre de l'appelant. La correction a consisté à séparer nettement les phases, comme le montre l'extrait suivant.

```
1 /* 1) arguments évalués dans le cadre de l'APPELANT */
2 for (i = 0; i < node->nargs; i++) argvals[i] = eval(node->args[i]);
3 /* 2) seulement maintenant on empile le cadre de l'appelle */
4 env_push();
5 for (i = 0; i < decl->nargs; i++) env_set(decl->args[i]->name, argvals[i]);
```

Listing 6.2 – L'ordre correct : évaluer les arguments avant d'empiler le cadre.

La seconde subtilité, plus profonde, concerne la **nature de la portée**. Ma première implémentation de la recherche de variable parcourait toute la pile d'appel, du sommet jusqu'à la base. Or ce parcours complet réalise en fait une portée *dynamique* : une fonction pouvait y lire les variables locales de la fonction qui l'avait appelée, ce qui n'est pas le comportement attendu d'un langage moderne. Le test reproducteur ci-dessous affichait 123 au lieu d'échouer, puisque `g` voyait le `secret` local de `f`.

```
1 fonction g() { retourner(secret) }
2 fonction f() { secret = 123 retourner(g()) }
3 afficher(f())           // affichait 123 : g voyait une locale de f !
```

Listing 6.3 – Cas qui révèle la portée dynamique (corrigé depuis).

La spécification réclamant une portée « lexicale », j'ai corrigé la résolution pour qu'elle ne consulte que le cadre courant, puis en repli l'environnement global, sans traverser les cadres intermédiaires de la pile d'appel.

```
1 int env_get(const char *name, Value *out) {
2     if (lookup_in(current_env, name, out)) return 1;           /* cadre courant
3     /*
4     if (current_env != global_env && lookup_in(global_env, name, out)) return 1;
5     return 0;                                                  /* sinon : absente
6     */
7 }
```

Listing 6.4 – Résolution lexicale corrigée (extrait de `syntab.c`).

Désormais le programme ci-dessus signale correctement que `secret` n'est pas défini dans `g`. Je relève au passage une contradiction du cahier des charges lui-même, qui qualifiait de « lexicale » une portée dont il décrivait pourtant l'algorithme dynamique. J'ai tranché en faveur du terme employé, qui correspond aussi au comportement le plus utile.

6.2.3 Le piège du pliage de la division par zéro

Le pliage de constantes consiste à calculer dès la compilation les sous-expressions dont les deux opérandes sont des littéraux. Le piège est que $10 / 0$ est, lui aussi, une expression à deux opérandes littéraux. Si je le pliais sans précaution, deux issues fâcheuses étaient possibles : ou bien l’optimiseur lui-même provoquait une division entière par zéro et un SIGFPE au moment de la compilation, ou bien il produisait une valeur arbitraire qui aurait court-circuité le message d’erreur que l’évaluateur doit émettre à l’exécution, message exigé par le cahier des charges. La solution consiste à interdire le pliage des seules opérations qui échoueraient, en laissant intact le sous-arbre concerné pour que l’évaluateur le rencontre normalement à l’exécution. C’est le rôle du test `can_fold`, invoqué avant tout pliage :

```

1 if (node->left->type == NODE_NUM && node->right->type == NODE_NUM
2     && can_fold(node->op, node->left->num, node->right->num)) {
3     Value v = do_fold(node->op, node->left->num, node->right->num);
4     /* ... le noeud BINOP est remplacé par le littéral résultat ... */
5 }
6 /* sinon : on laisse le noeud tel quel, l'évaluateur tranchera à l'exécution */

```

Listing 6.5 – Le pliage n’est tenté que si l’opération est sûre (extrait de `optim.c`).

Ce petit garde-fou illustre une idée importante : une optimisation ne doit jamais changer le comportement observable d’un programme, pas même ses messages d’erreur.

6.2.4 Détails d’environnement et de robustesse

Plusieurs petits obstacles, sans gloire mais instructifs, ont jalonné le travail. Je les mentionne car leur résolution méthodique a directement contribué à la propreté finale du logiciel.

Le premier est apparu dès la toute première compilation, sous la forme d’un avertissement déroutant sur la fonction `strdup` :

```

1 implicit declaration of function 'strdup'
2 assignment to 'char *' from 'int' makes pointer from integer without a cast

```

La cause n’est pas une faute de ma part mais une subtilité de la norme : `strdup` ne fait pas partie du C standard, c’est une extension POSIX. Compilé en `-std=c11` strict, l’en-tête masque son prototype, le compilateur suppose alors un retour de type `int`, et sur une machine 64 bits le pointeur se trouve tronqué. La correction tient en un mot dans le `Makefile` :

```

1 CFLAGS = -Wall -Wextra -std=gnu11 -g      # gnu11 expose les extensions POSIX

```

Le deuxième obstacle, encore plus discret, fut une ligne de dépendance du `Makefile` indentée par mégarde avec une espace, là où Make attend soit une tabulation pour une recette, soit aucune indentation pour une dépendance. Le troisième fut le tampon interne de Flex, jamais libéré, que Valgrind signalait comme « encore atteignable » et que l’appel `yylex_destroy()` en fin de `main` a fait disparaître. Le dernier, enfin, fut un débordement silencieux : le littéral `999999999` s’affichait `1410065407` parce que le lexer le convertissait avec `atoi`, limité à 32 bits, alors que mes valeurs sont des `long`. La correction a consisté à remplacer la conversion :

```

1 {CHIFFRE}+    { yylval.num = atoi(yytext); ... }    /* avant : tronque a 32 bits
    */
2 {CHIFFRE}+    { yylval.num = strtol(yytext, NULL, 10); ... } /* apres : 64 bits
    */

```

Listing 6.6 – Avant et après, dans `ilang.1`.

Aucun de ces points n’est spectaculaire pris isolément, mais c’est l’attention portée à chacun qui distingue un programme qui « marche sur les exemples » d’un programme réellement soigné.

6.3 L’extension : du parcours d’arbre au code natif

Le développement du back-end assembleur, bien que hors périmètre, mérite une discussion car il éclaire la nature même du projet.

6.3.1 Modèle et choix

Le générateur applique la convention `cdecl` et matérialise, en assembleur, ce que l’évaluateur réalisait abstraitement : la pile d’appels devient une vraie pile de cadres `%ebp`, les paramètres et locales des déplacements, les expressions une suite d’opérations sur `%eax` et la pile. Les structures de contrôle se traduisent en labels et sauts ; `arrêter` et `continuer` deviennent naturellement des `jmp` vers les labels de fin et de continuation de boucle.

6.3.2 Obstacles spécifiques

Trois difficultés propres à la cible native ont dû être traitées : l’installation de la chaîne 32 bits (`gcc-multilib`) ; un avertissement de l’éditeur de liens (`DT_TEXTREL`) dû à l’adressage absolu incompatible avec le PIE, résolu en liant avec `-no-pie` ; et l’alignement de la pile requis par `printf`, assuré par un `andl $-16, %esp` en tête de `main` et l’arrondi des cadres à 16 octets.

6.3.3 Enseignement

Voir le *même* AST se transformer tantôt en valeurs (interprétation), tantôt en instructions machine (compilation), rend tangible toute la théorie de la chaîne de compilation. C’est, de l’avis de l’auteur, la partie la plus formatrice du projet, et cela tient sans doute précisément au fait qu’elle dépassait la commande.

6.4 Compromis effectués

En synthèse, les principaux compromis assumés sont : la *lisibilité* au détriment de la performance, avec un parcours d’arbre plutôt qu’une machine à bytecode ; la *simplicité* au détriment de l’exhaustivité, sans portée de bloc, sans typage statique et avec un back-end limité aux entiers ; et la *fidélité à la spécification* partout où elle était claire. Là où le cahier des charges se contredisait lui-même, j’ai fait des choix explicites et documentés. Pour la portée, j’ai retenu le sens lexical conforme au terme employé plutôt que l’algorithme dynamique décrit. Pour le point-virgule,

présent dans la BNF mais absent des programmes de démonstration, j’ai préféré le rendre optionnel afin d’honorer les deux sources à la fois plutôt que d’en sacrifier une. Ces choix sont cohérents avec la finalité pédagogique du projet.

Chapitre 7

Conclusion et perspectives

7.1 Synthèse des contributions

Ce projet a abouti à un interpréteur complet et fonctionnel pour le mini-langage impératif ILANG. L'ensemble de la chaîne demandée a été réalisé : analyse lexicale par Flex, analyse syntaxique LALR(1) par Bison avec construction d'un arbre syntaxique abstrait, table des symboles gérant la portée par une pile d'environnements, deux passes d'optimisation, et un évaluateur récursif gérant variables, expressions, structures de contrôle, fonctions récursives et entrées-sorties. La gestion des erreurs couvre toutes les catégories du cahier des charges, chacune localisée à la ligne.

À cette base se sont ajoutées trois extensions, à savoir les nombres flottants, les instructions de rupture de boucle **arrêter** et **continuer**, et la localisation systématique des erreurs. S'y est jointe une exploration assumée hors du périmètre demandé, un second back-end générant du code assembleur i386, dont les sorties coïncident exactement avec celles de l'interpréteur. La qualité d'ingénierie visée a été tenue : zéro avertissement, zéro conflit grammatical, zéro fuite mémoire.

7.2 Prolongements possibles

Plusieurs directions permettraient d'enrichir le projet, et l'architecture retenue rend la plupart d'entre elles abordables sans refonte. Je les présente en indiquant à chaque fois ce qu'elles impliqueraient concrètement, car c'est cette estimation du coût qui distingue une vraie perspective d'un simple vœu.

Chaînes de caractères. Ajouter un type **string** serait sans doute l'extension la plus naturelle après les flottants, car elle suit exactement le même schéma. Il suffirait d'enrichir l'union étiquetée **Value** d'une troisième variante, et de gérer l'allocation et la libération des chaînes :

```
1 typedef enum { VAL_INT, VAL_FLOAT, VAL_STR } ValType;
2 typedef struct Value {
3     ValType type;
4     long    ival;
5     double  fval;
6     char    *sval;    /* nouveau : pointeur sur la chaîne (à libérer) */
```

```
7 } Value;
```

Listing 7.1 – Esquisse d’extension de `Value` aux chaînes.

La vraie difficulté ne serait pas le type lui-même mais la gestion de la durée de vie des chaînes, qui demanderait soit un comptage de références, soit un ramasse-miettes, pour éviter fuites et libérations prématurées.

Tableaux. Introduire des tableaux, avec une syntaxe du type `tab = [1, 2, 3]` et un accès indexé `tab[i]`, demanderait deux nouveaux types de nœuds dans l’AST, l’un pour le littéral de tableau et l’autre pour l’indexation, ainsi qu’une nouvelle variante de `Value` pointant vers un bloc alloué. L’évaluateur s’enrichirait de deux cas dans son `switch`, ce qui reste dans la logique du parcours d’arbre existant.

Machine virtuelle à bytecode. Pour dépasser les deux principales limites de l’interpréteur, la lenteur du parcours d’arbre et la profondeur de récursion bornée, la voie classique consiste à compiler l’AST vers un jeu d’instructions de bas niveau, exécuté par une boucle dédiée disposant de sa propre pile explicite. On définirait d’abord un jeu d’instructions :

```
1 typedef enum {
2     OP_PUSH, OP_LOAD, OP_STORE,      /* constantes et variables */
3     OP_ADD, OP_SUB, OP_MUL, OP_DIV,  /* arithmétique sur la pile */
4     OP_JMP, OP_JZ,                   /* sauts (contrôle de flot) */
5     OP_CALL, OP_RET, OP_PRINT        /* fonctions et affichage */
6 } ByteOp;
```

Listing 7.2 – Esquisse d’un jeu d’instructions de bytecode.

L’évaluation reviendrait alors à une boucle lisant ces instructions une à une. C’est l’approche de Python ou de Lua, et elle supprimerait du même coup la limite de récursion, puisque la pile d’appels deviendrait une structure de données gérée explicitement plutôt que la pile du langage hôte.

Back-end assembleur complet, vérification de types, afficheur d’AST. Trois autres prolongements me semblent intéressants. Le back-end assembleur pourrait être étendu aux flottants, au prix d’une gestion du jeu d’instructions flottantes du processeur, et reproduire la gestion « douce » de la division par zéro en émettant un test avant chaque division. Une passe d’analyse sémantique préalable pourrait signaler davantage d’erreurs avant même l’exécution, par exemple l’usage d’une variable jamais affectée. Enfin, un afficheur d’AST reconstituant le source avec un parenthésage minimal serait un outil précieux, aussi bien pour le débogage que pour l’enseignement, car il rendrait visible la structure que l’analyseur a réellement construite.

7.3 Applications potentielles

Au-delà de l’exercice, les techniques mises en œuvre dépassent largement la construction de langages généralistes. Elles s’appliquent à tout traitement de texte structuré : langages de

configuration, langages dédiés (*DSL*) métier, moteurs de calcul de formules (tableurs), analyseurs de protocoles, ou encore préprocesseurs et transpileurs. La compétence centrale, qui consiste à décrire formellement une syntaxe puis à lui associer une sémantique, est directement réutilisable dans tous ces contextes, bien au-delà du seul cas des langages de programmation généralistes.

7.4 Recommandations

Pour qui reprendrait ou referait un tel projet, trois recommandations se dégagent de l'expérience. Premièrement, **construire incrémentalement** : un cœur minimal qui fonctionne de bout en bout vaut mieux qu'un édifice complet jamais exécuté. Deuxièmement, **tester tôt et systématiquement**, y compris les cas limites et la mémoire (Valgrind), plutôt que de se fier aux seuls exemples heureux. Troisièmement, **exploiter les outils** plutôt que de les contourner : laisser Bison signaler les conflits, laisser le compilateur signaler les avertissements, et traiter chacun comme une information à comprendre.

7.5 Apports personnels et conclusion

Sur le plan des apprentissages, ce projet a transformé des notions jusque-là théoriques en savoir-faire concret. La distinction entre niveaux régulier et hors-contexte, la mécanique d'une grammaire LALR(1), le rôle d'un arbre syntaxique comme représentation pivot, la gestion d'une pile d'appels : autant de concepts qui prennent un sens entièrement nouveau une fois qu'on les a implémentés et débogués soi-même, plutôt que seulement lus dans un cours. Le moment le plus marquant aura été de voir le même arbre produire, selon le back-end choisi, soit un résultat calculé, soit des instructions machine : cette dualité résume à elle seule ce que le cours cherche à transmettre.

En définitive, ILANG n'est pas un langage destiné à être utilisé, mais un langage destiné à être *compris*. C'est à cette aune qu'il faut juger le projet : non à la richesse de ses fonctionnalités, mais à la clarté avec laquelle il rend visible le chemin qui mène d'un texte source à son exécution.

Bibliographie et webographie

Bibliographie

- [1] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers : Principles, Techniques, and Tools* (le « Dragon Book »), 2^e édition, Pearson/Addison-Wesley, 2006.
- [2] J. Levine, *flex & bison : Text Processing Tools*, O'Reilly Media, 2009.
- [3] *Flex : The Fast Lexical Analyzer Generator*, manuel de référence. <https://github.com/westes/flex> (consulté en 2026).
- [4] *GNU Bison Manual*, Free Software Foundation. <https://www.gnu.org/software/bison/manual/> (consulté en 2026).
- [5] D. E. Knuth, *On the Translation of Languages from Left to Right*, Information and Control, vol. 8, n° 6, p. 607–639, 1965.
- [6] N. Chomsky, *Three Models for the Description of Language*, IRE Transactions on Information Theory, vol. 2, n° 3, 1956.
- [7] M. E. Lesk, E. Schmidt, *Lex : A Lexical Analyzer Generator*, Bell Laboratories, Computing Science Technical Report n° 39, 1975.
- [8] S. C. Johnson, *Yacc : Yet Another Compiler-Compiler*, Bell Laboratories, Computing Science Technical Report n° 32, 1975.
- [9] ISO/IEC 9899 :2011, *Programming languages : C* (norme C11).
- [10] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer's Manual*, volume 2 (Instruction Set Reference).
- [11] *System V Application Binary Interface : Intel386 Architecture Processor Supplement* (convention d'appel cdecl).
- [12] P. Kislin-Duval, *Interprétation & Compilation*, support de cours, Licence Informatique, IED, Université Paris 8.

Annexe A

Code source complet

Cette annexe reproduit l'intégralité du code source du projet. Les fichiers générés automatiquement par Flex et Bison (`lex.yy.c`, `ilang.tab.c`, `ilang.tab.h`) ne sont pas reproduits, étant entièrement déduits de `ilang.l` et `ilang.y`.

A.1 ast.h

```
1 #ifndef AST_H
2 #define AST_H
3
4 /*
5  * ast.h - Definition de l'arbre syntaxique abstrait (AST) du langage ILANG.
6  *
7  * Chaque noeud est une structure C avec un type (NodeType) et un ensemble
8  * de fils. Tous les noeuds partagent la meme structure Node ; selon le type,
9  * seuls certains champs sont significatifs (cf. commentaires ci-dessous).
10 *
11 * Ce decoupage suit le cahier des charges (section 3.5).
12 */
13
14 /* ---- Valeur d'execution ----
15  * ILANG manipule deux types numeriques : des entiers et des flottants.
16  * Une Value est etiquetee (tagged union) : le champ type indique lequel des
17  * deux champs (ival ou fval) est significatif. Cela permet de conserver la
18  * semantique entiere d'origine (division entiere, modulo) tout en ajoutant
19  * le support des flottants sans casser l'existant. */
20 typedef enum { VAL_INT, VAL_FLOAT } ValType;
21
22 typedef struct Value {
23     ValType type;
24     long ival; /* significatif si type == VAL_INT */
25     double fval; /* significatif si type == VAL_FLOAT */
26 } Value;
27
28 /* Type d'un noeud de l'AST. */
29 typedef enum {
```

```

30     NODE_NUM,          /* littéral numerique (entier ou flottant) : num
                          */
31     NODE_VAR,          /* reference de variable      : name
                          */
32     NODE_BINOP,        /* operation binaire          : op, left, right
                          */
33     NODE_UNOP,         /* operation unaire (-)      : op, left
                          */
34     NODE_ASSIGN,       /* affectation                : name, left (= expression)
                          */
35     NODE_IF,           /* condition si/sinon        : cond, body, elsebody
                          */
36     NODE_WHILE,        /* boucle tant que           : cond, body
                          */
37     NODE_FOR,          /* boucle pour               : left(init), cond,
                          */
38                          /*                          right(incr), body
                          */
39     NODE_FUNC_DECL,    /* declaration de fonction   : name, args(params), nargs,
                          */
40                          /*                          body
                          */
41     NODE_FUNC_CALL,    /* appel de fonction         : name, args, nargs
                          */
42     NODE_PRINT,        /* afficher(expr)            : left
                          */
43     NODE_READ,         /* lire()                    : (aucun champ)
                          */
44     NODE_RETURN,       /* retourner(expr)           : left
                          */
45     NODE_BREAK,        /* arreter (sortie de boucle) : (aucun champ)
                          */
46     NODE_CONTINUE,     /* continuer (iteration suiv.): (aucun champ)
                          */
47     NODE_BLOCK         /* suite d'instructions      : args(statements), nargs
                          */
48 } NodeType;
49
50 /* Operateurs pour NODE_BINOP et NODE_UNOP. */
51 typedef enum {
52     OP_ADD, OP_SUB, OP_MUL, OP_DIV, OP_MOD, /* arithmetique */
53     OP_EQ, OP_NE, OP_LT, OP_GT, OP_LE, OP_GE, /* comparaison */
54     OP_NEG /* moins unaire (UMINUS) */
55 } OpType;
56
57 /* Noeud generique de l'AST. */
58 typedef struct Node {
59     NodeType type;
60
61     Value num; /* NODE_NUM (entier ou flottant) */
62     char *name; /* NODE_VAR, NODE_ASSIGN, NODE_FUNC_* */
63     int op; /* NODE_BINOP, NODE_UNOP (OpType) */

```



```

64
65     struct Node *left;      /* operande gauche / init de boucle / expr */
66     struct Node *right;     /* operande droit / incr de boucle */
67     struct Node *cond;      /* condition (if / while / for) */
68     struct Node *body;      /* corps (if-then / while / for / fonction) */
69     struct Node *elsebody;   /* corps du sinon */
70
71     struct Node **args;      /* NODE_FUNC_DECL (params), NODE_FUNC_CALL */
72                             /* (arguments), NODE_BLOCK (instructions) */
73     int nargs;              /* nombre d'elements dans args */
74
75     int line;               /* ligne source (pour les messages d'erreur)*/
76 } Node;
77
78 /* ---- Utilitaires sur les valeurs (definis dans ast.c) ---- */
79
80 Value make_int(long i);
81 Value make_float(double f);
82 int value_is_true(Value v); /* 0 = faux, tout le reste = vrai */
83 void value_print(Value v);  /* affiche la valeur + retour ligne */
84
85 /* ---- Constructeurs (definis dans ast.c) ---- */
86
87 Node *new_num(long value); /* litteral entier */
88 Node *new_fnum(double value); /* litteral flottant */
89 Node *new_var(char *name);
90 Node *new_binop(int op, Node *left, Node *right);
91 Node *new_unop(int op, Node *operand);
92 Node *new_assign(char *name, Node *expr);
93 Node *new_if(Node *cond, Node *body, Node *elsebody);
94 Node *new_while(Node *cond, Node *body);
95 Node *new_for(Node *init, Node *cond, Node *incr, Node *body);
96 Node *new_func_decl(char *name, Node **params, int nparams, Node *body);
97 Node *new_func_call(char *name, Node **args, int nargs);
98 Node *new_print(Node *expr);
99 Node *new_read(void);
100 Node *new_return(Node *expr);
101 Node *new_break(void);
102 Node *new_continue(void);
103 Node *new_block(void);
104
105 /* Ajoute une instruction a un noeud NODE_BLOCK (croissance dynamique). */
106 void block_add(Node *block, Node *stmt);
107
108 /* ---- Liste temporaire de noeuds (parametres / arguments) ----
109  * Utilisee uniquement par le parseur pour accumuler les listes de
110  * parametres et d'arguments avant de les transferer dans un noeud. */
111 typedef struct NodeList {
112     Node **items;
113     int count;
114 } NodeList;
115

```

```

116 NodeList *nodelist_new(void);
117 void nodelist_add(NodeList *list, Node *node);
118
119 /* Libere recursivement l'AST. */
120 void free_node(Node *node);
121
122 #endif /* AST_H */

```

A.2 ast.c

```

1  /*
2   * ast.c - Construction et liberation des noeuds de l'AST ILANG.
3   *
4   * Les constructeurs sont appeles depuis les actions semantiques de Bison
5   * (ilang.y). Chaque constructeur alloue un Node, l'initialise a zero puis
6   * renseigne uniquement les champs pertinents pour son type.
7   */
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12 #include "ast.h"
13
14 /* La ligne courante est maintenue par le lexer (yylineno via une variable
15  * exportee). On la recopie dans chaque noeud cree pour pouvoir afficher
16  * des messages d'erreur localises a l'evaluation. */
17 extern int yylineno;
18
19 /* Allocation de base : alloue un Node, le met a zero et fixe le type. */
20 static Node *new_node(NodeType type)
21 {
22     Node *n = (Node *)calloc(1, sizeof(Node));
23     if (n == NULL) {
24         fprintf(stderr, "Erreur fatale : memoire insuffisante (calloc)\n");
25         exit(1);
26     }
27     n->type = type;
28     n->line = yylineno;
29     return n;
30 }
31
32 /* ---- Utilitaires sur les valeurs ---- */
33
34 Value make_int(long i)
35 {
36     Value v;
37     v.type = VAL_INT;
38     v.ival = i;
39     v.fval = 0.0;
40     return v;
41 }

```

```

42
43 Value make_float(double f)
44 {
45     Value v;
46     v.type = VAL_FLOAT;
47     v.ival = 0;
48     v.fval = f;
49     return v;
50 }
51
52 int value_is_true(Value v)
53 {
54     if (v.type == VAL_INT) {
55         return v.ival != 0;
56     }
57     return v.fval != 0.0;
58 }
59
60 void value_print(Value v)
61 {
62     if (v.type == VAL_INT) {
63         printf("%ld\n", v.ival);
64     } else {
65         /* %g : notation compacte (6.28 reste 6.28, 3.0 s'affiche 3) */
66         printf("%g\n", v.fval);
67     }
68 }
69
70 /* ---- Constructeurs ---- */
71
72 Node *new_num(long value)
73 {
74     Node *n = new_node(NODE_NUM);
75     n->num = make_int(value);
76     return n;
77 }
78
79 Node *new_fnum(double value)
80 {
81     Node *n = new_node(NODE_NUM);
82     n->num = make_float(value);
83     return n;
84 }
85
86 Node *new_var(char *name)
87 {
88     Node *n = new_node(NODE_VAR);
89     n->name = name; /* deja duplique par le lexer (strdup) */
90     return n;
91 }
92
93 Node *new_binop(int op, Node *left, Node *right)

```

```
94 {
95     Node *n = new_node(NODE_BINOP);
96     n->op = op;
97     n->left = left;
98     n->right = right;
99     return n;
100 }
101
102 Node *new_unop(int op, Node *operand)
103 {
104     Node *n = new_node(NODE_UNOP);
105     n->op = op;
106     n->left = operand;
107     return n;
108 }
109
110 Node *new_assign(char *name, Node *expr)
111 {
112     Node *n = new_node(NODE_ASSIGN);
113     n->name = name;
114     n->left = expr;
115     return n;
116 }
117
118 Node *new_if(Node *cond, Node *body, Node *elsebody)
119 {
120     Node *n = new_node(NODE_IF);
121     n->cond = cond;
122     n->body = body;
123     n->elsebody = elsebody;
124     return n;
125 }
126
127 Node *new_while(Node *cond, Node *body)
128 {
129     Node *n = new_node(NODE_WHILE);
130     n->cond = cond;
131     n->body = body;
132     return n;
133 }
134
135 Node *new_for(Node *init, Node *cond, Node *incr, Node *body)
136 {
137     Node *n = new_node(NODE_FOR);
138     n->left = init;
139     n->cond = cond;
140     n->right = incr;
141     n->body = body;
142     return n;
143 }
144
145 Node *new_func_decl(char *name, Node **params, int nparams, Node *body)
```

```
146 {
147     Node *n = new_node(NODE_FUNC_DECL);
148     n->name = name;
149     n->args = params;
150     n->nargs = nparams;
151     n->body = body;
152     return n;
153 }
154
155 Node *new_func_call(char *name, Node **args, int nargs)
156 {
157     Node *n = new_node(NODE_FUNC_CALL);
158     n->name = name;
159     n->args = args;
160     n->nargs = nargs;
161     return n;
162 }
163
164 Node *new_print(Node *expr)
165 {
166     Node *n = new_node(NODE_PRINT);
167     n->left = expr;
168     return n;
169 }
170
171 Node *new_read(void)
172 {
173     return new_node(NODE_READ);
174 }
175
176 Node *new_return(Node *expr)
177 {
178     Node *n = new_node(NODE_RETURN);
179     n->left = expr;
180     return n;
181 }
182
183 Node *new_break(void)
184 {
185     return new_node(NODE_BREAK);
186 }
187
188 Node *new_continue(void)
189 {
190     return new_node(NODE_CONTINUE);
191 }
192
193 Node *new_block(void)
194 {
195     Node *n = new_node(NODE_BLOCK);
196     n->args = NULL;
197     n->nargs = 0;
```

```

198     return n;
199 }
200
201 void block_add(Node *block, Node *stmt)
202 {
203     if (stmt == NULL) {
204         return; /* instruction vide eliminee par l'optimiseur, p.ex. */
205     }
206     block->args = (Node **)realloc(block->args,
207                                   sizeof(Node *) * (block->nargs + 1));
208     if (block->args == NULL) {
209         fprintf(stderr, "Erreur fatale : memoire insuffisante (realloc)\n");
210         exit(1);
211     }
212     block->args[block->nargs] = stmt;
213     block->nargs++;
214 }
215
216 NodeList *nodelist_new(void)
217 {
218     NodeList *l = (NodeList *)malloc(sizeof(NodeList));
219     if (l == NULL) {
220         fprintf(stderr, "Erreur fatale : memoire insuffisante (nodelist)\n");
221         exit(1);
222     }
223     l->items = NULL;
224     l->count = 0;
225     return l;
226 }
227
228 void nodelist_add(NodeList *list, Node *node)
229 {
230     list->items = (Node **)realloc(list->items,
231                                   sizeof(Node *) * (list->count + 1));
232     if (list->items == NULL) {
233         fprintf(stderr, "Erreur fatale : memoire insuffisante (nodelist_add)\n");
234         ;
235         exit(1);
236     }
237     list->items[list->count] = node;
238     list->count++;
239 }
240
241 void free_node(Node *node)
242 {
243     int i;
244
245     if (node == NULL) {
246         return;
247     }
248     /* Liberation recursive de tous les fils possibles. */

```

```

249     free_node(node->left);
250     free_node(node->right);
251     free_node(node->cond);
252     free_node(node->body);
253     free_node(node->elsebody);
254
255     if (node->args != NULL) {
256         for (i = 0; i < node->nargs; i++) {
257             free_node(node->args[i]);
258         }
259         free(node->args);
260     }
261
262     if (node->name != NULL) {
263         free(node->name);
264     }
265
266     free(node);
267 }

```

A.3 symtab.h

```

1  #ifndef SYMTAB_H
2  #define SYMTAB_H
3
4  /*
5   * symtab.h - Table des symboles d'ILANG.
6   *
7   * Conformement au cahier des charges, la table des symboles combine deux
8   * idées :
9   *   - une TABLE DE HACHAGE pour stocker les variables (section 2.2) ;
10  *   - une PILE D'ENVIRONNEMENTS (frames) pour gerer la portee locale des
11  *     fonctions, a la maniere de la pile %ebp en assembleur (section 3.6).
12  *
13  * Chaque frame de la pile est une table de hachage independante. On empile
14  * un frame a l'entree d'une fonction, on le depile a la sortie. La base de
15  * la pile est l'environnement global du programme principal. Le champ parent
16  * sert de lien de controle (restauration du frame courant a la sortie de
17  * fonction) ; la resolution des variables, elle, est LEXICALE : frame courant
18  * puis environnement global, sans passer par les frames intermediaires (voir
19  * env_get).
20  *
21  * Les fonctions, elles, sont globales : elles sont rangees dans une table
22  * separee (func table) accessible depuis n'importe quel frame.
23  */
24
25  #include "ast.h"
26
27  #define HASH_SIZE 211    /* nombre premier : limite les collisions */
28
29  /* Une entree (variable) dans une table de hachage. */

```

```

30 typedef struct Symbol {
31     char *name;
32     Value value;           /* entier ou flottant */
33     struct Symbol *next;   /* chainage en cas de collision */
34 } Symbol;
35
36 /* Un environnement = un frame de la pile = une table de hachage. */
37 typedef struct Env {
38     Symbol *buckets[HASH_SIZE];
39     struct Env *parent;    /* frame precedent dans la pile (NULL = global) */
40 } Env;
41
42 /* ---- Gestion de la pile d'environnements ---- */
43
44 void env_init(void);       /* cree l'environnement global */
45 void env_push(void);       /* empile un nouveau frame (entree fonction) */
46 void env_pop(void);        /* depile le frame courant (sortie fonction) */
47
48 /* Recherche une variable (portee lexicale). Renvoie 1 si trouvee (et place la
49  * valeur dans *out), 0 sinon. La recherche regarde le frame courant puis, en
50  * repli, l'environnement global ; elle ignore les frames intermediaires de la
51  * pile d'appel (une fonction ne voit pas les locales de son appelant). */
52 int env_get(const char *name, Value *out);
53
54 /* Affecte une variable dans le frame COURANT (sommet de pile).
55  * Cree la variable si elle n'existe pas encore dans ce frame : c'est ce
56  * qui rend les variables locales aux fonctions et globales au programme
57  * principal (dont le frame courant est l'environnement global). */
58 void env_set(const char *name, Value value);
59
60 /* ---- Gestion de la table des fonctions (globale) ---- */
61
62 void func_define(Node *decl); /* enregistre une NODE_FUNC_DECL */
63 Node *func_lookup(const char *name); /* NULL si fonction inconnue */
64
65 /* Liberation complete (variables + fonctions). */
66 void symtab_free(void);
67
68 #endif /* SYMTAB_H */

```

A.4 symtab.c

```

1  /*
2   * symtab.c - Implementation de la table des symboles ILANG.
3   *
4   * Voir symtab.h pour le modele d'ensemble (pile de frames, chaque frame
5   * etant une table de hachage ; table des fonctions globale separee).
6   */
7
8  #include <stdio.h>
9  #include <stdlib.h>

```



```

10 #include <string.h>
11 #include "syntab.h"
12
13 /* Sommet de la pile d'environnements (frame courant). */
14 static Env *current_env = NULL;
15
16 /* Base de la pile : l'environnement global du programme principal. Il sert de
17  * portee de repli pour la resolution des variables (voir env_get). */
18 static Env *global_env = NULL;
19
20 /* Tete de la liste chainee des fonctions definies (table globale). */
21 typedef struct FuncEntry {
22     char *name;
23     Node *decl;
24     struct FuncEntry *next;
25 } FuncEntry;
26
27 static FuncEntry *func_list = NULL;
28
29 /* Fonction de hachage djb2 (Dan Bernstein), classique et efficace. */
30 static unsigned int hash(const char *s)
31 {
32     unsigned int h = 5381;
33     int c;
34     while ((c = (unsigned char)*s++) != 0) {
35         h = ((h << 5) + h) + c; /* h * 33 + c */
36     }
37     return h % HASH_SIZE;
38 }
39
40 void env_init(void)
41 {
42     current_env = (Env *)calloc(1, sizeof(Env));
43     if (current_env == NULL) {
44         fprintf(stderr, "Erreur fatale : memoire insuffisante (env_init)\n");
45         exit(1);
46     }
47     current_env->parent = NULL; /* base de la pile = environnement global */
48     global_env = current_env; /* memorise la base pour la portee lexicale */
49 }
50
51 void env_push(void)
52 {
53     Env *e = (Env *)calloc(1, sizeof(Env));
54     if (e == NULL) {
55         fprintf(stderr, "Erreur fatale : memoire insuffisante (env_push)\n");
56         exit(1);
57     }
58     e->parent = current_env;
59     current_env = e;
60 }
61

```

```

62 void env_pop(void)
63 {
64     Env *old = current_env;
65     int i;
66
67     if (old == NULL) {
68         return;
69     }
70
71     /* Libérer toutes les variables de ce frame. */
72     for (i = 0; i < HASH_SIZE; i++) {
73         Symbol *s = old->buckets[i];
74         while (s != NULL) {
75             Symbol *next = s->next;
76             free(s->name);
77             free(s);
78             s = next;
79         }
80     }
81
82     current_env = old->parent;
83     free(old);
84 }
85
86 /* Cherche un nom dans un frame donne uniquement. Renvoie 1 si trouve. */
87 static int lookup_in(Env *e, const char *name, Value *out)
88 {
89     Symbol *s;
90     if (e == NULL) {
91         return 0;
92     }
93     for (s = e->buckets[hash(name)]; s != NULL; s = s->next) {
94         if (strcmp(s->name, name) == 0) {
95             *out = s->value;
96             return 1;
97         }
98     }
99     return 0;
100 }
101
102 int env_get(const char *name, Value *out)
103 {
104     /* Portée LEXICALE : on cherche d'abord dans le frame courant (les locales
105      * de la fonction en cours, ou les globales si l'on est dans le programme
106      * principal), puis en repli dans l'environnement global. On ne consulte
107      * PAS les frames intermediaires de la pile d'appel : une fonction ne voit
108      * donc jamais les variables locales de son appelant. */
109     if (lookup_in(current_env, name, out)) {
110         return 1;
111     }
112     if (current_env != global_env && lookup_in(global_env, name, out)) {
113         return 1;

```

```

114     }
115     return 0;
116 }
117
118 void env_set(const char *name, Value value)
119 {
120     unsigned int idx = hash(name);
121     Symbol *s = current_env->buckets[idx];
122
123     /* La variable existe-t-elle déjà dans le frame courant ? */
124     while (s != NULL) {
125         if (strcmp(s->name, name) == 0) {
126             s->value = value;    /* mise à jour */
127             return;
128         }
129         s = s->next;
130     }
131
132     /* Sinon, création dans le frame courant (variable locale/globale). */
133     s = (Symbol *)malloc(sizeof(Symbol));
134     if (s == NULL) {
135         fprintf(stderr, "Erreur fatale : memoire insuffisante (env_set)\n");
136         exit(1);
137     }
138     s->name = strdup(name);
139     s->value = value;
140     s->next = current_env->buckets[idx];
141     current_env->buckets[idx] = s;
142 }
143
144 void func_define(Node *decl)
145 {
146     FuncEntry *f;
147
148     /* Redefinition : on remplace simplement le corps existant. */
149     for (f = func_list; f != NULL; f = f->next) {
150         if (strcmp(f->name, decl->name) == 0) {
151             f->decl = decl;
152             return;
153         }
154     }
155
156     f = (FuncEntry *)malloc(sizeof(FuncEntry));
157     if (f == NULL) {
158         fprintf(stderr, "Erreur fatale : memoire insuffisante (func_define)\n");
159         exit(1);
160     }
161     f->name = strdup(decl->name);
162     f->decl = decl;
163     f->next = func_list;
164     func_list = f;
165 }

```

```

166
167 Node *func_lookup(const char *name)
168 {
169     FuncEntry *f;
170     for (f = func_list; f != NULL; f = f->next) {
171         if (strcmp(f->name, name) == 0) {
172             return f->decl;
173         }
174     }
175     return NULL;
176 }
177
178 void symtab_free(void)
179 {
180     FuncEntry *f;
181
182     /* Depiler tous les frames restants (normalement il ne reste que le
183      * global, mais on est robuste). */
184     while (current_env != NULL) {
185         env_pop();
186     }
187
188     /* Libérer la table des fonctions (les Node sont libérées via free_node
189      * sur la racine de l'AST, on ne libère ici que les entrées). */
190     f = func_list;
191     while (f != NULL) {
192         FuncEntry *next = f->next;
193         free(f->name);
194         free(f);
195         f = next;
196     }
197     func_list = NULL;
198 }

```

A.5 ilang.l : analyseur lexical (Flex)

```

1 /*
2  * ilang.l - Analyseur lexical (Flex) du langage ILANG.
3  *
4  * Reconnaît les nombres, identifiants, mots-cles, operateurs et delimitateurs,
5  * et ignore les espaces et les commentaires (ligne // et bloc /* ... */).
6  * Les tokens sont ceux déclarés par Bison dans ilang.tab.h.
7  */
8
9 %{
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <string.h>
13 #include "ast.h"
14 #include "ilang.tab.h" /* genere par bison : definitions des tokens + yylval
15 */

```

```

15 %}
16
17 /* yylineno : numero de ligne automatique (utilise pour les messages).
18  * noyywrap : pas de fonction yywrap(), un seul fichier d'entree.
19  * nounput / noinput : evite des warnings sur des fonctions non utilisees. */
20 %option yylineno noyywrap nounput noinput
21
22 /* Etat exclusif pour les commentaires de bloc. */
23 %x COMMENT
24
25 CHIFFRE      [0-9]
26 LETTRE       [a-zA-Z_]
27 ID           {LETTRE}({LETTRE}|{CHIFFRE})*
28
29 %%
30
31 /* ---- Espaces (ignorees) ---- */
32 [ \t\r]+      { /* rien */ }
33 \n            { /* yylineno gere par %option yylineno */ }
34
35 /* ---- Commentaires ---- */
36 "/" "[^\n]*    { /* commentaire de ligne ignore */ }
37 "/*"          { BEGIN(COMMENT); }
38 <COMMENT>"*/"  { BEGIN(INITIAL); }
39 <COMMENT>\n    { /* compte la ligne, contenu ignore */ }
40 <COMMENT>.     { /* contenu du commentaire ignore */ }
41 <COMMENT><<EOF>> { fprintf(stderr,
42                      "Erreur lexicale ligne %d : commentaire de bloc "
43                      "non termine\n", yylineno);
44                      exit(1); }
45
46 /* ---- Mots-cles ---- */
47 /* "tant que" comporte un espace : on l'attrape avant l'identifiant. */
48 "tant" [ \t]+"que" { return TANTQUE; }
49 "si"    { return SI; }
50 "sinon" { return SINON; }
51 "pour"  { return POUR; }
52 "fonction" { return FONCTION; }
53 "retourner" { return RETOURNER; }
54 "afficher" { return AFFICHER; }
55 "lire"    { return LIRE; }
56 /* Ruptures de boucle. "arreter" accepte avec ou sans accent circonflexe
57  * (le fichier source peut etre en UTF-8 ou en ASCII). */
58 "arr"("ê"|"e")"ter" { return ARRETER; }
59 "continuer" { return CONTINUER; }
60
61 /* ---- Operateurs de comparaison (avant les operateurs simples) ---- */
62 "==" { return EQ; }
63 "!=" { return NE; }
64 "<=" { return LE; }
65 ">=" { return GE; }
66 "<"  { return LT; }

```

```

67 ">"                { return GT; }
68
69  /* ---- Operateurs arithmetiques et delimitateurs (tokens char) ---- */
70 "+"                { return '+'; }
71 "-"                { return '-'; }
72 "*"                { return '*'; }
73 "/"                { return '/'; }
74 "%"                { return '%'; }
75 "="                { return '='; }
76 "("                { return '('; }
77 ")"                { return ')'; }
78 "{"                { return '{'; }
79 "}"                { return '}'; }
80 ";"                { return ';'; }
81 ","                { return ','; }
82
83  /* ---- Litteraux et identifiants ---- */
84  /* Le flottant est teste avant l'entier. Formes acceptees : 3.14, 3.
85   * (point final) et .5 (point initial). */
86 ({CHIFFRE}+"."{CHIFFRE}*)|(""{CHIFFRE}+) { yylval.fnum = strtod(yytext, NULL);
      return FLOTTANT; }
87 {CHIFFRE}+         { yylval.num = strtol(yytext, NULL, 10); return NOMBRE; }
88 {ID}                { yylval.str = strdup(yytext); return IDENTIFIANT; }
89
90  /* ---- Caractere invalide ---- */
91 .                  { fprintf(stderr,
92                          "Erreur lexicale ligne %d : caractere invalide "
93                          "'%s'\n", yylineno, yytext);
94                          exit(1); }
95
96 %%

```

A.6 ilang.y : analyseur syntaxique (Bison)

```

1  /*
2   * ilang.y - Analyseur syntaxique (Bison) du langage ILANG.
3   *
4   * Grammaire LALR(1) qui construit l'arbre syntaxique abstrait (AST) via les
5   * constructeurs definis dans ast.c. La precedence des operateurs est fixee
6   * par les directives %left / %right (section 2.3 du cahier des charges).
7   *
8   * Note sur le "dangling else" : contrairement au cas classique du cours, il
9   * n'y a ici AUCUN conflit shift/reduce. La raison est que les corps de SI
10  * sont obligatoirement entre accolades (regle "bloc"). Un SINON est donc
11  * toujours precede d'un '}', si bien que l'etat d'analyse n'est jamais
12  * ambigu : SINON n'appartient pas a l'ensemble FOLLOW de la reduction du SI
13  * sans sinon. La grammaire se compile sans conflit, sans aucune directive de
14  * precedence dediee (verifie en retirant toute resolution : bison reste muet).
15  */
16
17 %{

```

```

18 #include <stdio.h>
19 #include <stdlib.h>
20 #include <string.h>
21 #include "ast.h"
22
23 /* Fournis par Flex / cette unite. */
24 int yylex(void);
25 void yyerror(const char *s);
26 extern int yylineno;
27
28 /* Racine de l'AST, renseignée par la règle "programme" et lue par main.c. */
29 Node *program_root = NULL;
30 %}
31
32 /* Le type des valeurs sémantiques utilise Node* et NodeList* : ast.h doit
33 * donc être inclus dans l'en-tête genere (ilang.tab.h) avant l'union. */
34 %code requires {
35     #include "ast.h"
36 }
37
38 %union {
39     long num;
40     double fnum;
41     char *str;
42     Node *node;
43     NodeList *list;
44 }
45
46 /* Messages d'erreur détaillés (token attendu vs reçu). */
47 %define parse.error verbose
48
49 /* ---- Tokens ---- */
50 %token <num> NOMBRE
51 %token <fnum> FLOTTANT
52 %token <str> IDENTIFIANT
53 %token SI SINON TANTQUE POUR FONCTION RETOURNER AFFICHER LIRE
54 %token ARRETER CONTINUER
55 %token EQ NE LT GT LE GE
56
57 /* ---- Types des non-terminaux ---- */
58 %type <node> instruction expression bloc liste_instructions
59 %type <node> affectation si_alors tant_que pour
60 %type <node> declaration_fonction appel_fonction
61 %type <list> liste_params liste_args params_nonvide args_nonvide
62
63 /* ---- Precedences (de la plus faible à la plus forte) ---- */
64 %left EQ NE LT GT LE GE
65 %left '+' '-'
66 %left '*' '/' '%'
67 %right UMINUS /* moins unaire */
68
69 %start programme

```

```

70
71 %%
72
73 programme
74     : liste_instructions          { program_root = $1; }
75     ;
76
77 liste_instructions
78     : /* vide */                  { $$ = new_block(); }
79     | liste_instructions instruction { block_add($1, $2); $$ = $1; }
80     ;
81
82 bloc
83     : '{' liste_instructions '}'   { $$ = $2; }
84     ;
85
86 instruction
87     : affectation osemi            { $$ = $1; }
88     | si_alors                    { $$ = $1; }
89     | tant_que                    { $$ = $1; }
90     | pour                        { $$ = $1; }
91     | declaration_fonction        { $$ = $1; }
92     | appel_fonction osemi         { $$ = $1; }
93     | AFFICHER '(' expression ')' osemi { $$ = new_print($3); }
94     | RETOURNER '(' expression ')' osemi { $$ = new_return($3); }
95     | ARRETER osemi                { $$ = new_break(); }
96     | CONTINUER osemi              { $$ = new_continue(); }
97     ;
98
99     /* Point-virgule optionnel en fin d'instruction : la grammaire BNF du
100     * cahier des charges (3.3) le fait figurer apres affectation, appel,
101     * afficher et retourner, mais les programmes de demonstration (4) n'en
102     * mettent aucun. On accepte donc les deux styles. */
103 osemi
104     : /* vide */
105     | ';'
106     ;
107
108 affectation
109     : IDENTIFIANT '=' expression   { $$ = new_assign($1, $3); }
110     ;
111
112 si_alors
113     : SI '(' expression ')' bloc
114         { $$ = new_if($3, $5, NULL); }
115     | SI '(' expression ')' bloc SINON bloc
116         { $$ = new_if($3, $5, $7); }
117     ;
118
119 tant_que
120     : TANTQUE '(' expression ')' bloc { $$ = new_while($3, $5); }
121     ;

```



```

122
123 pour
124     : POUR '(' affectation ';' expression ';' affectation ')' bloc
125         { $$ = new_for($3, $5, $7, $9); }
126     ;
127
128 declaration_fonction
129     : FONCTION IDENTIFIANT '(' liste_params ')' bloc
130         {
131             $$ = new_func_decl($2, $4->items, $4->count, $6);
132             free($4); /* le tableau items est transfere au noeud */
133         }
134     ;
135
136 appel_fonction
137     : IDENTIFIANT '(' liste_args ')'
138         {
139             $$ = new_func_call($1, $3->items, $3->count);
140             free($3);
141         }
142     ;
143
144 liste_params
145     : /* vide */ { $$ = nodelist_new(); }
146     | params_nonvide { $$ = $1; }
147     ;
148
149 params_nonvide
150     : IDENTIFIANT
151         { $$ = nodelist_new(); nodelist_add($$, new_var($1)); }
152     | params_nonvide ',' IDENTIFIANT
153         { nodelist_add($1, new_var($3)); $$ = $1; }
154     ;
155
156 liste_args
157     : /* vide */ { $$ = nodelist_new(); }
158     | args_nonvide { $$ = $1; }
159     ;
160
161 args_nonvide
162     : expression
163         { $$ = nodelist_new(); nodelist_add($$, $1); }
164     | args_nonvide ',' expression
165         { nodelist_add($1, $3); $$ = $1; }
166     ;
167
168 expression
169     : expression '+' expression { $$ = new_binop(OP_ADD, $1, $3); }
170     | expression '-' expression { $$ = new_binop(OP_SUB, $1, $3); }
171     | expression '*' expression { $$ = new_binop(OP_MUL, $1, $3); }
172     | expression '/' expression { $$ = new_binop(OP_DIV, $1, $3); }
173     | expression '%' expression { $$ = new_binop(OP_MOD, $1, $3); }

```

```

174 | expression EQ expression      { $$ = new_binop(OP_EQ, $1, $3); }
175 | expression NE expression      { $$ = new_binop(OP_NE, $1, $3); }
176 | expression LT expression      { $$ = new_binop(OP_LT, $1, $3); }
177 | expression GT expression      { $$ = new_binop(OP_GT, $1, $3); }
178 | expression LE expression      { $$ = new_binop(OP_LE, $1, $3); }
179 | expression GE expression      { $$ = new_binop(OP_GE, $1, $3); }
180 | '-' expression %prec UMINUS   { $$ = new_unop(OP_NEG, $2); }
181 | '(' expression ')'           { $$ = $2; }
182 | NOMBRE                       { $$ = new_num($1); }
183 | FLOTTANT                     { $$ = new_fnum($1); }
184 | IDENTIFIANT                  { $$ = new_var($1); }
185 | appel_fonction                { $$ = $1; }
186 | LIRE '(' ')'                 { $$ = new_read(); }
187 ;
188
189 %%
190
191 void yyerror(const char *s)
192 {
193     fprintf(stderr, "Erreur de syntaxe ligne %d : %s\n", yylineno, s);
194 }

```

A.7 eval.c : évaluateur

```

1  /*
2   * eval.c - Evalueur recursif de l'AST ILANG.
3   *
4   * L'évaluateur parcourt l'arbre en profondeur :
5   *   - post-order pour les expressions (on evalue les fils avant le noeud) ;
6   *   - pre-order pour les structures de controle (on teste la condition puis
7   *     on descend dans le corps).
8   *
9   * Valeurs : chaque expression s'évalue en une Value etiquetee entier/flottant
10  * (cf. ast.h). Les operations entre deux entiers restent entieres (division
11  * entiere, modulo) ; des qu'un flottant intervient, le calcul bascule en
12  * flottant. Le modulo n'est defini que sur les entiers.
13  *
14  * Controle de flot non local : trois signaux globaux portent les ruptures
15  * de flot :
16  *   - return_flag / return_value : "retourner" dans une fonction ;
17  *   - loop_signal : "arreter" (break) et "continuer" (continue) dans une
18  *     boucle.
19  * Un bloc arrete d'exécuter ses instructions des qu'un de ces signaux est
20  * leve ; les boucles consomment loop_signal, l'appel de fonction consomme
21  * return_flag.
22  */
23
24 #include <stdio.h>
25 #include <stdlib.h>
26 #include "ast.h"
27 #include "syntab.h"

```

```

28
29 /* Prototypes publics (appeles par main.c). */
30 Value eval(Node *node);
31 void eval_program(Node *root);
32
33 /* Retour de fonction. */
34 static int return_flag = 0;
35 static Value return_value;
36
37 /* Rupture de boucle. */
38 typedef enum { SIG_NONE, SIG_BREAK, SIG_CONTINUE } LoopSignal;
39 static LoopSignal loop_signal = SIG_NONE;
40
41 /* Profondeur de boucle dans la fonction courante : permet de detecter un
42 * "arreter"/"continuer" utilise hors d'une boucle. Remis a zero a l'entree
43 * de chaque fonction (une boucle de l'appelant ne "couvre" pas l'appelle). */
44 static int loop_depth = 0;
45
46 /* Conversion utilitaire vers double (pour les calculs mixtes). */
47 static double as_double(Value v)
48 {
49     return (v.type == VAL_INT) ? (double)v.ival : v.fval;
50 }
51
52 /* Evaluation d'une operation binaire. */
53 static Value eval_binop(Node *node)
54 {
55     Value a = eval(node->left);
56     Value b = eval(node->right);
57     int both_int = (a.type == VAL_INT && b.type == VAL_INT);
58
59     switch (node->op) {
60
61         /* --- Arithmetique --- */
62         case OP_ADD:
63             return both_int ? make_int(a.ival + b.ival)
64                             : make_float(as_double(a) + as_double(b));
65         case OP_SUB:
66             return both_int ? make_int(a.ival - b.ival)
67                             : make_float(as_double(a) - as_double(b));
68         case OP_MUL:
69             return both_int ? make_int(a.ival * b.ival)
70                             : make_float(as_double(a) * as_double(b));
71         case OP_DIV:
72             if (both_int) {
73                 if (b.ival == 0) {
74                     fprintf(stderr, "Erreur ligne %d : division par zero\n",
75                             node->line);
76                     return make_int(0);
77                 }
78                 return make_int(a.ival / b.ival);
79             } else {

```

```

80     double db = as_double(b);
81     if (db == 0.0) {
82         fprintf(stderr, "Erreur ligne %d : division par zero\n",
83             node->line);
84         return make_float(0.0);
85     }
86     return make_float(as_double(a) / db);
87 }
88 case OP_MOD:
89     if (!both_int) {
90         fprintf(stderr, "Erreur ligne %d : modulo non defini sur les "
91             "flottants\n", node->line);
92         return make_int(0);
93     }
94     if (b.ival == 0) {
95         fprintf(stderr, "Erreur ligne %d : modulo par zero\n", node->line);
96         return make_int(0);
97     }
98     return make_int(a.ival % b.ival);
99
100 /* --- Comparaisons (resultat toujours entier 0/1) --- */
101 case OP_EQ:
102     return make_int(both_int ? (a.ival == b.ival)
103         : (as_double(a) == as_double(b)));
104 case OP_NE:
105     return make_int(both_int ? (a.ival != b.ival)
106         : (as_double(a) != as_double(b)));
107 case OP_LT:
108     return make_int(both_int ? (a.ival < b.ival)
109         : (as_double(a) < as_double(b)));
110 case OP_GT:
111     return make_int(both_int ? (a.ival > b.ival)
112         : (as_double(a) > as_double(b)));
113 case OP_LE:
114     return make_int(both_int ? (a.ival <= b.ival)
115         : (as_double(a) <= as_double(b)));
116 case OP_GE:
117     return make_int(both_int ? (a.ival >= b.ival)
118         : (as_double(a) >= as_double(b)));
119
120 default:
121     fprintf(stderr, "Erreur interne : operateur binaire inconnu (%d)\n",
122         node->op);
123     exit(1);
124 }
125 }
126
127 /* Evaluation d'un appel de fonction (avec gestion de la pile de frames). */
128 static Value eval_func_call(Node *node)
129 {
130     Node *decl = func_lookup(node->name);
131     Value *argvals;

```

```

132     Value result;
133     int i;
134
135     if (decl == NULL) {
136         fprintf(stderr, "Erreur ligne %d : fonction '%s' non definie\n",
137             node->line, node->name);
138         exit(1);
139     }
140
141     if (node->nargs != decl->nargs) {
142         fprintf(stderr,
143             "Erreur ligne %d : fonction '%s' attend %d argument(s), "
144             "%d fourni(s)\n",
145             node->line, node->name, decl->nargs, node->nargs);
146         exit(1);
147     }
148
149     /* 1) Evaluer les arguments dans l'environnement de l'APPELANT. */
150     argvals = NULL;
151     if (node->nargs > 0) {
152         argvals = (Value *)malloc(sizeof(Value) * node->nargs);
153         if (argvals == NULL) {
154             fprintf(stderr, "Erreur fatale : memoire insuffisante (appel)\n");
155             exit(1);
156         }
157         for (i = 0; i < node->nargs; i++) {
158             argvals[i] = eval(node->args[i]);
159         }
160     }
161
162     /* 2) Empiler un nouveau frame et y lier les parametres. */
163     env_push();
164     for (i = 0; i < decl->nargs; i++) {
165         env_set(decl->args[i]->name, argvals[i]);
166     }
167     free(argvals);
168
169     /* 3) Evaluer le corps. Le drapeau de retour est remis a zero juste
170      * avant, puis consomme juste apres. La profondeur de boucle est
171      * sauvegardee puis remise a zero : un "arreter" dans cette fonction
172      * ne doit pas s'appliquer a une boucle de l'appelant. */
173     {
174         int saved_depth = loop_depth;
175         loop_depth = 0;
176         return_flag = 0;
177         eval(decl->body);
178         result = return_flag ? return_value : make_int(0);
179         return_flag = 0;
180         loop_depth = saved_depth;
181     }
182
183     /* 4) Depiler le frame (les variables locales disparaissent). */

```

```

184     env_pop();
185
186     return result;
187 }
188
189 Value eval(Node *node)
190 {
191     Value v;
192     int i;
193
194     if (node == NULL) {
195         return make_int(0);
196     }
197
198     switch (node->type) {
199
200     case NODE_NUM:
201         return node->num;
202
203     case NODE_VAR:
204         if (!env_get(node->name, &v)) {
205             fprintf(stderr, "Erreur ligne %d : variable '%s' non definie\n",
206                     node->line, node->name);
207             exit(1);
208         }
209         return v;
210
211     case NODE_BINOP:
212         return eval_binop(node);
213
214     case NODE_UNOP:
215         /* Seul le moins unaire existe. */
216         v = eval(node->left);
217         return (v.type == VAL_INT) ? make_int(-v.ival) : make_float(-v.fval);
218
219     case NODE_ASSIGN:
220         v = eval(node->left);
221         env_set(node->name, v);
222         return v;
223
224     case NODE_IF:
225         if (value_is_true(eval(node->cond))) {
226             eval(node->body);
227         } else if (node->elsebody != NULL) {
228             eval(node->elsebody);
229         }
230         return make_int(0);
231
232     case NODE_WHILE:
233         loop_depth++;
234         while (value_is_true(eval(node->cond))) {
235             eval(node->body);

```

```

236         if (return_flag) {
237             break;
238         }
239         if (loop_signal == SIG_BREAK) {
240             loop_signal = SIG_NONE;
241             break;
242         }
243         if (loop_signal == SIG_CONTINUE) {
244             loop_signal = SIG_NONE;
245             continue;
246         }
247     }
248     loop_depth--;
249     return make_int(0);
250
251 case NODE_FOR:
252     loop_depth++;
253     for (eval(node->left);
254          value_is_true(eval(node->cond));
255          eval(node->right)) {
256         eval(node->body);
257         if (return_flag) {
258             break;
259         }
260         if (loop_signal == SIG_BREAK) {
261             loop_signal = SIG_NONE;
262             break;
263         }
264         if (loop_signal == SIG_CONTINUE) {
265             loop_signal = SIG_NONE;
266             continue; /* execute quand meme l'increment */
267         }
268     }
269     loop_depth--;
270     return make_int(0);
271
272 case NODE_FUNC_DECL:
273     /* Enregistrement de la fonction quand on la rencontre dans le flot
274      * d'instructions (les fonctions sont definies avant usage dans les
275      * programmes de demonstration). */
276     func_define(node);
277     return make_int(0);
278
279 case NODE_FUNC_CALL:
280     return eval_func_call(node);
281
282 case NODE_PRINT:
283     v = eval(node->left);
284     value_print(v);
285     return make_int(0);
286
287 case NODE_READ:

```

```

288     {
289         long x;
290         if (scanf("%ld", &x) != 1) {
291             x = 0;    /* entree invalide ou fin de flux */
292         }
293         return make_int(x);
294     }
295
296     case NODE_RETURN:
297         return_value = eval(node->left);
298         return_flag = 1;
299         return return_value;
300
301     case NODE_BREAK:
302         if (loop_depth == 0) {
303             fprintf(stderr, "Erreur ligne %d : 'arreter' utilise hors d'une "
304                          "boucle\n", node->line);
305             exit(1);
306         }
307         loop_signal = SIG_BREAK;
308         return make_int(0);
309
310     case NODE_CONTINUE:
311         if (loop_depth == 0) {
312             fprintf(stderr, "Erreur ligne %d : 'continuer' utilise hors d'une "
313                          "boucle\n", node->line);
314             exit(1);
315         }
316         loop_signal = SIG_CONTINUE;
317         return make_int(0);
318
319     case NODE_BLOCK:
320         for (i = 0; i < node->nargs; i++) {
321             eval(node->args[i]);
322             /* On stoppe le bloc des qu'une rupture de flot est declenchee. */
323             if (return_flag || loop_signal != SIG_NONE) {
324                 break;
325             }
326         }
327         return make_int(0);
328
329     default:
330         fprintf(stderr, "Erreur interne : type de noeud inconnu (%d)\n",
331                node->type);
332         exit(1);
333 }
334 }
335
336 void eval_program(Node *root)
337 {
338     int i;
339

```



```

340  /* Pre-passe : on enregistre toutes les fonctions declarees au premier
341  * niveau du programme AVANT de l'executer. Cela autorise l'appel d'une
342  * fonction definie plus loin dans le source (reference avant declaration)
343  * ainsi que la recursion croisee, sans imposer un ordre de declaration. */
344  if (root != NULL && root->type == NODE_BLOCK) {
345      for (i = 0; i < root->nargs; i++) {
346          if (root->args[i]->type == NODE_FUNC_DECL) {
347              func_define(root->args[i]);
348          }
349      }
350  }
351
352  eval(root);
353  }

```

A.8 optim.c : optimisations

```

1  /*
2  * optim.c - Optimisations sur l'AST (chapitre 8 du cours).
3  *
4  * Deux passes, appliquees avant l'evaluation et fusionnees ici en un seul
5  * parcours recursif (un seul passage sur l'arbre) :
6  *
7  * 1. Pliage de constantes (constant folding) : un noeud BINOP dont les
8  *    deux fils sont des litteraux est remplace par le litteral resultat.
9  *    Exemple : 3 + 4 -> 7, ou 3.14 * 2 -> 6.28. Idem pour le moins
10  *    unaire : -(5) -> -5.
11  *
12  * 2. Elimination du code mort (dead code elimination) : un NODE_IF dont
13  *    la condition est un litteral faux (0 ou 0.0) est supprime ; il est
14  *    remplace par sa branche "sinon" (ou par rien). Symetriquement, si la
15  *    condition est un litteral vrai, on garde directement le "alors".
16  *
17  * optimize() renvoie le noeud (eventuellement remplace). Le code appelant
18  * doit donc reassigner : racine = optimize(racine);
19  *
20  * Garde-fou : on NE plie PAS une division/modulo qui echouerait (par zero,
21  * ou modulo sur flottants), afin de laisser l'evalateur emettre le message
22  * d'erreur attendu a l'execution.
23  */
24
25  #include <stdio.h>
26  #include <stdlib.h>
27  #include "ast.h"
28
29  Node *optimize(Node *node);
30
31  static double as_double(Value v)
32  {
33      return (v.type == VAL_INT) ? (double)v.ival : v.fval;
34  }

```

```

35
36 /* Peut-on plier ce BINOP a constantes sans provoquer d'erreur ? */
37 static int can_fold(int op, Value a, Value b)
38 {
39     int both_int = (a.type == VAL_INT && b.type == VAL_INT);
40
41     if (op == OP_DIV) {
42         return both_int ? (b.ival != 0) : (as_double(b) != 0.0);
43     }
44     if (op == OP_MOD) {
45         return both_int && (b.ival != 0);    /* modulo entiers uniquement */
46     }
47     return 1;
48 }
49
50 /* Calcule la valeur d'un BINOP a fils constants (sans verification : le
51 * code appelant a deja teste can_fold). Memes regles que l'evalateur. */
52 static Value do_fold(int op, Value a, Value b)
53 {
54     int both_int = (a.type == VAL_INT && b.type == VAL_INT);
55
56     switch (op) {
57     case OP_ADD: return both_int ? make_int(a.ival + b.ival)
58                               : make_float(as_double(a) + as_double(b));
59     case OP_SUB: return both_int ? make_int(a.ival - b.ival)
60                               : make_float(as_double(a) - as_double(b));
61     case OP_MUL: return both_int ? make_int(a.ival * b.ival)
62                               : make_float(as_double(a) * as_double(b));
63     case OP_DIV: return both_int ? make_int(a.ival / b.ival)
64                               : make_float(as_double(a) / as_double(b));
65     case OP_MOD: return make_int(a.ival % b.ival);
66     case OP_EQ:  return make_int(both_int ? (a.ival == b.ival)
67                               : (as_double(a) == as_double(b)));
68     case OP_NE:  return make_int(both_int ? (a.ival != b.ival)
69                               : (as_double(a) != as_double(b)));
70     case OP_LT:  return make_int(both_int ? (a.ival < b.ival)
71                               : (as_double(a) < as_double(b)));
72     case OP_GT:  return make_int(both_int ? (a.ival > b.ival)
73                               : (as_double(a) > as_double(b)));
74     case OP_LE:  return make_int(both_int ? (a.ival <= b.ival)
75                               : (as_double(a) <= as_double(b)));
76     case OP_GE:  return make_int(both_int ? (a.ival >= b.ival)
77                               : (as_double(a) >= as_double(b)));
78     default:     return make_int(0);    /* inatteignable */
79     }
80 }
81
82 Node *optimize(Node *node)
83 {
84     int i;
85
86     if (node == NULL) {

```

```

87     return NULL;
88 }
89
90 switch (node->type) {
91
92     case NODE_NUM:
93     case NODE_VAR:
94     case NODE_READ:
95     case NODE_BREAK:
96     case NODE_CONTINUE:
97         return node;    /* feuilles : rien a faire */
98
99     case NODE_BINOP: {
100         node->left  = optimize(node->left);
101         node->right = optimize(node->right);
102
103         if (node->left->type == NODE_NUM && node->right->type == NODE_NUM
104             && can_fold(node->op, node->left->num, node->right->num)) {
105             Value folded = do_fold(node->op, node->left->num, node->right->num);
106             Node *n = (folded.type == VAL_INT) ? new_num(folded.ival)
107                                                  : new_fnum(folded.fval);
108             n->line = node->line;
109             free_node(node);    /* libere aussi les deux NODE_NUM fils */
110             return n;
111         }
112         return node;
113     }
114
115     case NODE_UNOP:
116         node->left = optimize(node->left);
117         if (node->left->type == NODE_NUM) {
118             Value c = node->left->num;
119             Node *n = (c.type == VAL_INT) ? new_num(-c.ival)
120                                                  : new_fnum(-c.fval);
121             n->line = node->line;
122             free_node(node);
123             return n;
124         }
125         return node;
126
127     case NODE_ASSIGN:
128         node->left = optimize(node->left);
129         return node;
130
131     case NODE_IF:
132         node->cond      = optimize(node->cond);
133         node->body       = optimize(node->body);
134         node->elsebody   = optimize(node->elsebody);
135
136         if (node->cond->type == NODE_NUM) {
137             Node *keep;
138             if (!value_is_true(node->cond->num)) {

```

```

139         /* Condition toujours fausse : on garde le "sinon". */
140         keep = node->elsebody;
141         node->elsebody = NULL; /* detache pour ne pas le liberer */
142     } else {
143         /* Condition toujours vraie : on garde le "alors". */
144         keep = node->body;
145         node->body = NULL;
146     }
147     free_node(node); /* libere la condition et la branche morte */
148     return keep; /* peut etre NULL (if sans sinon supprime) */
149 }
150 return node;
151
152 case NODE_WHILE:
153     node->cond = optimize(node->cond);
154     node->body = optimize(node->body);
155     return node;
156
157 case NODE_FOR:
158     node->left = optimize(node->left);
159     node->cond = optimize(node->cond);
160     node->right = optimize(node->right);
161     node->body = optimize(node->body);
162     return node;
163
164 case NODE_FUNC_DECL:
165     node->body = optimize(node->body);
166     return node;
167
168 case NODE_FUNC_CALL:
169     for (i = 0; i < node->nargs; i++) {
170         node->args[i] = optimize(node->args[i]);
171     }
172     return node;
173
174 case NODE_PRINT:
175 case NODE_RETURN:
176     node->left = optimize(node->left);
177     return node;
178
179 case NODE_BLOCK: {
180     /* On optimise chaque instruction ; certaines (NODE_IF mort sans
181     * sinon) peuvent devenir NULL : on les retire alors du bloc. */
182     int w = 0; /* indice d'écriture (compactage) */
183     for (i = 0; i < node->nargs; i++) {
184         Node *opt = optimize(node->args[i]);
185         if (opt != NULL) {
186             node->args[w++] = opt;
187         }
188     }
189     node->nargs = w;
190     return node;

```

```

191     }
192
193     default:
194         return node;
195     }
196 }

```

A.9 codegen.c : back-end assembleur (extension)

```

1  /*
2  * codegen.c - Generation de code assembleur i386 (32 bits, syntaxe AT&T).
3  *
4  * =====
5  * AVERTISSEMENT : ce module est HORS PERIMETRE du cahier des charges.
6  * Il a ete ecrit par curiosite, comme prolongement du chapitre 2
7  * (assembleur i386) et de l'exercice ppcm : au lieu d'evaluer l'AST, on
8  * emet un fichier .s compilable avec 'gcc -m32'. Il ne gere QUE les
9  * entiers (les flottants de l'extension ne sont pas supportes ici) et
10 * ne reproduit pas la gestion "douce" de la division par zero de
11 * l'interpreteur (un 'idiv' par zero leve un SIGFPE, comme en C).
12 * =====
13 *
14 * Modele :
15 * - Convention d'appel cdecl 32 bits : arguments empiles de droite a
16 *   gauche par l'appelant, resultat dans %eax, %ebp/%esp pour le cadre
17 *   de pile (comme le %ebp du chapitre 2).
18 * - Variables du programme principal -> globales en .bss (gv_<nom>).
19 * - Parametres et variables locales d'une fonction -> sur la pile, en
20 *   deplacement par rapport a %ebp (params en +8/+12..., locales en
21 *   -4/-8...).
22 * - Les expressions sont evaluees facon "machine a pile" : le resultat
23 *   courant est dans %eax, les operandes intermediaires sont empiles.
24 *
25 * Usage : ./ilang -s programme.ilang > programme.s
26 *          gcc -m32 programme.s -o programme && ./programme
27 */
28
29 #include <stdio.h>
30 #include <stdlib.h>
31 #include <string.h>
32 #include "ast.h"
33
34 void generate_asm(Node *root);
35
36 /* ----- Compteur de labels uniques ----- */
37 static int lblctr = 0;
38 static int newlbl(void) { return lblctr++; }
39
40 /* ----- Ensemble des variables globales rencontrees ----- */
41 static char **gnames = NULL;
42 static int gcount = 0;

```

```

43
44 static void add_global(const char *name)
45 {
46     int i;
47     for (i = 0; i < gcount; i++) {
48         if (strcmp(gnames[i], name) == 0) {
49             return;
50         }
51     }
52     gnames = (char **)realloc(gnames, sizeof(char *) * (gcount + 1));
53     gnames[gcount++] = strdup(name);
54 }
55
56 /* ----- Table des locales de la fonction en cours ----- */
57 static char **lnames = NULL;
58 static int *loffs = NULL;
59 static int lcount = 0;
60
61 static void locals_reset(void)
62 {
63     int i;
64     for (i = 0; i < lcount; i++) {
65         free(lnames[i]);
66     }
67     free(lnames);
68     free(loffs);
69     lnames = NULL;
70     loffs = NULL;
71     lcount = 0;
72 }
73
74 static void local_add(const char *name, int off)
75 {
76     lnames = (char **)realloc(lnames, sizeof(char *) * (lcount + 1));
77     loffs = (int *) realloc(loffs, sizeof(int) * (lcount + 1));
78     lnames[lcount] = strdup(name);
79     loffs[lcount] = off;
80     lcount++;
81 }
82
83 /* Renvoie 1 et place le déplacement si name est une locale, 0 sinon. */
84 static int local_off(const char *name, int *off)
85 {
86     int i;
87     for (i = 0; i < lcount; i++) {
88         if (strcmp(lnames[i], name) == 0) {
89             *off = loffs[i];
90             return 1;
91         }
92     }
93     return 0;
94 }

```

```

95
96  /* ----- Pile des labels de boucle (pour arreter / continuer) ----- */
97  static int brk_lbl[64];    /* cible de "arreter" */
98  static int cont_lbl[64];   /* cible de "continuer" */
99  static int loop_sp = 0;
100
101  /* ----- Collecte des variables affectees (pour les offsets) ----- */
102  /* Ajoute a la table des locales tout nom affecte dans le sous-arbre, sans
103  * descendre dans une declaration de fonction imbriquee. negidx suit le
104  * prochain déplacement négatif disponible. */
105  static void collect_assigned(Node *n, int *negidx)
106  {
107      int i;
108      int dummy;
109
110      if (n == NULL || n->type == NODE_FUNC_DECL) {
111          return;
112      }
113
114      if (n->type == NODE_ASSIGN) {
115          if (!local_off(n->name, &dummy)) {
116              local_add(n->name, -4 * (++(*negidx)));
117          }
118      }
119
120      collect_assigned(n->left, negidx);
121      collect_assigned(n->right, negidx);
122      collect_assigned(n->cond, negidx);
123      collect_assigned(n->body, negidx);
124      collect_assigned(n->elsebody, negidx);
125      if (n->args != NULL) {
126          for (i = 0; i < n->nargs; i++) {
127              collect_assigned(n->args[i], negidx);
128          }
129      }
130  }
131
132  /* ----- Generation des expressions (resultat dans %eax) ----- */
133  static void gen_expr(FILE *out, Node *n);
134
135  static void gen_binop(FILE *out, Node *n)
136  {
137      gen_expr(out, n->left);
138      fprintf(out, "\tpushl %%eax\n");
139      gen_expr(out, n->right);
140      fprintf(out, "\tmovl %%eax, %%ecx\n");    /* ecx = operande droit */
141      fprintf(out, "\tpopl %%eax\n");          /* eax = operande gauche */
142
143      switch (n->op) {
144      case OP_ADD: fprintf(out, "\taddl %%ecx, %%eax\n"); break;
145      case OP_SUB: fprintf(out, "\tsubl %%ecx, %%eax\n"); break;
146      case OP_MUL: fprintf(out, "\timull %%ecx, %%eax\n"); break;

```

```

147     case OP_DIV: fprintf(out, "\tcltd\n\tdivl %%ecx\n"); break;          /* eax =
        quotient */
148     case OP_MOD: fprintf(out, "\tcltd\n\tdivl %%ecx\n\tmovl %%edx, %%eax\n");
        break;
149     case OP_EQ:  fprintf(out, "\tcmpl %%ecx, %%eax\n\tsete %%al\n\tmovzbl %%al,
        %%eax\n"); break;
150     case OP_NE:  fprintf(out, "\tcmpl %%ecx, %%eax\n\tsetne %%al\n\tmovzbl %%al,
        %%eax\n"); break;
151     case OP_LT:  fprintf(out, "\tcmpl %%ecx, %%eax\n\tsetl %%al\n\tmovzbl %%al,
        %%eax\n"); break;
152     case OP_GT:  fprintf(out, "\tcmpl %%ecx, %%eax\n\tsetg %%al\n\tmovzbl %%al,
        %%eax\n"); break;
153     case OP_LE:  fprintf(out, "\tcmpl %%ecx, %%eax\n\tsetle %%al\n\tmovzbl %%al,
        %%eax\n"); break;
154     case OP_GE:  fprintf(out, "\tcmpl %%ecx, %%eax\n\tsetge %%al\n\tmovzbl %%al,
        %%eax\n"); break;
155     default:
156         fprintf(stderr, "codegen: operateur binaire inconnu\n");
157         exit(1);
158 }
159 }
160
161 static void gen_call(FILE *out, Node *n)
162 {
163     int i;
164     /* Arguments empiles de droite a gauche (cdecl). */
165     for (i = n->nargs - 1; i >= 0; i--) {
166         gen_expr(out, n->args[i]);
167         fprintf(out, "\tpushl %%eax\n");
168     }
169     fprintf(out, "\tcall il_%s\n", n->name);
170     if (n->nargs > 0) {
171         fprintf(out, "\taddl $%d, %%esp\n", 4 * n->nargs);
172     }
173     /* resultat deja dans %eax */
174 }
175
176 static void gen_expr(FILE *out, Node *n)
177 {
178     int off;
179
180     switch (n->type) {
181
182     case NODE_NUM:
183         if (n->num.type == VAL_FLOAT) {
184             fprintf(stderr, "codegen: les flottants ne sont pas supportes en "
185                 "mode assembleur (entiers uniquement)\n");
186             exit(1);
187         }
188         fprintf(out, "\tmovl $%ld, %%eax\n", n->num.ival);
189         break;
190

```



```

191     case NODE_VAR:
192         if (local_off(n->name, &off)) {
193             fprintf(out, "\tmovl %d(%ebp), %%eax\n", off);
194         } else {
195             add_global(n->name);
196             fprintf(out, "\tmovl gv_%s, %%eax\n", n->name);
197         }
198         break;
199
200     case NODE_BINOP:
201         gen_binop(out, n);
202         break;
203
204     case NODE_UNOP:
205         gen_expr(out, n->left);
206         fprintf(out, "\tnegl %%eax\n");
207         break;
208
209     case NODE_FUNC_CALL:
210         gen_call(out, n);
211         break;
212
213     case NODE_READ:
214         fprintf(out, "\tpushl $gv__readtmp\n");
215         fprintf(out, "\tpushl $.LCin\n");
216         fprintf(out, "\tcall scanf\n");
217         fprintf(out, "\taddl $8, %%esp\n");
218         fprintf(out, "\tmovl gv__readtmp, %%eax\n");
219         break;
220
221     default:
222         fprintf(stderr, "codegen: expression non supportee (type %d)\n",
223             n->type);
224         exit(1);
225 }
226 }
227
228 /* ----- Generation des instructions ----- */
229 static void gen_stmt(FILE *out, Node *n);
230
231 static void store_var(FILE *out, const char *name)
232 {
233     int off;
234     if (local_off(name, &off)) {
235         fprintf(out, "\tmovl %%eax, %d(%ebp)\n", off);
236     } else {
237         add_global(name);
238         fprintf(out, "\tmovl %%eax, gv_%s\n", name);
239     }
240 }
241
242 static void gen_stmt(FILE *out, Node *n)

```

```

243 {
244     int i;
245
246     if (n == NULL) {
247         return;
248     }
249
250     switch (n->type) {
251
252     case NODE_BLOCK:
253         for (i = 0; i < n->nargs; i++) {
254             gen_stmt(out, n->args[i]);
255         }
256         break;
257
258     case NODE_ASSIGN:
259         gen_expr(out, n->left);
260         store_var(out, n->name);
261         break;
262
263     case NODE_PRINT:
264         gen_expr(out, n->left);
265         fprintf(out, "\tpushl %%eax\n");
266         fprintf(out, "\tpushl $.LCout\n");
267         fprintf(out, "\tcall printf\n");
268         fprintf(out, "\taddl $8, %%esp\n");
269         break;
270
271     case NODE_FUNC_CALL:
272         gen_call(out, n);    /* resultat ignore */
273         break;
274
275     case NODE_RETURN:
276         gen_expr(out, n->left);
277         fprintf(out, "\tleave\n\tret\n");
278         break;
279
280     case NODE_IF: {
281         int el = newlbl(), end = newlbl();
282         gen_expr(out, n->cond);
283         fprintf(out, "\tcmpl $0, %%eax\n\tjz .L%d\n", el);
284         gen_stmt(out, n->body);
285         fprintf(out, "\tjmp .L%d\n", end);
286         fprintf(out, ".L%d:\n", el);
287         gen_stmt(out, n->elsebody);
288         fprintf(out, ".L%d:\n", end);
289         break;
290     }
291
292     case NODE_WHILE: {
293         int start = newlbl(), end = newlbl();
294         fprintf(out, ".L%d:\n", start);

```

```

295     gen_expr(out, n->cond);
296     fprintf(out, "\tcmpl $0, %%eax\n\tjz .L%d\n", end);
297     cont_lbl[loop_sp] = start;
298     brk_lbl[loop_sp] = end;
299     loop_sp++;
300     gen_stmt(out, n->body);
301     loop_sp--;
302     fprintf(out, "\tjmp .L%d\n", start);
303     fprintf(out, ".L%d:\n", end);
304     break;
305 }
306
307 case NODE_FOR: {
308     int start = newlbl(), cont = newlbl(), end = newlbl();
309     gen_stmt(out, n->left);           /* initialisation */
310     fprintf(out, ".L%d:\n", start);
311     gen_expr(out, n->cond);
312     fprintf(out, "\tcmpl $0, %%eax\n\tjz .L%d\n", end);
313     cont_lbl[loop_sp] = cont;
314     brk_lbl[loop_sp] = end;
315     loop_sp++;
316     gen_stmt(out, n->body);
317     loop_sp--;
318     fprintf(out, ".L%d:\n", cont);
319     gen_stmt(out, n->right);          /* increment */
320     fprintf(out, "\tjmp .L%d\n", start);
321     fprintf(out, ".L%d:\n", end);
322     break;
323 }
324
325 case NODE_BREAK:
326     if (loop_sp == 0) {
327         fprintf(stderr, "codegen: 'arreter' hors d'une boucle\n");
328         exit(1);
329     }
330     fprintf(out, "\tjmp .L%d\n", brk_lbl[loop_sp - 1]);
331     break;
332
333 case NODE_CONTINUE:
334     if (loop_sp == 0) {
335         fprintf(stderr, "codegen: 'continuer' hors d'une boucle\n");
336         exit(1);
337     }
338     fprintf(out, "\tjmp .L%d\n", cont_lbl[loop_sp - 1]);
339     break;
340
341 case NODE_FUNC_DECL:
342     /* Les fonctions imbriquées ne sont pas gérées par le backend. */
343     break;
344
345 default:
346     fprintf(stderr, "codegen: instruction non supportée (type %d)\n",

```

```

347         n->type);
348     exit(1);
349 }
350 }
351
352 /* ----- Generation d'une fonction ----- */
353 static void gen_function(FILE *out, Node *decl)
354 {
355     int i;
356     int negidx = 0;
357     int framesize;
358
359     locals_reset();
360
361     /* Parametres : déplacements positifs +8, +12, ... */
362     for (i = 0; i < decl->nargs; i++) {
363         local_add(decl->args[i]->name, 8 + 4 * i);
364     }
365     /* Variables locales affectees : déplacements négatifs -4, -8, ... */
366     collect_assigned(decl->body, &negidx);
367
368     framesize = 4 * negidx;
369     framesize = (framesize + 15) & ~15;    /* arrondi a 16 (alignement) */
370
371     fprintf(out, "\n\t.globl il_%s\n", decl->name);
372     fprintf(out, "il_%s:\n", decl->name);
373     fprintf(out, "\tpushl %%ebp\n\tmovl %%esp, %%ebp\n");
374     if (framesize > 0) {
375         fprintf(out, "\tsubl $%d, %%esp\n", framesize);
376     }
377
378     gen_stmt(out, decl->body);
379
380     /* Retour par défaut (si la fonction ne fait pas de "retourner"). */
381     fprintf(out, "\tmovl $0, %%eax\n\tleave\n\tret\n");
382
383     locals_reset();
384 }
385
386 /* ----- Point d'entree du backend ----- */
387 void generate_asm(Node *root)
388 {
389     FILE *out = stdout;
390     int i;
391
392     if (root == NULL || root->type != NODE_BLOCK) {
393         return;
394     }
395
396     fprintf(out, "# Genere par ILANG (backend i386, hors perimetre du projet)\n");
397     fprintf(out, "\t.text\n");

```

```

398
399  /* 1) Les fonctions. */
400  for (i = 0; i < root->nargs; i++) {
401      if (root->args[i]->type == NODE_FUNC_DECL) {
402          gen_function(out, root->args[i]);
403      }
404  }
405
406  /* 2) Le programme principal -> fonction main (variables = globales). */
407  locals_reset(); /* aucune locale : tout est global au niveau principal */
408  fprintf(out, "\n\t.globl main\nmain:\n");
409  fprintf(out, "\tpushl %%ebp\n\tmovl %%esp, %%ebp\n");
410  fprintf(out, "\tandl $-16, %%esp\n"); /* alignement de pile (printf) */
411  for (i = 0; i < root->nargs; i++) {
412      if (root->args[i]->type != NODE_FUNC_DECL) {
413          gen_stmt(out, root->args[i]);
414      }
415  }
416  fprintf(out, "\tmovl $0, %%eax\n\tleave\n\tret\n");
417
418  /* 3) Donnees : formats + variables globales. */
419  fprintf(out, "\n\t.section .rodata\n");
420  fprintf(out, ".LCout:\n\t.string \"%%d\\n\\n\"");
421  fprintf(out, ".LCin:\n\t.string \"%%d\\n\\n\"");
422
423  fprintf(out, "\n\t.bss\n");
424  fprintf(out, "\t.align 4\ngv__readtmp:\n\t.zero 4\n");
425  for (i = 0; i < gcount; i++) {
426      fprintf(out, "\t.align 4\ngv_%%s:\n\t.zero 4\n", gnames[i]);
427  }
428 }

```

A.10 main.c : point d'entrée

```

1  /*
2   * main.c - Point d'entree de l'interpreteur ILANG.
3   *
4   * Chaîne de traitement (cf. cahier des charges, section 3.1) :
5   *
6   *   source ILANG -> [Flex] tokens -> [Bison] AST -> [optim] AST -> [eval]
7   *
8   * Usage :
9   *   ./ilang fichier.ilang      execute un fichier source
10  *   ./ilang                    mode interactif : lit le programme sur stdin
11  *   ./ilang -s fichier.ilang   genere l'assembleur i386 (extension hors-scope)
12  */
13
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17 #include "ast.h"

```

```

18 #include "syntab.h"
19
20 /* Fournis par Bison / Flex. */
21 extern int yyparse(void);
22 extern int yylex_destroy(void); /* libere le buffer interne de Flex */
23 extern FILE *yyin;
24 extern Node *program_root;
25
26 /* Fournis par eval.c, optim.c et codegen.c. */
27 void eval_program(Node *root);
28 Node *optimize(Node *node);
29 void generate_asm(Node *root); /* backend assembleur (hors perimetre) */
30
31 int main(int argc, char **argv)
32 {
33     int parse_status;
34     int asm_mode = 0; /* 1 = generer de l'assembleur (option -s) */
35     int no_opt = 0; /* 1 = desactiver les optimisations (--no-opt) */
36     const char *path = NULL; /* fichier source, NULL = entree standard */
37     int i;
38
39     /* Analyse des arguments : les options commencent par '-', le premier
40      * argument restant est le fichier source. */
41     for (i = 1; i < argc; i++) {
42         if (strcmp(argv[i], "-s") == 0) {
43             asm_mode = 1;
44         } else if (strcmp(argv[i], "--no-opt") == 0) {
45             no_opt = 1;
46         } else if (path == NULL) {
47             path = argv[i];
48         }
49     }
50
51     /* Choix de la source : fichier ou entree standard (mode interactif). */
52     if (path != NULL) {
53         yyin = fopen(path, "r");
54         if (yyin == NULL) {
55             fprintf(stderr, "Erreur : impossible d'ouvrir le fichier '%s'\n",
56                     path);
57             return 1;
58         }
59     } else {
60         if (!asm_mode) {
61             printf("ILANG - interpreteur (mode interactif)\n");
62             printf("Saisissez votre programme, puis Ctrl-D pour l'executer.\n");
63         }
64         yyin = stdin;
65     }
66
67     /* Initialisation de l'environnement global. */
68     env_init();
69

```

```

70  /* Analyse lexicale + syntaxique : construit l'AST dans program_root. */
71  parse_status = yyparse();
72
73  if (parse_status == 0 && program_root != NULL) {
74      /* Optimisations sur l'AST (pliage de constantes, code mort).
75       * Desactivables via --no-opt (utile pour mesurer leur effet). */
76      if (!no_opt) {
77          program_root = optimize(program_root);
78      }
79
80      if (asm_mode) {
81          /* Backend : on emet l'assembleur sur la sortie standard. */
82          generate_asm(program_root);
83      } else {
84          /* Evaluation. */
85          eval_program(program_root);
86      }
87  }
88
89  /* Nettoyage. */
90  if (program_root != NULL) {
91      free_node(program_root);
92  }
93  symtab_free();
94  if (yyin != NULL && yyin != stdin) {
95      fclose(yyin);
96  }
97  yylex_destroy();
98
99  return parse_status;
100 }

```

A.11 Makefile

```

1  # Makefile - Interpréteur ILANG
2  #
3  # Cibles :
4  #   make          compile l'interpreteur (binaire ./ilang)
5  #   make clean    supprime les fichiers generes
6  #   make test     compile puis execute les programmes de demonstration
7  #
8  # Les fichiers lex.yy.c (Flex) et ilang.tab.c/.h (Bison) sont generes
9  # automatiquement a partir de ilang.l et ilang.y.
10
11 CC          = gcc
12 CFLAGS      = -Wall -Wextra -std=gnu11 -g
13 LEX         = flex
14 YACC        = bison
15
16 # Code que nous ecrivons : compile avec tous les avertissements actives.
17 SRC         = ast.c symtab.c eval.c optim.c codegen.c main.c

```

```

18 OBJ      = $(SRC:.c=.o)
19
20 # Code genere (Flex/Bison) : compile sans -Wextra pour eviter le bruit des
21 # avertissements propres au code machine-genere (zero warning de notre cote).
22 GENOBJ    = ilang.tab.o lex.yy.o
23 GENFLAGS  = -std=gnu11 -g
24
25 BIN       = ilang
26
27 .PHONY: all clean test asm
28
29 all: $(BIN)
30
31 $(BIN): $(OBJ) $(GENOBJ)
32         $(CC) $(CFLAGS) -o $@ $(OBJ) $(GENOBJ)
33
34 # --- Bison : grammaire -> parseur + en-tete des tokens ---
35 ilang.tab.c ilang.tab.h: ilang.y
36         $(YACC) -d -o ilang.tab.c ilang.y
37
38 # --- Flex : regles -> analyseur lexical (depend de l'en-tete des tokens) ---
39 lex.yy.c: ilang.l ilang.tab.h
40         $(LEX) -o lex.yy.c ilang.l
41
42 # --- Compilation du code genere ---
43 ilang.tab.o: ilang.tab.c ast.h
44         $(CC) $(GENFLAGS) -c ilang.tab.c -o ilang.tab.o
45
46 lex.yy.o: lex.yy.c ilang.tab.h ast.h
47         $(CC) $(GENFLAGS) -c lex.yy.c -o lex.yy.o
48
49 # --- Compilation de notre code ---
50 ast.o:      ast.c ast.h
51 symtab.o:   symtab.c symtab.h ast.h
52 eval.o:     eval.c ast.h symtab.h
53 optim.o:    optim.c ast.h
54 codegen.o:  codegen.c ast.h
55 main.o:     main.c ast.h symtab.h
56
57 %.o: %.c
58         $(CC) $(CFLAGS) -c $< -o $@
59
60 test: $(BIN)
61         @echo "=== Fibonacci ==="
62         ./$(BIN) programmes/fibonacci.ilang
63         @echo "=== PGCD ==="
64         ./$(BIN) programmes/pgcd.ilang
65         @echo "=== Interactif (entree = 7) ==="
66         @echo 7 | ./$(BIN) programmes/interactif.ilang
67         @echo "=== Flottants ==="
68         ./$(BIN) programmes/flottants.ilang
69         @echo "=== Boucles (arreter / continuer) ==="

```



```

70     ./$(BIN) programmes/boucles.ilang
71
72 # Demonstration du backend assembleur (extension hors-perimetre, entiers).
73 # Necessite gcc-multilib (paquet pour compiler en 32 bits : gcc -m32).
74 asm: $(BIN)
75     @echo "=== Generation assembleur : pgcd.ilang -> pgcd.s ==="
76     ./$(BIN) -s programmes/pgcd.ilang > pgcd.s
77     gcc -m32 -no-pie pgcd.s -o pgcd_asm
78     @echo "=== Execution du binaire assemble ==="
79     ./pgcd_asm
80
81 clean:
82     rm -f $(OBJ) $(GENOBJ) $(BIN) ilang.tab.c ilang.tab.h lex.yy.c pgcd.s
        pgcd_asm

```

A.12 Programmes de démonstration

A.12.1 fibonacci.ilang

```

1  // Suite de Fibonacci, version recursive.
2  // Affiche fibonacci(0) a fibonacci(14).
3
4  fonction fibonacci(n) {
5      si (n < 2) {
6          retourner(n)
7      }
8      retourner(fibonacci(n - 1) + fibonacci(n - 2))
9  }
10
11 pour (i = 0; i < 15; i = i + 1) {
12     afficher(fibonacci(i))
13 }

```

A.12.2 pgcd.ilang

```

1  // PGCD par l'algorithme d'Euclide (version iterative).
2
3  fonction pgcd(a, b) {
4      tant que (b != 0) {
5          temp = b
6          b = a % b
7          a = temp
8      }
9      retourner(a)
10 }
11
12 afficher(pgcd(48, 18))

```

A.12.3 interactif.ilang

```

1 // Programme interactif : lit un entier et affiche son carre s'il est
2 // positif, sinon affiche son oppose.
3
4 afficher(999) // prompt (valeur sentinelle avant la saisie)
5 x = lire()
6 si (x > 0) {
7     afficher(x * x)
8 } sinon {
9     afficher(0 - x)
10 }

```

A.12.4 flottants.ilang (extension)

```

1 // Demonstration des nombres flottants.
2
3 afficher(3.14 * 2) // 6.28 (flottant)
4 afficher(2.5 + 1) // 3.5 (flottant + entier -> flottant)
5 afficher(10 / 4) // 2 (division ENTIERE, comportement d'origine)
6 afficher(10.0 / 4) // 2.5 (au moins un flottant -> division reelle)
7
8 // Aire d'un disque de rayon 2.5
9 rayon = 2.5
10 afficher(3.14159 * rayon * rayon) // 19.6349
11
12 // Comparaison mixte entier / flottant
13 si (3.5 > 3) {
14     afficher(1)
15 } sinon {
16     afficher(0)
17 }

```

A.12.5 boucles.ilang (extension)

```

1 // Demonstration de "arreter" (break) et "continuer" (continue).
2
3 // arreter : on quitte la boucle des qu'on atteint 5
4 // -> affiche 0 1 2 3 4
5 pour (i = 0; i < 100; i = i + 1) {
6     si (i == 5) {
7         arreter
8     }
9     afficher(i)
10 }
11
12 // continuer : on saute les nombres pairs
13 // -> affiche 1 3 5 7 9
14 pour (j = 0; j < 10; j = j + 1) {
15     si (j % 2 == 0) {
16         continuer
17     }
18     afficher(j)

```

```
19 }
20
21 // arreter dans une boucle "tant que"
22 // -> affiche 10 9 8 (puis arret a 7)
23 n = 10
24 tant que (n > 0) {
25     si (n == 7) {
26         arrêter
27     }
28     afficher(n)
29     n = n - 1
30 }
```

Annexe B

Manuel d'installation

B.1 Prérequis

Une distribution GNU/Linux (ou WSL sous Windows) disposant de la chaîne C, de Flex et de Bison. Sur une base Debian/Ubuntu :

```
1 sudo apt update
2 sudo apt install build-essential bison flex make
3 # Optionnel, pour l'extension assembleur 32 bits :
4 sudo apt install gcc-multilib
```

B.2 Compilation

Depuis le répertoire du projet :

```
1 make          # compile l'interpreteur -> binaire ./ilang
2 make clean    # supprime les fichiers generes
3 make test     # compile puis lance tous les programmes de demonstration
4 make asm      # demonstration du back-end assembleur (extension)
```

La compilation ne doit produire aucun avertissement ni conflit.

Annexe C

Guide utilisateur

C.1 Invocation

```
1 ./ilang programme.ilang      # execute un fichier source
2 ./ilang                      # mode interactif (Ctrl-D pour executer)
3 echo 7 | ./ilang interactif.ilang  # alimenter lire() via un tube
4 ./ilang -s programme.ilang      # genere l'assembleur i386 (extension)
5 ./ilang --no-opt programme.ilang  # desactive les optimisations
```

C.2 Aide-mémoire du langage

Construction	Syntaxe
Affectation	<code>x = 42</code>
Flottant	<code>r = 2.5</code>
Arithmétique	<code>+ - * / %</code> , <code>-x</code> , parenthèses
Comparaisons	<code>== != < > <= >=</code>
Condition	<code>si (c) { ... } sinon { ... }</code>
Boucle tant que	<code>tant que (c) { ... }</code>
Boucle pour	<code>pour (i=0; i<n; i=i+1) { ... }</code>
Rupture de boucle	<code>arrêter</code> , <code>continuer</code>
Fonction	<code>fonction f(a,b) { ... retourner(e) }</code>
Affichage / saisie	<code>afficher(e)</code> , <code>lire()</code>
Commentaires	<code>// ligne</code> , <code>/* bloc */</code>

Annexe D

Données expérimentales

Les mesures ont été obtenues sur AMD Ryzen 5 5500, sous Ubuntu (WSL), meilleur de trois exécutions. Elles sont reproductibles via les scripts de mesure fournis.

TABLE D.1 – Données brutes des benchmarks.

Expérience	Paramètre	Mesure
Fibonacci (interpréteur)	$n = 28$	0,43 s
Fibonacci (interpréteur)	$n = 30$	1,13 s
Fibonacci (interpréteur)	$n = 32$	2,97 s
Fibonacci (asm i386)	$n = 32$	0,01 s
Boucle 30 M (avec optim.)	pliage actif	4,05 s
Boucle 30 M (sans optim.)	<code>-no-opt</code>	6,02 s
Récursion <code>somme</code>	$n = 5000$	ok
Récursion <code>somme</code>	$n = 10000$	débordement de pile

Annexe E

Assembleur i386 généré (exemples)

Cette annexe reproduit le code assembleur réellement produit par le back-end (`./ilang -s`) pour deux programmes. Ce code est compilable tel quel avec `gcc -m32 -no-pie` et produit, à l'exécution, exactement les mêmes résultats que l'interpréteur (respectivement 6 et 120).

E.1 PGCD (boucle tant que)

Source : voir `pgcd.ilang` en annexe A. Le code généré illustre la traduction d'une boucle en labels et sauts conditionnels.

```
1  # Genere par ILANG (backend i386, hors perimetre du projet)
2      .text
3
4      .globl il_pgcd
5  il_pgcd:
6      pushl %ebp
7      movl %esp, %ebp
8      subl $16, %esp
9  .L0:
10     movl 12(%ebp), %eax
11     pushl %eax
12     movl $0, %eax
13     movl %eax, %ecx
14     popl %eax
15     cmpl %ecx, %eax
16     setne %al
17     movzbl %al, %eax
18     cmpl $0, %eax
19     jz .L1
20     movl 12(%ebp), %eax
21     movl %eax, -4(%ebp)
22     movl 8(%ebp), %eax
23     pushl %eax
24     movl 12(%ebp), %eax
25     movl %eax, %ecx
26     popl %eax
```

```

27     cltd
28     idivl %ecx
29     movl %edx, %eax
30     movl %eax, 12(%ebp)
31     movl -4(%ebp), %eax
32     movl %eax, 8(%ebp)
33     jmp .L0
34 .L1:
35     movl 8(%ebp), %eax
36     leave
37     ret
38     movl $0, %eax
39     leave
40     ret
41
42     .globl main
43 main:
44     pushl %ebp
45     movl %esp, %ebp
46     andl $-16, %esp
47     movl $18, %eax
48     pushl %eax
49     movl $48, %eax
50     pushl %eax
51     call il_pgcd
52     addl $8, %esp
53     pushl %eax
54     pushl $.LCout
55     call printf
56     addl $8, %esp
57     movl $0, %eax
58     leave
59     ret
60
61     .section .rodata
62 .LCout:
63     .string "%d\n"
64 .LCin:
65     .string "%d"
66
67     .bss
68     .align 4
69 gv__readtmp:
70     .zero 4

```

E.2 Factorielle (fonction récursive)

Pour `factorielle(5)`, le code généré illustre la récursion : la fonction `il_factorielle` s'appelle elle-même, chaque appel disposant de son propre cadre de pile.

```

1  # Genere par ILANG (backend i386, hors perimetre du projet)

```



```
2      .text
3
4      .globl il_factorielle
5 il_factorielle:
6      pushl %ebp
7      movl %esp, %ebp
8      movl 8(%ebp), %eax
9      pushl %eax
10     movl $1, %eax
11     movl %eax, %ecx
12     popl %eax
13     cmpl %ecx, %eax
14     setle %al
15     movzbl %al, %eax
16     cmpl $0, %eax
17     jz .L0
18     movl $1, %eax
19     leave
20     ret
21     jmp .L1
22 .L0:
23 .L1:
24     movl 8(%ebp), %eax
25     pushl %eax
26     movl 8(%ebp), %eax
27     pushl %eax
28     movl $1, %eax
29     movl %eax, %ecx
30     popl %eax
31     subl %ecx, %eax
32     pushl %eax
33     call il_factorielle
34     addl $4, %esp
35     movl %eax, %ecx
36     popl %eax
37     imull %ecx, %eax
38     leave
39     ret
40     movl $0, %eax
41     leave
42     ret
43
44     .globl main
45 main:
46     pushl %ebp
47     movl %esp, %ebp
48     andl $-16, %esp
49     movl $5, %eax
50     pushl %eax
51     call il_factorielle
52     addl $4, %esp
53     pushl %eax
```

```
54     pushl $.LCout
55     call printf
56     addl $8, %esp
57     movl $0, %eax
58     leave
59     ret
60
61     .section .rodata
62 .LCout:
63     .string "%d\n"
64 .LCin:
65     .string "%d"
66
67     .bss
68     .align 4
69 gv__readtmp:
70     .zero 4
```

Annexe F

Contenu de l'archive

Le code source est livré dans l'archive IED-L3-IC-CHAPITRE_10-ILIAS_AIT_BENAISSA-82508880-sources.zip, dont l'arborescence est la suivante :

```
1  ilang/
2  |-- ilang.l          analyseur lexical (Flex)
3  |-- ilang.y          grammaire et construction de l'AST (Bison)
4  |-- ast.h  ast.c     definition et manipulation de l'AST, type Value
5  |-- symtab.h  symtab.c table des symboles (pile de cadres + hachage)
6  |-- eval.c       evaluateur recursif
7  |-- optim.c      optimisations (pliage de constantes, code mort)
8  |-- codegen.c    back-end assembleur i386 (extension hors-perimetre)
9  |-- main.c       point d'entree (fichier, interactif, -s, --no-opt)
10 |-- Makefile      compilation automatisee
11 |-- README.md     instructions de compilation et d'utilisation
12 |-- JOURNAL.md    journal de bord (difficultes, choix de conception)
13 |-- programmes/   programmes de demonstration
14 |   |-- fibonacci.ilang
15 |   |-- pgcd.ilang
16 |   |-- interactif.ilang
17 |   |-- flottants.ilang      (extension : flottants)
18 |   |-- boucles.ilang       (extension : arreter / continuer)
19 |-- tests/          cas de test
20   |-- err_variable.ilang   err_divzero.ilang   err_fonction.ilang
21   |-- err_arguments.ilang  err_lexical.ilang   err_syntaxe.ilang
22   |-- optim.ilang         run_errors.sh
```

Remarque sur les fichiers générés. L'archive ne contient volontairement *pas* les fichiers produits automatiquement lors de la compilation, à savoir `lex.yy.c` (engendré par Flex à partir de `ilang.l`), ainsi que `ilang.tab.c` et `ilang.tab.h` (engendrés par Bison à partir de `ilang.y`). Ces fichiers sont entièrement reconstruits par la commande `make` et n'ont donc pas à figurer dans les sources. De même, les fichiers objets `.o` et le binaire `ilang` ne sont pas livrés, puisqu'ils résultent de la compilation.

Récapitulatif des sorties. Le tableau F.1 rassemble, pour mémoire, les sorties attendues des programmes de démonstration, déjà détaillées au chapitre 5.

TABLE F.1 – Récapitulatif des sorties des programmes de démonstration.

Programme	Entrée	Sortie
fibonacci.ilang	aucune	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
pgcd.ilang	aucune	6
interactif.ilang	7	999 puis 49
interactif.ilang	-5	999 puis 5
flottants.ilang	aucune	6.28 / 3.5 / 2 / 2.5 / 19.6349 / 1
boucles.ilang	aucune	0 1 2 3 4 / 1 3 5 7 9 / 10 9 8

Annexe G

Glossaire

AST

Abstract Syntax Tree, arbre syntaxique abstrait : représentation arborescente du programme, débarrassée des détails de syntaxe concrète.

BNF

Backus-Naur Form, notation pour décrire formellement une grammaire hors-contexte.

cdecl

convention d'appel de fonction sur architecture x86 (arguments empilés de droite à gauche, nettoyage par l'appelant).

LALR(1)

Look-Ahead LR, classe de grammaires analysables de manière ascendante avec un symbole de prévision ; celle qu'accepte Bison.

Lexème / token

unité lexicale élémentaire reconnue par l'analyseur lexical (nombre, identifiant, mot-clé, opérateur).

PIE

Position-Independent Executable, format d'exécutable par défaut sur les systèmes modernes, incompatible avec l'adressage absolu utilisé par le back-end (d'où l'option `-no-pie`).

Pliage de constantes

optimisation calculant à la compilation les sous-expressions entièrement constantes.

Portée lexicale

règle de visibilité des variables fondée sur la structure du programme (et non sur la chaîne dynamique des appels).

Shift/reduce

type de conflit qu'un analyseur LR peut rencontrer lorsqu'il hésite entre décaler un symbole et réduire une règle.